

Supplementary Material

This document is the **Supplementary Material of the manuscript**, submitted together with it.

Numbering convention: the main text uses §n for sections and S n for subsections of this document.

0. Generation and verifiability (**read this section first**)

This document **was not written by hand**: the body text of every section **embeds verbatim** the content of the following **verified source files**,

so "the numbers in this document" and "the numbers in the source files" **cannot be inconsistent**. Every source file records an md5 and a byte count, so anyone can re-check it.

Subsection	Content	Source file	md5	Bytes
---	---	---	---	---
S1	S1 · Per-row marking table and mechanical recomputation over 19 public benchmarks × 4 counting units	split_units_19rows_20260916.md	2d542d10226347dffffb69dbab8b61d1a	5805
S2	S2 · Correction record for the counting convention (old convention → recomputed convention)	the audit-correction and adjudication record, §2 superseding block	94e85a41a6f7c5a5b6788a11b96c2c8e	24638
S3	S3 · Per-row evidence from the cross-framework configuration audit (5 frameworks / 25 official files)	xframe_evidence_20260916.txt	78b3e251f5d4c8db3b6378ecbbd2b3e0	13361
S4	S4 · 2×2 evaluation matrix over the three-split protocol (41 runs × {best,last} × {val,test})	xeval_analysis_20260916.txt	8b74b2eb6228c078d5a38131d119567b	8534

S5	S5 · Selection premium on public third-party logs (sample and how to read it)	selection_premium_public_20260916.md	04950c92d1ab4bcfb5bbe616b2dcbec4	9479
----	---	--------------------------------------	----------------------------------	------

Generation script: work\gen_supplementary_20260916.py (purely local and re-runnable; S5 reads the collaborating archive in read-only mode).

Re-check entry point: see §S7 of this document.

Terms this paper uses (one place, so they need not be reconstructed)

term	what it means here
---	---
arm	one member of a paired comparison. The baseline arm follows the published convention; the strategy arm is the intervention under test (e.g. a learning-rate change). "Baseline" does not mean unmodified software — both arms run the same code
cell	one (corpus pair × arm) unit of the design. A cell name such as shwd2sf reads source → target (SHWD → SFCHD); the corpora are dota15, aitod, visdrone, smoke, shwd, sfchd, mende, mask
<code>`best` / `last`</code>	the two checkpoints compared throughout: <code>best.pt</code> , written by the training script by fitness on the monitored split, and the final epoch
selection premium	best – last within a run : what selecting the best checkpoint adds, measured on the split the numbers are reported from
Δgap	the paired difference of the premium between the two arms , per corpus. Positive means val understates the strategy arm's effect (§5.1, Tier 2)
realization rate	<code>prem_test / prem_val</code> , aggregated over cells (§8.6); the share of the premium selected on val that survives on a disjoint test
κ	through-origin slope of ΔT on ΔV over a run's epochs (§8.7): how much of a run's movement on val appears on test
same directory / alias	a dataset config whose val: and test: point at the same path, so the checkpoint is selected on the split it is scored on
<code>`shapeiou`</code>	the shape-aware IoU variant used in the registered replication's strategy arm (§9)
<code>`sio_b_results.csv`</code>	the per-run results table of the clean-protocol batch; its unwritten endpoint convention is disclosed as a defect in §10
FROZEN-HASH	the registration's self-referential stamp: the md5 of the registration text above the stamp line (§9, verifiable by re-hashing that text)

S1 · Per-row marking table and mechanical recomputation over 19 public benchmarks × 4 counting units

Purpose: main text §3, §6; this is the **complete per-row evidence** for the claim "how many benchmarks are problematic".

Source: split_units_19rows_20260916.md (md5 2d542d10226347dffffb69dbab8b61d1a, 5805 bytes); the content below is **embedded verbatim, unmodified**.

19-row × 5-column unit marking table (front matter of the companion paper)

> Generated by work/recompute_split_units_20260916.py; all percentages are mechanically recomputed from the markings in this table, with nothing hard-coded.

> Established 2026-09-16. The (h-1) ruling has been applied (D-Fire counts as clean).

The three units (the main text must state which one it uses)

Unit	Definition
release	The benchmark's own release layer: whether the official split artifacts/scripts provide an independent held-out test
protocol	The official protocol layer: the split declared by the official source, and whether that test can be self-evaluated locally
yolo_dist	Generic YOLO distribution-package layer (= the column measured in the §1 table of the audit, i.e. column 4)
reported	The actually-reported layer: which split the numbers reported in the literature come from, and whether they are independent of checkpoint selection

Evidence marks: (A) explicit in the audit | (D) derived from audit facts | (?) the audit takes no position

Marking table

#	benchmark	release	protocol	yolo_dist	reported	Evidence category	Confidence
1	COCO	n_a(A)	independent_test(A)	clean(A)	clean(A)	Official counts + mirror yaml	high
2	PASCAL VOC	n_a(A)	test_gated(A)	alias(A)	alias(A)	mirror VOC.yaml (both keys cited per row)	high
3	Objects365	no_test(A)	no_test(A)	no_test_key(A)	alias(D)		

mirror yaml + Ultralytics docs | high |
 | 4 | Open Images v7 | independent_test(A) | independent_test(A) |
 no_test_key(A) | alias(D) | Official facts page + mirror yaml | high |
 | 5 | DOTA v1.0 | independent_test(A) | independent_test(A) | clean(A) |
 clean(A) | Official download page + mirror DOTAv1.yaml | high |
 | 6 | DOTA v2.0 | independent_test(A) | test_gated(A) | clean(A) | alias(D) |
 Official/mirror documentation | high |
 | 7 | VisDrone-DET | independent_test(A) | independent_test(A) | clean(A) |
 clean(A) | mirror + converter | high |
 | 8 | AI-TOD | contradictory(A) | contradictory(A) | unknown(A) | unknown(?) |
 Official README (contains two mutually contradictory release statements)
 | low |
 | 9 | UAVDT | no_val(A) | no_val(A) | unknown(A) | alias(A) | Official paper
 + two mirrors (the audit calls these unverified beyond these two) |
 medium |
 | 10 | xView | no_test(A) | no_test(A) | no_test_key(A) | alias(A) | mirror
 xView.yaml | low |
 | 11 | DIOR | independent_test(A) | independent_test(A) | no_test_key(A) |
 unknown(?) | Official description (no counting) + mirror DIOR-R yaml |
 low |
 | 12 | NWPU VHR-10 | no_split(A) | no_split(A) | unknown(A) | alias(A) |
 Official TorchGeo (the split means positive/negative) | high |
 | 13 | SHWD | no_yaml(A) | no_val(A) | train_val_alias(A) | alias(A) |
 Official README + mirror Reflective_vests.yaml | medium |
 | 14 | SFCHD | alias(A) | no_val(A) | alias(A) | alias(A) | Official first-
 party release (machine comparison _ev_sfchd_check2.py) | highest |
 | 15 | MAFA | no_val(A) | no_val(A) | unknown(?) | alias(A) | Official CVPR
 PDF (extracted with pypdf) | high |
 | 16 | Mendeley face-mask | no_split(A) | no_split(A) | alias(D) | unknown(?) |
 Official data record + mirror generator | lowest |
 | 17 | WIDER FACE | independent_test(A) | test_gated(A) | no_test_key(A) |
 alias(A) | Official site (GT not released, verified word by word) +
 mirror | high |
 | 18 | CrowdHuman | test_gated(A) | alias(A) | no_test_key(A) | alias(A) |
 Official paper + mirror | high |
 | 19 | D-Fire (FireSmoke) | independent_test(A) | independent_test(A) |
 clean(A) | unknown(?) | Official README | medium |

Recomputation results

Unit	alias/non-independent	Among decidable rows	unknown
release (the benchmark's own release layer)	**10/19 = 52.6%**	10/19 =	

52.6% | 0 |
 | protocol (official protocol layer) | **13/19 = 68.4%** | 13/19 = 68.4% |
 0 |
 | yolo (generic YOLO distribution-package layer (yolo_dist)) | **4/19 = 21.1%** | 4/15 = 26.7% | 4 |
 | reported (actually-reported layer) | **12/19 = 63.2%** | 12/15 = 80.0% |
 4 |

Cross-check against the numbers already published in the audit

- Audit 3a "literal val == test **2/19**" → this table's "the two keys of the distribution-package config point to the same path" (A evidence source) yields **2/19, reproduced ✓**
 - The other 1 alias row is produced by a **generator** (Mendeley face-mask), not by a distribution-package config, and is not counted under the 3a convention
- Audit "**14/19**" → this table's reported yields **12/19** (difference -2)
- Audit "**5/19 clean**" → this table's reported=clean yields **3/19** (difference -2)
- The audit's "12/19" was obtained by manually adjusting two rows and is **not** a recomputation under a consistent unit; this table recomputes by a consistent rule, so the values need not be identical

A self-consistent set of numbers for the main text to adopt

- Literal val == test (generic distribution-package config, A evidence source): **2/19 = 11%** – PASCAL VOC, SFCHD
- train/val from the same source, produced by the generator: **1/19** (Mendeley face-mask), **a different convention, which must be stated separately**
- Non-independent at the official protocol layer: **13/19 = 68%**
- Non-independent at the actually-reported layer: **12/19 = 63%** (4 unknown rows) – **13/19** if a
 - paper's own val numbers, rather than community test-dev numbers, are what the unit judges
 - for COCO (the main text states both readings)
- Clean at the actually-reported layer: **3/19 = 16%**

The four levels of "same directory" (resolving (g0))

- **L1** literal val==test in a generic distribution package: 2 rows – PASCAL VOC, SFCHD
- **L2** distribution package with train==val (F-C): 1 row – SHWD

- **L3** distribution package clean but reported layer non-independent: 9 rows – Objects365, Open Images v7, DOTA v2.0, UAVDT, xView, NWPU VHR-10, MAFA, WIDER FACE, CrowdHuman
- **L4** all the rest: 7 rows

Corrections already applied

- **(h-1) D-Fire / FireSmoke family:** the "mixed" criterion for the family row is taken from a config of SHWD lineage (A:59 cites Reflective_vests.yaml; A:52 notes that it is based on SHWD). All three copies belonging to FireSmoke/D-Fire are distinct. Under the "release layer" unit explicitly adopted in 3a, D-Fire goes into the clean group.
- **(d-1) AI-TOD / DOTA asymmetry:** the audit marks F-C for AI-TOD, which contains trainval, but does not mark DOTA v1/v2, which likewise contain trainval. This table uniformly adopts the "release/protocol layer" unit: trainval is a user-protocol caveat and is not used as grounds for a verdict; neither place is re-judged, but AI-TOD is contradictory because its v2 release statement contradicts itself.

S2 · Correction record for the counting conventions (old convention → recomputed convention)

Purpose: main text §6.3; explains why the same fact once had two sets of numbers, and which set has been invalidated.

Source: the audit-correction and adjudication record, §2 superseding block (md5 94e85a41a6f7c5a5b6788a11b96c2c8e, 24638 bytes); the following content is **embedded verbatim, unmodified**.

```
> ### The four counts in this section (§2) have been superseded by a
mechanical recomputation (note added 2026-09-16)
>
> The 12/19 (release/protocol layer), 13/19 (actual reporting layer),
6/19 (clean) and ≤14/19 (upper bound) in the §2 table
> are inconsistent with the mechanical recomputation completed 4
minutes later, and must no longer be cited. The recomputation is the
sole counting source: analysis/eval_validity/split_units_recompute_20260916.txt
and
> split_units_19rows_20260916.md
>
> | Unit | Non-independent | Clean | Undecidable |
> |---|---|---|---|
> | Literal val == test (the benchmark's own release) | 2/19 ≈ 11% | – |
```

– |

> | release (release layer) | **10/19 ≈ 53%** | – | – |

> | protocol (official protocol layer) | **13/19 ≈ 68%** | – | – |

> | yolo_dist (generic YOLO distribution package layer) | **4/19 ≈ 21%** |

– | 4 |

> | **reported (actual reporting layer)** | **12/19 ≈ 63%** (12/15 = **80%** of the decidable rows) | **3/19 ≈ 16%** | 4 |

>

> The four "undecidable" rows are AI-TOD, DIOR, Mendeley and D-Fire – **the audit expressed no position on their reporting practice**; it does not mean they are clean.

>

> The four concrete discrepancies and their dispositions:

> 1. §2's "release/protocol layer 12/19" does not hold under the recomputed convention – the recomputed release is 10/19 and protocol is 13/19,

> **12 is neither the one nor the other** (consistent with the ruling in §1(g): it is the result of two rows being hand-adjusted).

> 2. §2's "actual reporting layer 13/19" should be **12/19** (4 rows undecidable).

> 3. §2's "clean 6/19" should be **3/19**; the reason the clean group dropped from 5 to 3 is that those 4 rows **were never addressed by the audit** – this is a finding, not a gap.

> 4. §2's "≤14/19 upper bound" is replaced by a derivable statement: **80% of the decidable rows are non-independent (12/15)**, which is both stronger and more defensible than an unenumerated upper bound.

>

> **"79%" has already been ruled inadmissible in §1(e)**; this is confirmed here as well: the main text uses only the four enumerated points in the table above.

> **⚠ This section was revised on 2026-09-16 (second version). The first version (09:56) was written before the 19×5 marking table was built and gave predicted values**

> (release/protocol layer 12/19, actual reporting layer 13/19, clean 6/19). The mechanical recomputation 4 minutes later

> (eval_validity\split_units_recompute_20260916.txt, 10:00, with a per-unit 19-row subtotal check) gave different results.

> **The two sets of numbers cannot be cited side by side.** Following the disposition recommendation in §7 #0 of the asset inventory,

> **the recomputation is taken as the sole counting source**; the first version's predicted values are retained in the "trail" table below for traceability.

S3 · Line-by-line evidence for the cross-framework configuration audit (5 frameworks / 25 official files)

Purpose: main text §6.5; the commit pin and the line-by-line evidence for each framework (the judgements are in the main text, the evidence is here).

Source: xframe_evidence_20260916.txt (md5 78b3e251f5d4c8db3b6378ecbbd2b3e0, 13361 bytes); the following content is **embedded verbatim, unmodified**.

```
=====
=====
```

Item C: extraction of cross-framework "split and checkpoint-selection" evidence (mechanical extraction, with line numbers; no judgements in this file)

```
=====
=====
```

```
repo      default_branch  commit_sha
mmdetection  open-mmlab/mmdetection  main
           cfd5d3a985b0249de009b67d04f37263e11cdf3d
detectron2   facebookresearch/detectron2  main
           a2f4a8771ab77e8411c26b27f24f9489a28a2453
yolox       Megvii-BaseDetection/YOLOX    main
           6ddff4824372896469a7fae2dc3206c7aa4bbaee
paddledetection PaddlePaddle/PaddleDetection  release/2.9
           b25522a0f4bde8c80603f3ba5e3472059972e3b5
detr         facebookresearch/detr    main
           29901c51d7fe8712168b8d0d64351170bc0f83e0
```

```
-----
-----
```

MMDetection ::

mmdetection/configs__base__datasets__coco_detection.py (96 lines)

```
3: data_root = 'data/coco/'
21: dict(type='LoadImageFromFile', backend_args=backend_args),
22: dict(type='LoadAnnotations', with_bbox=True),
23: dict(type='Resize', scale=(1333, 800), keep_ratio=True),
24: dict(type='RandomFlip', prob=0.5),
25: dict(type='PackDetInputs')
28: dict(type='LoadImageFromFile', backend_args=backend_args),
29: dict(type='Resize', scale=(1333, 800), keep_ratio=True),
31: dict(type='LoadAnnotations', with_bbox=True),
```

```

33:         type='PackDetInputs',
41:     sampler=dict(type='DefaultSampler', shuffle=True),
42:     batch_sampler=dict(type='AspectRatioBatchSampler'),
45:     data_root=data_root,
46:     ann_file='annotations/instances_train2017.json',
47:     data_prefix=dict(img='train2017/'),
56:     sampler=dict(type='DefaultSampler', shuffle=False),
59:     data_root=data_root,
60:     ann_file='annotations/instances_val2017.json',
61:     data_prefix=dict(img='val2017/'),
68:     type='CocoMetric',
69:     ann_file=data_root + 'annotations/instances_val2017.json',

```

```

-----
##### MMDetection :: mmdetection/configs__base__default_runtime.py
(25 lines)
(no hits)

```

```

-----
##### MMDetection :: mmdetection/configs__faster_rcnn__faster-
rcnn_r50_fpn_1x_coco.py (6 lines)
1: _base_ = [
2:     '../_base_/models/faster-rcnn_r50_fpn.py',
3:     '../_base_/datasets/coco_detection.py',
4:     '../_base_/schedules/schedule_1x.py',
'../_base_/default_runtime.py'
5: ]

```

```

-----
##### MMDetection :: mmdetection/tools__train.py (122 lines)
(no hits)

```

```

-----
##### Detectron2 :: detectron2/configs__COCO-
Detection__faster_rcnn_R_50_FPN_1x.yaml (7 lines)
1: _BASE_: "../Base-RCNN-FPN.yaml"
2: MODEL:
3:     WEIGHTS: "detectron2://ImageNetPretrained/MSRA/R-50.pkl"
4:     MASK_ON: False

```

```
5:   RESNETS:
6:     DEPTH: 50
```

```
-----
##### Detectron2 :: detectron2/configs__Base-RCNN-FPN.yaml   (43 lines)
14:   PRE_NMS_TOPK_TRAIN: 2000 # Per FPN level
15:   PRE_NMS_TOPK_TEST: 1000  # Per FPN level
19:   POST_NMS_TOPK_TRAIN: 1000
20:   POST_NMS_TOPK_TEST: 1000
32: DATASETS:
33:   TRAIN: ("coco_2017_train",)
34:   TEST: ("coco_2017_val",)
36:   IMS_PER_BATCH: 16
41:   MIN_SIZE_TRAIN: (640, 672, 704, 736, 768, 800)
```

```
-----
##### Detectron2 :: detectron2/tools__train_net.py   (168 lines)
105:           for name in cfg.DATASETS.TEST
```

```
-----
##### Detectron2 :: detectron2/detectron2__data__datasets__builtin.py
(260 lines)
11:   without having to download the dataset annotations
36:   "coco_2014_train": ("coco/train2014",
"coco/annotations/instances_train2014.json"),
37:   "coco_2014_val": ("coco/val2014",
"coco/annotations/instances_val2014.json"),
38:   "coco_2014_minival": ("coco/val2014",
"coco/annotations/instances_minival2014.json"),
41:   "coco/annotations/instances_valminusminival2014.json",
43:   "coco_2017_train": ("coco/train2017",
"coco/annotations/instances_train2017.json"),
44:   "coco_2017_val": ("coco/val2017",
"coco/annotations/instances_val2017.json"),
45:   "coco_2017_test": ("coco/test2017",
"coco/annotations/image_info_test2017.json"),
46:   "coco_2017_test-dev": ("coco/test2017",
"coco/annotations/image_info_test-dev2017.json"),
47:   "coco_2017_val_100": ("coco/val2017",
"coco/annotations/instances_val2017_100.json"),
```

```

53:         "coco/annotations/person_keypoints_train2014.json",
55:         "keypoints_coco_2014_val": ("coco/val2014",
"coco/annotations/person_keypoints_val2014.json"),
58:         "coco/annotations/person_keypoints_minival2014.json",
62:
"coco/annotations/person_keypoints_valminusminival2014.json",
64:         "keypoints_coco_2017_train": (
66:             "coco/annotations/person_keypoints_train2017.json",
68:             "keypoints_coco_2017_val": ("coco/val2017",
"coco/annotations/person_keypoints_val2017.json"),
69:             "keypoints_coco_2017_val_100": (
70:                 "coco/val2017",
71:                 "coco/annotations/person_keypoints_val2017_100.json",
77:             "coco_2017_train_panoptic": (
80:                 "coco/annotations/panoptic_train2017.json",
88:             "coco_2017_val_panoptic": (
89:                 "coco/panoptic_val2017",
90:                 "coco/annotations/panoptic_val2017.json",
91:                 "coco/panoptic_stuff_val2017",
93:             "coco_2017_val_100_panoptic": (
94:                 "coco/panoptic_val2017_100",
95:                 "coco/annotations/panoptic_val2017_100.json",
96:                 "coco/panoptic_stuff_val2017_100",
235:         gt_dir = os.path.join(root, "annotations_detectron2",
dirname)

```

```

-----
### Detectron2 :: detectron2/detectron2__config__defaults.py    (657
lines)

```

```

103: _C.DATASETS.TEST = ()
607: _C.TEST.EVAL_PERIOD = 0

```

```

-----
### YOLOX :: yolox/exps__default__yolox_s.py    (16 lines)

```

```

7: from yolox.exp import Exp as MyExp
15:         self.exp_name =
os.path.split(os.path.realpath(__file__))[1].split(".")[0]

```

```

-----
### YOLOX :: yolox/yolox__data__datasets__coco.py    (189 lines)

```

```
42:         json_file="instances_train2017.json",
43:         name="train2017",
54:         name (str): COCO data name (e.g. 'train2017' or
'val2017')
```

```
-----
##### YOLOX :: yolox/tools__train.py    (147 lines)
(no hits)
```

```
-----
##### YOLOX :: yolox/tools__eval.py    (221 lines)
93:         "--test",
```

```
-----
##### PaddleDetection ::
paddledetection/configs__yolov3__yolov3_darknet53_270e_coco.yml    (11
lines)
1: _BASE_: [
2:   '../datasets/coco_detection.yml',
3:   '../runtime.yml',
4:   '_base_/optimizer_270e.yml',
5:   '_base_/yolov3_darknet53.yml',
6:   '_base_/yolov3_reader.yml',
7: ]
9: snapshot_epoch: 5
10: weights: output/yolov3_darknet53_270e_coco/model_final
```

```
-----
##### PaddleDetection :: paddledetection/tools__train.py    (220 lines)
51:         "--eval",
54:         help="Whether to perform evaluation in train")
66:         help="If set True, enable continuous evaluation job."
94:         help='Whether to save the evaluation results only')
100:         "format \"key1=value1;key2=value2;key3=value3\"."
161:     trainer.train(FLAGS.eval)
```

```
-----
##### DETR :: detr/main.py    (249 lines)
```

```

    16: from engine import evaluate, train_one_epoch
    95:     parser.add_argument('--eval', action='store_true')
   143:     dataset_val = build_dataset(image_set='val', args=args)
   147:     sampler_val = DistributedSampler(dataset_val,
shuffle=False)
    150:     sampler_val =
torch.utils.data.SequentialSampler(dataset_val)
    157:     data_loader_val = DataLoader(dataset_val, args.batch_size,
sampler=sampler_val,
    162:     coco_val = datasets.coco.build("val", args)
    163:     base_ds = get_coco_api_from_dataset(coco_val)
    165:     base_ds = get_coco_api_from_dataset(dataset_val)
    179:     if not args.eval and 'optimizer' in checkpoint and
'lr_scheduler' in checkpoint and 'epoch' in checkpoint:
    184:         if args.eval:
    185:             test_stats, coco_evaluator = evaluate(model, criterion,
postprocessors,
    186:                                                     data_loader_val,
base_ds, device, args.output_dir)
    188:     utils.save_on_master(coco_evaluator.coco_eval["bbox"].eval, output_dir
/ "eval.pth")
    214:     test_stats, coco_evaluator = evaluate(
    215:         model, criterion, postprocessors, data_loader_val,
base_ds, device, args.output_dir
    228:         if coco_evaluator is not None:
    229:             (output_dir / 'eval').mkdir(exist_ok=True)
    230:             if "bbox" in coco_evaluator.coco_eval:
    235:
torch.save(coco_evaluator.coco_eval["bbox"].eval,
    236:         output_dir / "eval" / name)
    244:     parser = argparse.ArgumentParser('DETR training and
evaluation script', parents=[get_args_parser()])

```

```

-----
### DETR :: detr/datasets__coco.py    (159 lines)

```

```

    151:     PATHS = {
    152:         "train": (root / "train2017", root / "annotations" /
f'{mode}_train2017.json'),
    153:         "val": (root / "val2017", root / "annotations" /
f'{mode}_val2017.json'),
    156:     img_folder, ann_file = PATHS[image_set]

```

```

-----
##### DETR :: detr/engine.py (152 lines)
    3: Train and eval functions used in main.py
    13: from datasets.coco_eval import CocoEvaluator
    14: from datasets.panoptic_eval import PanopticEvaluator
    23:     metric_logger.add_meter('lr',
utils.SmoothedValue(window_size=1, fmt='{value:.6f}'))
    24:     metric_logger.add_meter('class_error',
utils.SmoothedValue(window_size=1, fmt='{value:.2f}'))
    43:     losses_reduced_scaled =
sum(loss_dict_reduced_scaled.values())
    45:     loss_value = losses_reduced_scaled.item()
    47:     if not math.isfinite(loss_value):
    48:         print("Loss is {}, stopping
training".format(loss_value))
    58:     metric_logger.update(loss=loss_value,
loss_dict_reduced_scaled, loss_dict_reduced_unscaled)
    68: def evaluate(model, criterion, postprocessors, data_loader,
base_ds, device, output_dir):
    69:     model.eval()
    70:     criterion.eval()
    73:     metric_logger.add_meter('class_error',
utils.SmoothedValue(window_size=1, fmt='{value:.2f}'))
    77:     coco_evaluator = CocoEvaluator(base_ds, iou_types)
    80:     panoptic_evaluator = None
    82:     panoptic_evaluator = PanopticEvaluator(
    85:         output_dir=os.path.join(output_dir,
"panoptic_eval"),
    102:
metric_logger.update(loss=sum(loss_dict_reduced_scaled.values()),
    113:         if coco_evaluator is not None:
    114:             coco_evaluator.update(res)
    116:         if panoptic_evaluator is not None:
    124:             panoptic_evaluator.update(res_pano)
    129:     if coco_evaluator is not None:
    130:         coco_evaluator.synchronize_between_processes()
    131:     if panoptic_evaluator is not None:
    132:         panoptic_evaluator.synchronize_between_processes()
    135:     if coco_evaluator is not None:
    136:         coco_evaluator.accumulate()
    137:         coco_evaluator.summarize()

```

```

139:     if panoptic_evaluator is not None:
140:         panoptic_res = panoptic_evaluator.summarize()
142:     if coco_evaluator is not None:
144:         stats['coco_eval_bbox'] =
coco_evaluator.coco_eval['bbox'].stats.tolist()
146:         stats['coco_eval_masks'] =
coco_evaluator.coco_eval['segm'].stats.tolist()
151:     return stats, coco_evaluator

```

```

=====
=====

```

Missing files: none

S4 · 2×2 evaluation matrix over the three-split protocol (41 runs × {best,last} × {val,test})

Purpose: main text §8.6 and §5.6; covers archive-reproduction verification, the decomposability of Δ_{gap} , and the realization rate of the selection gain.

Source: xeval_analysis_20260916.txt (md5 8b74b2eb6228c078d5a38131d119567b, 8534 bytes); the content below is **embedded verbatim, unchanged**.

```

=====
=====

```

2×2 evaluation matrix × archive: verification / locally recomputable / decomposability of Δ_{gap}

```

=====
=====

```

Matrix entries 164; sio entries 472; (commit/convention: split=test|val, batch=32, imgsz=640, aligned with train_obj.py L75)
Multiple evaluations of the same cell: all consistent

--- per run: archive vs this recomputation (pp)

run	Vmax_a	Vbest_r	Vfin_a
Vfin_r Tbest_a Tbest_r Tfin_r			
r10_aitod20_base30_3way_s42n	14.621	14.621	13.835
13.826 10.210 10.211 10.587			
r10_shwd2sf_base100_3way_s42n	60.694	60.707	59.055
58.977 43.510 43.513 42.946			
r10_shwd2sf_base100_3way_s43n	60.976	60.966	59.761
59.822 43.880 43.885 42.948			
r10_shwd2sf_base100_3way_s44n	61.120	61.135	59.230
59.326 43.050 43.053 42.394			
r10_shwd2sf_base100_3way_s45n	59.852	59.866	58.473

58.474	43.080	43.079	42.901			
r10_shwd2sf_base100_3way_s46n				60.509	60.507	59.293
59.284	43.640	43.641	42.845			
r10_shwd2sf_base100_3way_s47n				60.858	60.886	59.766
59.775	43.780	43.781	43.267			
r10_shwd2sf_base100_3way_s48n				60.554	60.516	59.575
59.615	43.390	43.390	42.784			
r10_shwd2sf_base100_3way_s49n				61.026	60.999	59.169
59.248	43.990	43.992	42.910			
r10_shwd2sf_base100_3way_s50n				60.896	60.854	58.967
59.018	43.640	43.641	42.608			
r10_shwd2sf_base100_3way_s51n				60.209	60.208	58.713
58.734	43.630	43.626	42.636			
r10_shwd2sf_lr005_100ep_3way_s42n				60.860	60.913	59.581
59.586	44.260	44.260	43.569			
r10_shwd2sf_lr005_100ep_3way_s43n				60.408	60.491	59.116
59.105	43.950	43.947	43.861			
r10_shwd2sf_lr005_100ep_3way_s44n				60.001	60.003	58.952
58.920	44.440	44.442	44.134			
r10_shwd2sf_lr005_100ep_3way_s45n				60.795	60.789	60.578
60.579	44.130	44.132	43.934			
r10_shwd2sf_lr005_100ep_3way_s46n				60.462	60.456	59.270
59.298	44.350	44.354	44.242			
r10_shwd2sf_lr005_100ep_3way_s47n				60.678	60.667	60.354
60.355	43.970	43.966	43.944			
r10_shwd2sf_lr005_100ep_3way_s48n				59.722	59.742	58.701
58.711	43.710	43.713	43.795			
r10_shwd2sf_lr005_100ep_3way_s49n				60.363	60.375	59.037
59.028	43.800	43.799	44.027			
r10_shwd2sf_lr005_100ep_3way_s50n				60.193	60.188	59.614
59.634	43.780	43.785	44.078			
r10_shwd2sf_lr005_100ep_3way_s51n				60.491	60.490	60.119
60.153	44.270	44.269	43.560			
r10_smoke2sf_base100_3way_s42n				58.724	58.696	57.618
57.689	42.100	42.103	41.661			
r10_smoke2sf_base100_3way_s43n				58.766	58.798	58.438
58.449	42.320	42.323	41.726			
r10_smoke2sf_base100_3way_s44n				58.864	58.882	58.513
58.526	42.280	42.282	41.367			
r10_smoke2sf_base100_3way_s45n				58.731	58.740	57.594
57.592	41.250	41.253	41.054			
r10_smoke2sf_base100_3way_s46n				58.652	58.689	57.887
57.862	41.900	41.899	41.258			

r10_smoke2sf_base100_3way_s47n	59.271	59.228	58.532
58.557 42.100 42.101 41.437			
r10_smoke2sf_base100_3way_s48n	58.697	58.682	57.778
57.755 42.400 42.402 41.441			
r10_smoke2sf_base100_3way_s49n	58.601	58.617	58.376
58.393 41.960 41.964 42.082			
r10_smoke2sf_base100_3way_s50n	58.449	58.444	57.430
57.410 42.110 42.108 41.923			
r10_smoke2sf_base100_3way_s51n	58.374	58.364	58.057
58.065 42.200 42.202 41.599			
r10_smoke2sf_lr005_100ep_3way_s42n	59.463	59.423	59.463
59.423 43.730 43.728 43.728			
r10_smoke2sf_lr005_100ep_3way_s43n	60.186	60.171	59.090
59.029 43.730 43.731 43.372			
r10_smoke2sf_lr005_100ep_3way_s44n	60.151	60.142	59.578
59.620 43.190 43.191 43.608			
r10_smoke2sf_lr005_100ep_3way_s45n	60.097	60.088	60.019
60.031 43.170 43.174 43.502			
r10_smoke2sf_lr005_100ep_3way_s46n	59.114	59.131	58.735
58.744 43.790 43.785 43.363			
r10_smoke2sf_lr005_100ep_3way_s47n	59.793	59.749	58.790
58.852 43.710 43.714 43.871			
r10_smoke2sf_lr005_100ep_3way_s48n	59.165	59.202	58.739
58.682 43.390 43.389 43.730			
r10_smoke2sf_lr005_100ep_3way_s49n	59.876	59.875	58.560
58.577 43.500 43.503 43.721			
r10_smoke2sf_lr005_100ep_3way_s50n	59.144	59.108	58.178
58.267 42.990 42.989 43.841			
r10_smoke2sf_lr005_100ep_3way_s51n	59.866	59.892	59.649
59.583 43.760 43.759 43.800			

=====

=====

(1) Verification: can this recomputation reproduce the existing archive

=====

=====

(best,test) recomputed vs sio archive: n=41 max diff 0.0047 median 0.0021 pp

⇒  digit-for-digit agreement (<0.05 pp): the evaluation chain follows the project convention, and the archive values can be reproduced independently

(best,val) recomputed vs results.csv maximum: n=41 max diff 0.0834 median 0.0148 pp

⇒ **i** One earlier attribution is withdrawn here. We had put this residual down to best.pt being selected by **fitness** rather than by mAP50-95; in the pinned version **fitness** = **mAP50-95** on the monitored split, so the selection rule does not explain it. The candidate causes are a difference between the training-time validator and this standalone re-evaluation, or best.pt sitting at an epoch other than the logged argmax. We do not know which, and record the residual as **unidentified**, not attributed.

(last,val) recomputed vs results.csv final epoch : n=41 max diff 0.0963 median 0.0215 pp

=====
 =====
 (3) Decomposability of Δ_{gap}
 =====
 =====

Identity: $\text{gap} = M_{\text{val}}(\hat{e}) - M_{\text{test}}(\hat{e}) = (V_{\text{final}} - T_{\text{final}}) + (\text{prem}_{\text{val}} - \text{prem}_{\text{test}})$
 $\text{prem}_{\text{val}} = V_{\text{max}} - V_{\text{final}}$ (results.csv); $\text{prem}_{\text{test}} = T_{\text{best}} - T_{\text{final}}$ (measured in this run)

cell	arm	n	V_fin	T_fin	prem_val	prem_test	gap
check							
aitod20	base	1	13.835	10.587	0.786	-0.377	4.411
shwd2sf	base	10	59.200	42.824	1.469	0.735	17.110
shwd2sf	strat	10	59.532	43.915	0.865	0.151	16.331
smoke2sf	base	10	58.022	41.555	0.691	0.507	16.651
smoke2sf	strat	10	59.080	43.654	0.605	-0.158	16.190

cell	n	Δ_{gap}	Δ_{diffgap}	Δ_{optimism}	sum(check)	dominant
p(Δ_{gap})						
shwd2sf	10	+0.779	+0.759	+0.020	+0.779	Δ_{diffgap}
smoke2sf	10	+0.461	+1.041	-0.580	+0.461	Δ_{diffgap}

0.02627

Reading:

shwd2sf $\Delta\text{gap}=+0.779$ ($p=0.005258$, $n=10$) = difficulty gap $+0.759$ (± 1.051) + selection optimism $+0.020$ (± 0.751)

smoke2sf $\Delta\text{gap}=+0.461$ ($p=0.02627$, $n=10$) = difficulty gap $+1.041$ (± 0.626) + selection optimism -0.580 (± 0.770)

=====

(4) The "realization rate" of the selection gain: how much of the score picked on val is left on test

=====

premier_val = V_max - V_final (book gain obtained by selecting on val)
premier_test = T_best - T_final (gain realized on test by the same selection action)

cell	arm	n	premier_val	premier_test	realized	per-run
premier_test						
aitod20	base	10	0.766	-0.218	-28%	-0.38 -0.24 -0.20
			-0.27 -0.22 -0.11 +0.14 -0.78 -0.18 +0.06			
shwd2sf	base	10	1.469	0.735	50%	+0.56 +0.93 +0.66
			+0.18 +0.79 +0.51 +0.61 +1.08 +1.03 +0.99			
shwd2sf	strat	10	0.865	0.151	18%	+0.69 +0.09 +0.31
			+0.20 +0.11 +0.03 -0.09 -0.23 -0.30 +0.71			
smoke2sf	base	10	0.691	0.507	73%	+0.44 +0.59 +0.91
			+0.20 +0.64 +0.66 +0.96 -0.12 +0.19 +0.60			
smoke2sf	strat	10	0.605	-0.158	-26%	+0.00 +0.36 -0.42
			-0.33 +0.43 -0.16 -0.34 -0.22 -0.85 -0.04			

Total (**headline convention = run-weighted**, $n = 50$, all five cells at $n = 10$): premier_val mean $+0.879$ pp, premier_test mean $+0.205$ pp, realization rate **23.3 %**

(run-level bootstrap 95% CI [8.4 %, 37.4 %], $P(\text{rate} \leq 0) = 0.002$, so the interval excludes no-realization; script work/item2_final_v2_20260925.py)

The three weightings are **identical** once every cell carries $n = 10$ (run-weighted $\equiv$ arm-cell-equal $\equiv n \geq 5$ only), all three giving **23.3 %**; on the previous 41-run sample (aitod20 at $n = 1$) they gave 19.6 % / 32.4 % / 34.5 %, and the ~15 pp spread came from that $n = 1$ cell – it carried 20 % of the arm-cell weight while contributing a

single run to the run-level mean. **Completing that cell removes the spread**, not a change of weighting.

The cell-level bootstrap interval of the arm-cell-equal estimator is [-17.3 %, 53.2 %] and **includes zero** (only five clusters), while the run-level interval [8.4 %, 37.4 %] does not;

so "partial realization" is significant only under run weighting (article §8.6).

Note: this version unifies the total's prem_val on the **declared** convention $V_{\max} - V_{\text{final}}$; the previous version's +0.896 used

best.pt - last.pt, which gives +0.905 on the same 41 runs (a 0.03 pp difference).

The previous version's **arm-cell-equal** totals (prem_val +0.883, prem_test +0.172, also over the 41 runs) are superseded by the 23.3 % above:

with the aitod20 cell completed, arm-cell-equal and run-weighted **are the same estimator**, so it is no longer listed as a separate "sensitivity".

Cells with **negative** prem_test: 2/5 — aitod20/base(-0.218), smoke2sf/strat(-0.157)

⇒ The gain selected on val is only partly realized on an independent split, and can even reverse.

S5 · Selection premium on public third-party logs (sample and reading)

Purpose: main text §5.5; the epoch count, final epoch, best epoch and premium of every log, plus four limitations.

Source: selection_premium_public_20260916.md (md5 04950c92d1ab4bcfb5bbe616b2dcbec4, 9479 bytes); the content below is **embedded verbatim, unchanged**.

Sample

File	Epochs	Final epoch	Best epoch (position)	Premium
max-final	Premium	max-mean of last five epochs	Metric	column name
---	---	---	---	---
aakashchalla12345678-yolov8n__unstated__yolov8n.csv	30	0.4861	0.4861	
(ep 30)	+0.00 pp	+0.26 pp	metrics/mAP50-95(B)	
adityadgmth070-yolov8-fs-grafik-adamw-results6__unstated__yolov8.csv	100	0.4284	0.4284 (ep 100)	+0.00 pp
		+0.54 pp	metrics/mAP50-95(B)	
adityadgmth070-yolov8-fs-grafik-adamw-results7__unstated__yolov8.csv	100	0.4403	0.4494 (ep 74)	+0.91 pp
		+0.79 pp	metrics/mAP50-95(B)	
adityadgmth070-yolov8-fs-grafik-adamw-results8__unstated__yolov8.csv	100			

0.4275 | 0.4511 (ep 30) | **+2.36 pp** | **+2.55 pp** | metrics/mAP50-95(B) |
 | adityadgm070-yolov8-fs-grafik-sgd-results1_unstated_yolov8.csv | 100 |
 0.3151 | 0.3167 (ep 97) | **+0.16 pp** | **+0.17 pp** | metrics/mAP50-95(B) |
 | adityadgm070-yolov8-fs-grafik-sgd-results2_unstated_yolov8.csv | 100 |
 0.4402 | 0.4459 (ep 65) | **+0.57 pp** | **+0.58 pp** | metrics/mAP50-95(B) |
 | adityadgm070-yolov8-fs-grafik-sgd-results3_unstated_yolov8.csv | 100 |
 0.4515 | 0.4648 (ep 53) | **+1.33 pp** | **+1.33 pp** | metrics/mAP50-95(B) |
 | adityadgm070-yolov8-fs-grafik-sgd-results4_unstated_yolov8.csv | 100 |
 0.4340 | 0.4458 (ep 84) | **+1.18 pp** | **+1.20 pp** | metrics/mAP50-95(B) |
 | ayyappakorla-yolov3-results_unstated_yolov3.csv | 200 | 0.9011 | 0.9041
 (ep 198) | **+0.31 pp** | **+0.20 pp** | metrics/mAP_0.5:0.95 |
 | dhanduramesh-yolov11s__ppe-css-data_yolo11n.csv | 1 | 0.4430 | 0.4430 (ep
 1) | **+0.00 pp** | - | metrics/mAP50-95(B) |
 | ericmorning-pcb-yolo-comparison__pcb-components_yolo11.csv | 145 | 0.4647 |
 0.4863 (ep 115) | **+2.16 pp** | **+2.33 pp** | metrics/mAP50-95(B) |
 | ericmorning-pcb-yolo-comparison__pcb-components_yolo12.csv | 150 | 0.4698 |
 0.4853 (ep 132) | **+1.55 pp** | **+1.57 pp** | metrics/mAP50-95(B) |
 | ericmorning-pcb-yolo-comparison__pcb-components_yolov8.csv | 93 | 0.4467 |
 0.4621 (ep 63) | **+1.54 pp** | **+1.49 pp** | metrics/mAP50-95(B) |
 | ikram0703-yolov8-ema-Model1_unstated_yolov8.csv | 123 | 0.6311 | 0.6385
 (ep 98) | **+0.74 pp** | **+0.62 pp** | metrics/mAP50-95(B) |
 | ikram0703-yolov8-ema-Model2_unstated_yolov8.csv | 131 | 0.6289 | 0.6301
 (ep 106) | **+0.10 pp** | **+0.10 pp** | metrics/mAP50-95(B) |
 | ikram0703-yolov8-ema-Model3_unstated_yolov8.csv | 81 | 0.6293 | 0.6435 (ep
 56) | **+1.42 pp** | **+1.27 pp** | metrics/mAP50-95(B) |
 | ikram0703-yolov8-ema-Model4_unstated_yolov8.csv | 182 | 0.6058 | 0.6071
 (ep 157) | **+0.14 pp** | **+0.15 pp** | metrics/mAP50-95(B) |
 | ikram0703-yolov8-ema-Model5_unstated_yolov8.csv | 109 | 0.6294 | 0.6363
 (ep 84) | **+0.70 pp** | **+0.72 pp** | metrics/mAP50-95(B) |
 | kimmkless-yolodetector__train2-vehicle-5class_yolov8n.csv | 100 | 0.4327 |
 0.4327 (ep 100) | **+0.00 pp** | **+0.83 pp** | metrics/mAP50-95(B) |
 | kimmkless-yolodetector__train4-face-expression_yolov8n.csv | 5 | 0.5803 |
 0.5803 (ep 5) | **+0.00 pp** | **+12.94 pp** | metrics/mAP50-95(B) |
 | kimmkless-yolodetector__train8-plant-disease_yolov8n.csv | 100 | 0.4745 |
 0.5029 (ep 80) | **+2.84 pp** | **+2.82 pp** | metrics/mAP50-95(B) |
 | nhnthanhhj-metrics-result-f1-yololight_unstated_yolo-light-seg.csv | 47 |
 0.6521 | 0.7086 (ep 42) | **+5.65 pp** | **+2.42 pp** | metrics/mAP50-95(B) |
 | nhnthanhhj-metrics-result-f1-yolov8n_unstated_yolov8n-seg.csv | 74 | 0.7093
 | 0.7300 (ep 54) | **+2.07 pp** | **+2.10 pp** | metrics/mAP50-95(B) |
 | ottogalt-yolo-result_unstated_yolov8.csv | 20 | 0.9595 | 0.9595 (ep 20) |
+0.00 pp | **+0.56 pp** | metrics/mAP50-95(B) |

> The "best epoch" is read from each log's own **epoch column** (the ep in

the table is the value of that column), not from the row number;
> most logs in this directory start their epoch at 1, while the yolov3 of ayyappakorla starts at 0; both use column values, so they can be compared across rows.

Logs not included in the statistics (too few epochs to speak of "selection"): aakashchalla12345678-yolov8n__unstated__yolov8n.csv (30 epochs), dhanduramesh-yolov11s__ppe-css-data__yolo11n.csv (1 epoch), kimmkless-yolodetector__train4-face-expression__yolov8n.csv (5 epochs), ottogalt-yolo-result__unstated__yolov8.csv (20 epochs)

Reading

- Complete logs (≥ 40 epochs) number **20** in total, coming from **6** source repositories; the median premium is **+1.04 pp**, the maximum **+5.65 pp**, the minimum **+0.00 pp**; **18/20** are positive.
- Distribution: +0.00, +0.00, +0.10, +0.14, +0.16, +0.31, +0.57, +0.70, +0.74, +0.91, +1.18, +1.33, +1.42, +1.54, +1.55, +2.07, +2.16, +2.36, +2.84, +5.65
- **The most important point:** the largest single premium **+5.65 pp** appears in an ordinary public release log, whereas the effects reported in this kind of literature are often only of the order of 0.5–2 pp—that is, **the selection component can be of the same order as, or even larger than, the reported effect**. The median +1.04 pp is already not small. This explains why "on which split the reported value was selected" is not a detail.
- But the sample is still small (20 logs, 6 sources, and most sources do not annotate the dataset/protocol), so it **can only** be written as "this component is present and quantifiable in other people's published logs"; it **cannot** provide a field-level distribution, nor **can** benchmark aliasing be inferred from it.

Verifiability

- **Byte-level verifiable:** the **SHA-256** and **byte count** of every log are given in SHA256SUMS.txt in the same directory (retrieval date 2026-09-16). Anyone can check whether "the bytes analysed" and "the bytes in circulation" agree.
- **Provenance completed:** PROVENANCE.md in the same directory has now been written. **All 24 files** are matched to **specific source paths** by content hash or unique byte count (8 Kaggle datasets + 1 GitHub repo, pinned at commit 2417aff0457c), with 0 untraced entries. The mapping is not guessed from filename similarity: the script hashes **every entry** in _work\kg*.zip and collides it against the sha256 of the local files. Check: python

work/build_provenance.py.

- Two things that **still** cannot be derived from this directory (see PROVENANCE.md §4): ① the **exact downloaded version** on the Kaggle side; ② whether the val/test of these benchmarks are in the same directory (the logs do not carry a yaml).

What is still missing (to be stated as well when writing the main text)

1. **Sample size:** 20 logs, 6 sources—enough to show that the component exists in other people's logs and to give its order of magnitude, but not enough to give a field-level distribution; discussing a distribution requires several tens of traceable logs.
2. **Protocol unknown:** public logs carry no yaml, so it is impossible to judge whether the val and test of a run are the same directory; therefore "the premium exists" and "benchmark aliasing" are two independent propositions and cannot stand in for each other.
3. **Endpoint convention:** the logs give only per-epoch validation values; public repositories usually do not record "which epoch was reported", so the premium is an **upper-bound** reading (assuming that what was reported is the best).
4. **Column-name convention:** this table accepts both the new Ultralytics spelling metrics/mAP50-95(B) and the old YOLOv5/v3 spelling metrics/mAP_0.5:0.95 (each row marks which one it uses). The two spellings denote the same quantity, but the **column names differ**—any script that "filters logs by column name" will miss the other branch; that is the purpose of this item: to make the convention difference visible.

S6 · Process protocols (shared tooling; used by both papers, neither claims originality)

Purpose: main text §2.3 states that "this paper and its companion paper share the same set of process protocols, and neither claims originality".

This section **grounds that statement in verifiable names and hashes** (this is how cross-citation rule ⑤b is implemented).

Form decision (this paper's side): these two protocols are **project-internal artifacts**, and **their text is not released with this paper**—they are not public publications,

and they bear no direct relation to this paper's scientific claims. This paper states only that "**the same set of protocols was used**", and gives the **file names and md5**,

so that a verifier can request the same files from the authors. **If a reviewer requires it, they can be supplied together with the revision.**

Protocol (internal artifact)	md5	Role in this paper	Claims originality?
---	---	---	---
the multi-model review checklist (shared protocol)	e4c73a5ca5992b7af6a5a66c84ba4561	Stratified review by external models; among the ten self-reported defects in §10, "silent exclusion" and "wrong column taken" were found by that review	No , neither paper claims it
the local/cloud working-method guide (shared protocol)	10d1b3a3e21f018f3b82e4a293242695	Local/cloud working method: scripted remote execution, all-or-nothing patches, audit anchors, a local pointer for every number	No , neither paper claims it

The same convention also applies to the audit-anchor system: the 289 anchors of this paper and the 494 anchors of the companion paper are the **same** shared tooling,

and neither paper presents it as a methodological contribution.

Why the text is not attached: ① both are **Chinese working documents** with no direct relation to this paper's scientific claims;

② attaching them would make "process" read as "methodological contribution", which is exactly what this rule is meant to avoid; ③ giving the md5 is already enough to confirm that "the two papers use the same one".

S7 · Reproduction entry points (anyone can recompute independently)

① Main-text anchors (289 entries; must ALL PASS)

```
python work\audit_anchors_fupaper.py
```

② 2x2 matrix analysis of S4 (reads the artifacts pulled back in this round)

```
python work\xeval_analyze_20260916.py
```

③ Mechanism check of §5.6 (reads the D: archive, read-only)

```
python work\gap_mechanism_20260916.py
```

④ Integrity of the artifacts pulled back in this round (sha256 + byte

counts)
python work\verify_pulled.py
python work\verify_pulled.py _xframe

Convention statement: the source files of S1–S7 are all different and **must not be mixed**——
S1/S2 are a **configuration-level audit** (no datasets are downloaded),

S3 is a **targeted sampling of official configs/scripts** (5 frameworks, not a census), while S4/S5
are **empirical** (the former on our own runs, the latter on other people's logs).

The strength of every statement in the main text is distinguished accordingly.

S8 · Tables moved from the main text

Why they are here: the venue caps submitted manuscripts at 20–35 double-spaced
numbered pages, and these evidence tables cost about eight pages in that format. ****No
value has been changed****; every table is reproduced here exactly as it appears, and the
main text states each table's headline result in prose.

Four of these tables are also printed in the article — main text Tables 1–4, which are
S4 (the Tier 1 test), S6 (the Tier 2 test), S11 (the four counting units) and S20 (the
per-cell realization) below — because each carries a headline number the reader is asked
to check; the article prints them in the same form as here.

One further table deliberately stays in the article and is not reproduced here: the
defect table of main text §10, because disclosure belongs in the article rather than in the
Supplementary. (The figure/table inventory has itself moved here — see the section before §S1.)

Index. (n = data rows.)

Supp. table	Moved from	Columns	n
---	---	---	---
Table S1	2.1 The closest work, and where this paper sits	Work, What it audits, What it measures, Reports a bias direction, Declares a unit	6
Table S2	3. Units: the methodological premise	Unit, Definition, Controlled by	4
Table S3	5. Finding 2: the selection premium — magnitude, direction, and mechanism	Cell, Arm, n, best, last, premium	6
Table S4	5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus	Cell, n, premium baseline, premium strategy, paired diff, sd, t, p, 95 % CI	4

Table S5	5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus	Cell, best (selectable), last, last5	3
Table S6	5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus	Corpus, n, Δ gap (pp), p, 95 % CI, Reading	5
Table S7	5.5 The same component in third-party public logs	Quantity, Value	5
Table S8	5.6 Why Δ gap's sign varies by corpus: three of five candidate mechanisms are refuted by the data	Cell, n, M_val base / strat, M_test base / strat, Δ gap (pp), p	13
Table S9	5.6 Why Δ gap's sign varies by corpus: three of five candidate mechanisms are refuted by the data	Candidate mechanism, Prediction, Verdict, Refutation / evidence	5
Table S10	5.6 Why Δ gap's sign varies by corpus: three of five candidate mechanisms are refuted by the data	Corpus, Δ gap, final-epoch val diff, Δ prem, -test diff, Dominant	7
Table S11	6.1 The result under four units	Unit, non-independent, of decidable rows, undecidable	6
Table S12	6.2 "Same directory" has four levels, and the audit reported only the narrowest	Level, Rows, Meaning	4
Table S13	6.3 We do not reproduce the original audit's two numbers, and we say why	Audit's number, Our recomputation, Reason	3
Table S14	6.5 Across frameworks: is the convention specific to YOLO?	Framework (pinned commit), Split the reported metric comes from, Also used for selection?, Status of an independent "test"	6
Table S15	7.2 Both conventions become significant only at $n \geq 6$	Cell, n, Δ , paired t p, permutation p (n = 10 floor 0.001953), seeds positive	2
Table S16	7.3 A cross-archive endpoint conflict on the same run (investigated — and itself a disclosure)	Reading, Δ , p	2
Table S17	8.1 The protocol is the companion's; the measurement under it is this paper's	Cell, clean protocol (n = 10), published (old protocol)	2
Table S18	8.3 🌟 A direct measurement of selection inflation	Cell, n = 6, n = 9, change	2
Table S19	8.4 The three variance components: the excluded one is the smaller in this sample	Source, What was varied, Magnitude	3
Table S20	8.6 🌟 The premium's realization rate: arm-cell median 17.6 % (five arm-	Cell, Arm, n, prem_val, prem_test, realization rate	5

	cells, $n = 10$; run-weighted 23.3 %)		
Table S21	8.7 The per-epoch view: the realization rate is not a constant	Cell · arm, epoch, $V(e)$, $T(e)$, $\Delta V(e)$, $\Delta T(e)$	4
Table S22	8.7 The per-epoch view: the realization rate is not a constant	Group, n , κ , R^2	3
Table S23	8.7 The per-epoch view: the realization rate is not a constant	Cell, checkpoints evaluated, $\Delta_{\text{gap}} > 0$, $\Delta_{\text{gap}} < 0$, epochs with $\Delta_{\text{gap}} < 0$, Δ_{gap} range	2
Table S24	9 (the registered replication): the criterion is not met (the third row is the one that passes)	Registered pair, n , Δ (pp), 95 % CI, paired t, permutation p, same sign, $\Delta \geq +0.30$, $p < 0.01$, $\geq 8/10$, unit criterion	3
Table S25	9 (the registered replication): the criterion is not met (the third row is the one that passes)	Question the frozen criterion asks, Measured	3
Table S26	9 (the near-matched control C): the gain does not depend on a label-space change	n , Δ (pp), 95 % CI, paired t, permutation p, same sign	1
Table S27	9. The registered multi-target replication: transferred to the companion paper	Registered pair, data YAML, val , test , Split	5
Table S28	5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus	seed, premium baseline, premium strategy, paired difference	11
Table S29	5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus	seed, Δ_{gap} p_aitovis, Δ_{gap} p_vistod15	10
Table S31	3.2 Six experiments, five judged criteria, on GPU machines, each with its criterion written down first (G2: the selection split's size)	selection split, arm, n , prem_val , prem_test , WC, realization rate	4
Table S32	3.2 Six experiments, five judged criteria, on GPU machines, each with its criterion written down first (G3: the epoch budget)	arm, n , premium at $E = 50/100/200/400$, ratio 400/100	2
Table S33	3.2 Six experiments, five judged criteria, on GPU machines, each with its criterion written down first (G4: a third corpus)	run, prem_val , prem_test , WC, realization rate	20

Two notes spanning S24–S27 (added 2026-09-18). ① The experiment contains

90 runs: three registered pairs $3 \times 2 \times 10 = 60$, the saturated control $2 \times 5 = 10$

(the registered set is 70), and the control appended by amendment $2 \times 10 = 20$ — counted

from the run directories, ten seeds per arm in every cell but the saturated control.

② **Val-side alias bias**, $\Delta \approx \Delta_{\text{reported}} + \bar{d}$ with $\bar{d} = \text{mean}[p_{\text{baseline}} - p_{\text{strategy}}]$

and $p = \text{max_e} - \text{final}$ from each run's own log: T1-a **+0.051** [0.006, 0.099], T1-c

+0.214 [0.086, 0.326], control C **+0.406** [0.313, 0.498], saturated control

+0.961 [0.440, 1.674] ($\Delta + \bar{d} = -0.529$). Its decomposition: baseline arm **2.505 pp**, strategy arm **1.543 pp** — which is a two-thirds reading of that cell's -1.490 pp. T1-b is excluded by construction — its YAML

separates the keys. Script `work/x10_alias_audit_20260918.py`. **No verdict changes.**

Table S1 — moved from *2.1 The closest work, and where this paper sits*

Table S1. 2.1 *The closest work, and where this paper sits* — columns: Work, What it audits, What it measures, Reports a bias direction, Declares a unit.

Work	What it audits	What it measures	Reports a bias direction	Declares a unit
---	---	---	---	---
Reveles-Martínez et al., <i>Technologies</i> 2026 [13]	Image level: near-duplicate video frames contaminating train/val	Controlled ablation: a random split scores 0.040 mAP50-95 higher than a sequence-aware one (0.787 vs 0.747)	✗	✗ (0 hits for "same directory"/"alias"/"checkpoint selection" in the full text)
Ruangsang & Pramkeaw, <i>Drones</i> 2026 [14]	Image level: near-duplicate training images inside the official test split (46.0 % / 90.7 %)	Removing contamination costs 2.8 / 12.4 points mAP@0.5 ; it does report a val-vs-test gap	Yes, but concludes "no systematic direction" (−0.41 to +0.49 points)	✗ (0 hits for "same directory"/"alias"/"reported split"/"selection bias")
roboflow/rf-detr PR #1329	Engineering fix	none	—	—
ultralytics issue #25650	Engineering fix (3 KITTI Eigen test drives were in train)	none	—	—
CVPR 2026 geospatial leakage audit [1]	Image level (geospatial)	detection + de-duplication	✗	✗
This paper	Configuration level: the release binds <code>val:/test:</code> to one path	Checkpoint selection premium (best – last) + paired difference Δ_{gap}	Yes, in two tiers: the premium is positive in four cells with the baseline arm gaining more; Δ_{gap} 's sign varies by corpus (2 understate / 2 overstate / 9 n.s.)	Yes: four units, with per-row evidence classes

Table S2 — moved from *3. Units: the methodological premise*

Table S2. 3. *Units: the methodological premise* — columns: Unit, Definition, Controlled by.

Unit	Definition	Controlled by
---	---	---
release	Does the benchmark's own release artifact (a split script, an archive, a YAML) provide an independent held-out test split?	the releasing party
protocol	What does the official protocol (README, paper, website) declare, and can that test split be evaluated locally ?	the releasing party
yolo_dist	How do the val:/test: keys in the generic YOLO distribution package (e.g. an Ultralytics-style directory) read?	a third party
reported	Which split does the number in the literature come from, and is that split independent of selection?	the user

Table S3 — moved from *5. Finding 2: the selection premium — magnitude, direction, and mechanism*

Table S3. 5. Finding 2: the selection premium — magnitude, direction, and mechanism — columns: Cell, Arm, n, best, last, premium.

Cell	Arm	n	best	last	premium
---	---	---	---	---	---
smoke→SFCHD (strong)	baseline	11	42.156	41.357	+0.799
	strategy	11	43.863	43.481	+0.382
SHWD→SFCHD (corroborating)	baseline	11	43.864	42.826	+1.038
	strategy	11	44.428	43.916	+0.512
dota15→aitod (directional probe)	baseline	10	11.802	11.268	+0.534
	strategy	10	15.401	15.139	+0.262

Table S4 — moved from *5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus*

Table S4. 5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus — columns: Cell, n, premium baseline, premium strategy, paired diff, sd, t, p, 95 % CI.

Cell	n	premium baseline	premium strategy	paired diff	sd	t	p	95 % CI
---	---	---	---	---	---	---	---	---
smoke2sf (strong)	11	+0.799	+0.382	+0.416	0.240	5.74	1.9×10^{-4}	[+0.255, +0.578]
shwd2sf (corroborating)	11	+1.038	+0.512	+0.526	0.207	8.41	7.6×10^{-6}	[+0.386, +0.665]
a2d15 (directional probe)	10	+0.534	+0.262	+0.272	0.183	4.69	1.1×10^{-3}	[+0.141, +0.403]
dota15 (within-domain)	10	+0.941	+0.492	+0.449	0.196	7.23	4.9×10^{-5}	[+0.309, +0.589]

Table S5 — moved from *5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus*

Table S5. *5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus* — columns: Cell, best (selectable), last, last5.

Cell	best (selectable)	last	last5
---	---	---	---
smoke→SFCHD	+1.708	+2.124	+2.086
SHWD→SFCHD	+0.564	+1.090	+1.079
dota15→aitod	+3.599	+3.871	+3.775

Table S6 — moved from *5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus*

Table S6. *5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus* — columns: Corpus, n, Δgap (pp), p, 95 % CI, Reading.

Corpus	n	Δgap (pp)	p	95 % CI	Reading
---	---	---	---	---	---
shwd2sf	10	+0.779	0.0053	[+0.297, +1.261]	val understates
smoke2sf	10	+0.461	0.0263	[+0.068, +0.855]	val understates
p_aitovis	10	-1.226	1.3×10^{-8}	[-1.370, -1.081]	val overstates
p_vistod15	10	-3.315	6.5×10^{-9}	[-3.676, -2.953]	val overstates
other 9 cells	3–10	-2.46 to +13.66	≥ 0.054	all include 0	not significant

Table S7 — moved from *5.5 The same component in third-party public logs*

Table S7. *5.5 The same component in third-party public logs* — columns: Quantity, Value.

Quantity	Value
---	---
Complete logs (≥ 40 epochs)	20 / 6 sources
Median premium	+1.04 pp
Max / min	+5.65 / +0.00 pp
Positive	18/20
Distribution	+0.00, +0.00, +0.10, +0.14, +0.16, +0.31, +0.57, +0.70, +0.74, +0.91, +1.18, +1.33, +1.42, +1.54, +1.55, +2.07, +2.16, +2.36, +2.84, +5.65

Table S8 — moved from *5.6 Why Δgap's sign varies by corpus: three of five candidate mechanisms are refuted by the data*

Table S8. *5.6 Why Δgap's sign varies by corpus: three of five candidate mechanisms are refuted by the data* — columns: Cell, n, M_val base / strat, M_test base / strat, Δgap (pp), p.

Cell	n	M_val base / strat	M_test base / strat	Δgap (pp)	p
---	---	---	---	---	---
shwd2sf	10	60.67 / 60.40	43.56 / 44.07	+0.779	0.0053

smoke2sf	10	58.71 / 59.69	42.06 / 43.50	+0.461	0.0263
p_aitovis	10	11.24 / 12.66	10.21 / 10.40	-1.226	1.3×10^{-8}
p_vistod15	10	16.96 / 23.93	11.63 / 15.26	-3.315	6.5×10^{-9}
aitod20	3	14.51 / 12.13	10.40 / 9.49	+1.481	0.054
d15d15	3	30.57 / 29.09	15.17 / 14.96	+1.278	0.092
mende20	3	73.40 / 68.03	73.08 / 70.17	+2.472	0.099
fire	3	60.81 / 42.76	39.58 / 35.20	+13.658	0.061
p_d15toai	3	12.83 / 12.31	9.59 / 9.86	+0.781	0.32
mask20	3	34.15 / 31.46	65.35 / 64.42	+1.754	0.66
p_masktomende	3	60.36 / 60.44	69.36 / 66.97	-2.463	0.16
dota	3	30.33 / 29.04	17.17 / 16.24	+0.355	0.56
vis	3	14.19 / 13.39	11.64 / 11.01	+0.169	0.49

Table S9 — moved from *5.6 Why Δ gap's sign varies by corpus: three of five candidate mechanisms are refuted by the data*

Table S9. 5.6 Why Δ gap's sign varies by corpus: three of five candidate mechanisms are refuted by the data — columns: Candidate mechanism, Prediction, Verdict, Refutation / evidence.

Candidate mechanism	Prediction	Verdict	Refutation / evidence						
---	---	---	---						
M1 multiplicative gain	if the strategy improves by a fixed ratio , Δ gap has the same sign as the raw gap	refuted	3 counterexamples: mask20 (gap -31.2 but Δ gap +1.75), p_aitovis (+1.03 but -1.23), p_vistod15 (+5.34 but -3.32)						
M2 ceiling compression	the easier val is (larger \	gap\	), the more the strategy's advantage is compressed, so Δ gap turns negative as \	gap \	grows	refuted	all six cells with \	gap \	> 10 have Δ gap positive (+0.35 to +13.66); overall $r = +0.32$, $p = 0.28$
M3 level determines resolvability	cells with higher mAP are more likely to be significant	not supported	significant cells' baseline val mean 36.83 pp , non-significant 36.79 pp (Welch $p = 0.998$)						
M4 whether the strategy truly gains on test	only cells where the strategy gains on test can show val understating	refuted	2 counterexamples: p_aitovis and p_vistod15						

			are better on test yet have negative Δ_{gap}						
M5 difference in val-side selection gain	Δ_{gap} has the same sign as (premium_baseline – premium_strategy)	not supported	only 8 of 13 cells share the sign; $r = +0.06$, $p = 0.84$						

Table S10 — moved from *5.6 Why Δ_{gap} 's sign varies by corpus: three of five candidate mechanisms are refuted by the data*

Table S10. *5.6 Why Δ_{gap} 's sign varies by corpus: three of five candidate mechanisms are refuted by the data* — columns: Corpus, Δ_{gap} , final-epoch val diff, Δ_{prem} , –test diff, Dominant. **The last three columns already carry the sign convention, so Δ_{gap} is their sum** (the –test diff column is printed with the sign the identity needs).

Corpus	Δ_{gap}	final-epoch val diff	Δ_{prem}	–test diff	Dominant
---	---	---	---	---	---
mask20	+1.75	+15.78	–13.10	–0.93	the two large components nearly cancel
fire	+13.66	+15.96	+2.08	–4.39	final-epoch val diff
p_vistod15	–3.315	–7.072	–0.213	+3.971	final-epoch val diff
p_aitovis	–1.226	–1.583	+0.090	+0.267	final-epoch val diff
shwd2sf	+0.78	–0.33	+0.60	+0.51	Δ_{prem}
smoke2sf	+0.46	–1.06	+0.09	+1.43	–test diff
dota	+0.35	+0.75	+0.54	–0.93	–test diff

Table S11 — moved from *6.1 The result under four units*

Table S11. *6.1 The result under four units* — columns: Unit, non-independent, of decidable rows, undecidable.

Unit	non-independent	of decidable rows	undecidable
---	---	---	---
release (the benchmark's own release)	10/19 $\approx$ 53 %	10/19	—
protocol (the official protocol)	13/19 $\approx$ 68 %	13/19	—
yolo_dist (the generic YOLO distribution package)	4/19 $\approx$ 21 %	4/15 $\approx$ 27 %	4 rows
reported (the layer actually reported)	12/19 $\approx$ 63 %	12/15 = 80 %	4 rows
↳ of which literal two-key same path (= the 2/19 the original audit reported)	2/19 $\approx$ 11 %	2/19	—
reported – clean	3/19 $\approx$ 16 %	3/15	4 rows

Table S12 — moved from *6.2 "Same directory" has four levels, and the audit reported only the narrowest*

Table S12. 6.2 "Same directory" has four levels, and the audit reported only the narrowest — columns: Level, Rows, Meaning.

Level	Rows	Meaning
---	---	---
L1 distribution package, literal <code>val == test</code>	2	PASCAL VOC, SFCHD
L2 distribution package, <code>train == val</code>	1	SHWD
L3 distribution package clean, but the reported layer is non-independent	9	Objects365, Open Images v7, DOTA v2.0, UAVDT, xView, NWPU VHR-10, MAFA, WIDER FACE, CrowdHuman
L4 the rest	7	

Table S13 — moved from *6.3 We do ****not**** reproduce the original audit's two numbers, and we say why*

Table S13. 6.3 We do not reproduce the original audit's two numbers, and we say why — columns: Audit's number, Our recomputation, Reason.

Audit's number	Our recomputation	Reason
---	---	---
14/19 (the reported layer)	12/19	(h-1) removes D-Fire; DIOR and Mendeley are evidence-insufficient at the reported layer
5/19 clean	3/19	the same two rows move to "undecidable" (they were never adjudicated), so the clean group loses two
12/19 (called "release/spec layer")	release 10/19, protocol 13/19	12 is neither: it was obtained by a two-row manual adjustment (removing WIDER FACE and CrowdHuman), not by applying one unit consistently to all 19 rows

Table S14 — moved from *6.5 Across frameworks: is the convention specific to YOLO?*

Table S14. 6.5 Across frameworks: is the convention specific to YOLO? — columns: Framework (pinned commit), Split the reported metric comes from, Also used for selection?, Status of an independent "test".

Framework (pinned commit)	Split the reported metric comes from	Also used for selection?	Status of an independent "test"
---	---	---	---
Ultralytics YOLO [19] (the <code>yolo_di</code>)	the YAML's <code>val:</code>	yes (<code>best.pt</code> ; §5's "selection premium" is defined on exactly this convention)	the config has a <code>test:</code> key, but 2 of 19 rows bind it to the same path (§6.1)

st unit of §6.1)			
Dectro n2 [33] a2f4a8771ab7	DATASETS.TEST: ("coco_2017_val",)	no (TEST.EVAL_PERIOD = 0, no periodic evaluation by default)	coco_2017_test exists, but its annotation file is <code>`image_info_test2017.json`</code> — image information only, no ground truth
MMDetection [34] cfd5d3a985b0	CocoMetric(ann_file='annotations/instances_val2017.json'), val_interval=1	no (CheckpointHook(interval=1), the base config has no save_best)	the base config has no test split at all (train and val dataloaders only)
YOLOX [32] 6ddff4824372	val_ann = "instances_val2017.json"	yes (trainer.py tracks self.best_ap and saves the best)	test_ann is used only when <code>`testdev=True`</code>
PaddleDetection [36] b25522a0f4bd	EvalDataset = val2017	not written by default, and now provable line by line — paddledetection/tools__train.py declares <code>--eval</code> as <code>action='store_true'</code> , <code>default=False</code> and passes it to <code>trainer.train(FLAGS.eval)</code> ; the assignment <code>status['save_best_model'] = True</code> sits inside the validation branch of <code>paddledetection/ppdet__engine__trainer.py</code> (if <code>validate</code> and <code>is_snapshot</code>), and <code>best_model</code> is written only through <code>paddledetection/ppdet__engine__callbacks.py</code> 's gate if <code>'save_best_model'</code> in <code>status</code> and <code>status['save_best_model']</code> : — both files are now in the SHA-256 list	<code>`TestDataset`</code> reads an annotation-free <code>`ImageFolder`</code> , with <code>`anno_path`</code> written as <code>`annotations/instances_val2017.json`</code>
DETR [35] 29901c51d7fe	build_dataset(image_set='val')	no (evaluates every epoch but does not track the best)	the PATHS in <code>datasets/coco.py</code> have only train and val, no test key

Table S15 — moved from *7.2 Both conventions become significant only at $n \geq 6$ *

Table S15. 7.2 Both conventions become significant only at $n \geq 6$ — columns: Cell, n , Δ , paired t p, permutation p ($n = 10$ floor 0.001953), seeds positive.

Cell	n	Δ	paired t p	permutation p ($n = 10$ floor 0.001953)	seeds positive
---	---	---	---	---	---
smoke→SFCHD	10	+1.434 pp	8.6×10^{-7}	0.001953 (= the floor)	10/10

SHWD→SFCHD	10	+0.507 pp	0.0094	0.009766	9/10 (one at -0.19)
------------	-----------	------------------	--------	-----------------	---------------------

Table S16 — moved from *7.3 A cross-archive endpoint conflict on the same run (investigated — and itself a disclosure)*

Table S16. 7.3 A cross-archive endpoint conflict on the same run (investigated — and itself a disclosure) — columns: Reading, Δ , p.

Reading	Δ	p
---	---	---
Taking the final-epoch row (A, 67.12)	-1.490	0.0141
Taking the best-epoch row (B, 70.06)	-0.902	0.2931

Table S17 — moved from *8.1 The protocol is the companion's; the measurement under it is this paper's*

Table S17. 8.1 The protocol is the companion's; the measurement under it is this paper's — columns: Cell, clean protocol (n = 10), published (old protocol).

Cell	clean protocol (n = 10)	published (old protocol)
---	---	---
smoke→SFCHD	+1.434 pp	+1.78 (3 seeds) / +1.76 (10 seeds)
SHWD→SFCHD	+0.507 pp	+0.51 (3 seeds) / +0.54 (10 seeds)

Table S18 — moved from *8.3 ★ A direct measurement of selection inflation*

Table S18. 8.3 ★ A direct measurement of selection inflation — columns: Cell, n = 6, n = 9, change.

Cell	n = 6	n = 9	change
---	---	---	---
smoke→SFCHD	+1.562	+1.420	-0.14 pp
SHWD→SFCHD	+0.693	+0.492	-0.20 pp

Table S19 — moved from *8.4 The three variance components: the excluded one is the smaller in this sample*

Table S19. 8.4 The three variance components: the excluded one is the smaller in this sample — columns: Source, What was varied, Magnitude.

Source	What was varied	Magnitude
---	---	---
augmentation RNG	3 augmentation seeds	0.246 pp (strategy) / 0.190 pp (baseline)
initialization (source-side retraining from scratch)	3 independent initializations	0.146 pp (gain) / 0.209 pp (absolute endpoint)
second-stage data order	9–10 seeds	0.403 pp (smoke) / 0.515 pp (SHWD)

The three rows are not in one unit, and we do not compare them as if they were. The augmentation and

data-order rows give **standard deviations of the run-to-run change (pp)**; the initialization row is

reported in **both** of the units its source record carries — **0.146 pp as a gain** and ****0.209 pp as an**

absolute endpoint — **because those two are not interchangeable. The claim §8.4 makes is a ranking****,

and that ranking holds under either reading; the three magnitudes are **not** comparable across rows in a

single unit, and **no cross-row arithmetic is performed on them**.

Table S20 — moved from *8.6 ★ The premium's realization rate: arm-cell median ****17.6 %**** (five arm-cells, n = 10; run-weighted ****23.3 %****)*

Table S20. 8.6 ★ *The premium's realization rate: arm-cell median 17.6 % (five arm-cells, n = 10; run-weighted 23.3 %)* — columns: Cell, Arm, n, prem_val, prem_test, realization rate.

Cell	Arm	n	prem_val	`prem_test`	realization rate
---	---	---	---	---	---
shwd2sf	baseline	10	+1.469	+0.736	50 %
shwd2sf	strategy	10	+0.865	+0.152	18 %
smoke2sf	baseline	10	+0.691	+0.509	74 %
smoke2sf	strategy	10	+0.605	−0.157	−26 %
aitod20	baseline	10	+0.766	−0.218	−28 %

One earlier reading of this same cell is left standing, on purpose. The verbatim analysis dump

in §S4 still shows aitod20 base 1 ... −0.377 — that is **this cell before it was completed**, at

n = 1, and §S4 is that sample's record, not a competing summary; the row above is the completed

n = 10 measurement. §S4 keeps its original bytes and its recorded hash because it is an embedded

dated artifact.

A theory-driven prediction, tested on the same runs (added later). §3.1 derives

realization rate = 1 − WC / prem_val with WC := E[ε_sel − ε_final] ≥ 0, the winner's-curse term.

On the archive: ① WC = prem_val − prem_test = **+0.463 pp** per run (t = 8.34, p = 1.8×10^{−12};

the realized premium is smaller in **68/81** runs); ② sorting runs by the noise scale $\hat{\sigma}$ of their

val curve gives realization-rate terciles of **65.6 % / 49.8 % / 13.8 %**, p = **−0.466** (p = 1.3×10^{−5}, n =

80). **Both readings are re-taken with arm-cell clustering, as the kappa fit already is: WC =**

+0.463 pp with a cluster-robust 95 % interval [+0.204, +0.721] and p = 0.0033, and p = −0.466

with a cluster-bootstrap interval [−0.651, −0.131] and p = 0.0115 — which at the cluster level (n

= 9) is −0.433 with p = 0.244 — so the clustered reading is the one this paper reports

(Supplementary S12, section 8.6, script work/r102_cluster_20260927.py). The stronger form

(premium ≈ $\sigma\sqrt{2 \ln E}$) was **withdrawn**:

slope 0.026 ± 0.222 , $p = 0.82$, $n = 101$. Scripts: `work/theory_p4_test_20260919.py`,
`work/theory_meta_and_tests_20260919.py`.

Table S21 — moved from *8.7 The per-epoch view: the realization rate is not a constant*

Table S21. 8.7 *The per-epoch view: the realization rate is not a constant* — columns: Cell · arm, epoch, V(e), T(e), ΔV(e), ΔT(e).

Cell · arm	epoch	V(e)	T(e)	ΔV(e)	ΔT(e)
---	---	---	---	---	---
shwd2sf · baseline	20 / 40 / 60 / 80	57.62 / 58.57 / 59.37 / 59.79	41.58 / 42.73 / 43.14 / 43.46	−1.58 / −0.63 / +0.17 / +0.59	−1.24 / −0.10 / +0.32 / +0.63
shwd2sf · strategy	20 / 40 / 60 / 80	52.34 / 56.17 / 58.51 / 59.38	37.98 / 41.21 / 42.69 / 43.91	−7.20 / −3.36 / −1.02 / −0.15	−5.94 / −2.71 / −1.22 / −0.00
smoke2sf · baseline	20 / 40 / 60 / 80	53.69 / 56.25 / 57.20 / 58.08	38.91 / 41.04 / 41.68 / 42.10	−4.33 / −1.77 / −0.82 / +0.05	−2.64 / −0.52 / +0.13 / +0.55
smoke2sf · strategy	20 / 40 / 60 / 80	50.99 / 55.15 / 57.38 / 58.46	36.91 / 41.03 / 42.73 / 43.55	−8.09 / −3.94 / −1.70 / −0.62	−6.74 / −2.63 / −0.92 / −0.11

Complete curve (added 2026-09-19). This table keeps the five-epoch sample. The complete evaluation — every retained checkpoint of the same 41 runs, 806 rows — is archived as `xeval_perepoch_20260918/matrix_perepoch.csv` (SHA-256 listed) rather than printed, and the two agree on all **202** overlapping (run, epoch) points to **0.0000 pp**.

Table S22 — moved from *8.7 The per-epoch view: the realization rate is not a constant*

Table S22. 8.7 *The per-epoch view: the realization rate is not a constant* — columns: Group, n, κ, R².

Group	n	κ	R ²
---	---	---	---
All four arm-cells	765	0.708	0.937
Baseline arm only	385	0.600	0.946
Strategy arm only	380	0.767	0.951

Complete curve (added 2026-09-19). These are the values over **every retained checkpoint** (765 (seed, epoch) points, 41 runs, 806 evaluations, 0 failures). The five-epoch grid of the first version gave **0.734 / 0.532 / 0.769** over $n = 161 / 81 / 80$; the overall estimate therefore moves **−0.026** with overlapping intervals, and the arm difference narrows from +0.237 to **+0.168**. Script: `work/x3b_perepoch_corrected_20260918.py`.

****Uncertainty (added 2026-09-18)** — scoped to the five-epoch grid ($n = 161$), not to the complete-curve

value the article prints.** The epochs within a run are not independent observations, so the fit is re-done with run-clustered inference (41 runs):

all four arm-cells $\kappa = 0.734$, 95 % CI [0.683, 0.784] (sandwich SE 0.025; a run-level bootstrap gives [0.685, 0.784]); baseline arm [0.456, 0.608]; strategy arm [0.712, 0.827].

Resampling runs *within* each arm (the two arms are never pooled) puts the arm difference at +0.237, 95 % CI [0.145, 0.323]; a within-run fit, which removes run-level differences entirely, gives 0.774. Scripts: work/x2_kappa_clustered_20260918.py, work/x2b_kappa_arms_20260918.py (both reproduce the published 0.734 / 0.902 / n = 161 first).

Table S23 — moved from *8.7 The per-epoch view: the realization rate is not a constant*

Table S23. 8.7 *The per-epoch view: the realization rate is not a constant* — columns: Cell, checkpoints evaluated, $\Delta\text{gap} > 0$, $\Delta\text{gap} < 0$, epochs with $\Delta\text{gap} < 0$, Δgap range.

Cell	checkpoints	$\Delta\text{gap} > 0$	$\Delta\text{gap} < 0$	epochs with $\Delta\text{gap} < 0$	Δgap range (pp)
---	---	---	---	---	---
shwd2sf	19	19	0	—	+0.404 to +4.474
smoke2sf	19	16	3	epochs 5, 10, 15	−4.454 to +1.501

Why this table has changed shape (2026-09-19). The first version sampled epochs {20, 40, 60, 80} and reported "positive at all four sampled epochs" for both cells. With **every** checkpoint evaluated, that holds for shwd2sf (all 19) but **not** for smoke2sf, whose Δgap is negative at epochs 5, 10 and 15 (−4.454 / −1.089 / −0.058 pp) and positive at every checkpoint from epoch 20 on. The sampled grid began at epoch 20. The four sampled values the old table printed (+1.678 / +0.886 / +0.404 / +0.860 and +0.702 / +1.092 / +0.875 / +1.062) are unchanged and remain correct for those four epochs. Script: work/x3b_perepoch_corrected_20260918.py.

Table S24 — moved from *§9: the registered replication (the criterion is not met — the third row is the one that passes)*

Table S24. §9: *the registered replication (the criterion is not met — the third row is the one that passes)* — columns: Registered pair, n, Δ (pp), 95 % CI, paired t, permutation p, same sign, $\Delta \geq +0.30$, $p < 0.01$, $\geq 8/10$, unit criterion.

Registered pair	n	Δ (pp)	95 % CI	paired t	permutation p	same sign	$\Delta \geq +0.30$	$p < 0.01$	$\geq 8/10$	unit criterion
-----------------	---	---------------	---------	----------	---------------	-----------	---------------------	------------	-------------	----------------

---	--	---	---	---	---	---	---	---	---	---
	-									
T1-a dota15→aitod	1 0	+0.147	[-0.036 , +0.330]	1.816 (p = 0.1027)	0.123	6+/4-	✗	✗	✗	FAIL
T1-b aitod→visdrone	1 0	+0.168	[+0.083 , +0.253]	4.455 (p = 0.0016)	0.001953	10+/0-	✗	✓	✓	FAIL
T1-c visdrone→dota15	1 0	+3.727	[+3.534 , +3.920]	43.774 (p = 8.5×10 ⁻¹²)	0.001953	10+/0-	✓	✓	✓	PASS

Table S25 — moved from *§9: the registered replication (the criterion is not met — the third row is the one that passes)*

Table S25. §9: the registered replication (the criterion is not met — the third row is the one that passes) — columns: Question the frozen criterion asks, Measured.

Question the frozen criterion asks	Measured		
---	---		
\	Δ \	< 0.15 pp? (a saturated domain should be ≈ 0)	no (T2 mask→mendeley -1.490 pp)
p > 0.1? (should be non-significant)	no (paired t = -4.164, p = 0.0141 , 0+/5-)		
A positive gain $\geq +0.30$ pp with p < 0.01? (if present, the narrative collapses)	no		

Table S26 — moved from *§9: the near-matched control C (the gain does **not depend on a label-space change)***

Table S26. §9: the near-matched control C (the gain does not depend on a label-space change) — columns: n, Δ (pp), 95 % CI, paired t, permutation p, same sign.

n	Δ (pp)	95 % CI	paired t	permutation p	same sign
---	---	---	---	---	---
10	+0.727	[+0.640, +0.814]	18.854 (p = 1.53×10⁻⁸)	0.001953 (= the n = 10 floor)	10+/0-

Table S27 — moved from *9. The registered multi-target replication: transferred to the companion paper*

Table S27. 9. The registered multi-target replication: transferred to the companion paper — columns: Registered pair, data YAML, val, test, Split.

Registered pair	data YAML	val	test	Split
---	---	---	---	---
T1-b aitod→visdrone	visdrone_20p.yaml	images/val	images/test	separated ✓
T1-a dota15→aitod	aitod_20p.yaml	images/val	images/val	same directory ✗
T1-c	dota15_20p.yaml	images/val	images/val	same directory ✗

visdrone→dota15				
T2 mask→mendeley	mende_20p.yaml	images/val	images/val	same directory X
C dota→dota15 (control)	dota15_20p.yaml	images/val	images/val	same directory X

Where these configurations live. The four per-pair configs named above are archived under eval_validity/yamls/, each copy named after the path it had on its pod (for example A__root__datasets__AI-TOD_yolo__aitod_20p.yaml); that directory also carries a MANIFEST.md listing all 28 archived configs and, for each, whether val and test point at the same directory. The table above is a recomputation from those files, not a restatement.

Table S28 — moved from *5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus*

Table S28. 5.1 (the dota15 escalation, X8) — columns: seed, premium baseline, premium strategy, paired difference. Ten seeds, from n = 3 to n = 10. Premium = best-epoch minus final-epoch mAP50-95 on the run's own monitored split; source: the twenty run directories in x8_dota15_20260918/runs/ (8 of them trained for this round, seeds 43/44 pre-existing).

seed	premium baseline	premium strategy	paired difference
---	---	---	---
42	+0.997	+0.370	+0.627
43	+1.004	+0.183	+0.821
44	+0.841	+0.317	+0.524
45	+0.775	+0.585	+0.190
46	+0.997	+0.740	+0.257
47	+1.027	+0.658	+0.369
48	+0.986	+0.492	+0.494
49	+0.952	+0.407	+0.545
50	+0.941	+0.715	+0.226
51	+0.891	+0.454	+0.437
mean (n = 10)	+0.941 (sd 0.082)	+0.492 (sd 0.182)	+0.449 (sd 0.196, t = 7.23, p = 4.924e-05)

The criterion was fixed before the runs existed: a ten-seed paired difference failing to be positive with $\geq 8/10$ same-sign and a **95 % CI excluding zero** would remove dota15 from the four-cell headline and force the abstract and Tier 1 to be rewritten. Measured: **+0.449 pp**, **10+/-**, CI [**+0.309, +0.589**] — met as stated.

Recipe identity. All twenty args.yaml files of the cell (the ten new runs plus the two pre-existing seeds, both arms) are identical in every recipe field: same pretrain

(dota15_pretrain.pt), same data YAML (dota15_20p.yaml), 100 epochs, patience 100, batch 32,

imgsz 640, save_period 5, SGD, lr 0.001 (baseline) versus 0.005 (strategy). Old and new runs are therefore poolable, and the seed is recorded in every directory name.

GPU-memory fallback, and its sensitivity. 10 of the 16 newly trained runs printed

CUDA OutOfMemoryError in TaskAlignedAssigner, using CPU (92 events; 79 of them in

dota15_lr005_100ep_s51n), while the per-epoch sweep of §8.7 shared the GPU. This is an

implementation-level fallback inside one batch, not a change of objective, and it is reported

rather than smoothed: dropping each seed pair in turn leaves the criterion satisfied in all ten

subsets (worst lower bound +0.277), and dropping the two most affected pairs (s51: 79 events,

s42: 7) gives +0.428 pp, CI [+0.252, +0.605], 8+/0-. Scripts:

work/x8_dota15_analysis_20260918.py, work/x8_oom_counts_20260918.py,

work/x8_oom_sensitivity_20260918.py.

Table S29 — moved from *5.1 Direction: reported in two tiers, and the second tier's sign varies by corpus*

Table S29. 5.1 (the two `val`-overstating cells, XI) — columns: seed, Δ gap for p_aitovis, Δ gap for p_vistod15. Ten seeds per cell, from n = 3 to n = 10. Δ gap = (arm difference on val, at each run's own val-curve maximum) – (arm difference on test, at best.pt); paired within seed. Sources: x1_tier2_20260918/runs/ (40 run records) and teval.csv (80 test evaluations, SHA-256 verified against the machine that produced them).

seed	Δ gap p_aitovis	Δ gap p_vistod15
---	---	---
42	-1.127	-3.131
43	-0.843	-4.563
44	-1.358	-3.058
45	-1.495	-2.819
46	-1.267	-3.468
47	-0.960	-3.453
48	-1.280	-2.952
49	-1.341	-3.590
50	-1.404	-2.995
51	-1.182	-3.118
mean (n = 10)	-1.226	-3.315
95 % CI	[-1.370, -1.081]	[-3.676, -2.953]
paired t / p	-19.17 / 1.3e-08	-20.76 / 6.5e-09
sign	0+/10-	0+/10-
sign-flip permutation p	0.001953	0.001953

What the comparison is, exactly. The plan fixed before the runs: 40 training runs (seeds 42–51 $\times$ 2 arms $\times$ 2 cells), paired *and* sign-flip tests, {best, last} $\times$ {val, test}, and a prespecified correction; the strong branch of the pre-declared outcome rule was *"both remaining negative and significant with permutation $p \leq 0.002$ "*, the weak branch being that the abstract's "direction is not single-valued" must be withdrawn or limited. All ten seeds of each cell are negative and the permutation p is $2/2^{10} = \mathbf{0.001953}$, the smallest value attainable at $n = 10$ (§7.1), so the strong branch applies. **Holm across the two targets** (the prespecified correction) multiplies the larger p by 2 and the smaller by 1: 2.6×10^{-8} and 6.5×10^{-9} , both ≤ 0.05 .

The endpoint was identified, not assumed. The archived `sio_b_results.csv` that the published Table 2 used for the $n = 3$ values carries no endpoint convention (§7.3, disclosed defect 3). Our own evaluation of `best.pt` on `test` reproduces its values for the twelve runs it covers to **0.0046 pp** (median 0.0023) — so that archive records **the best checkpoint evaluated on `test`**, and the $n = 3 \rightarrow n = 10$ comparison is like-for-like rather than a change of measurement. Under the same construction restricted to seeds 42–44 the two cells give -1.110 and -3.584 against the published -1.114 and -3.580 .

Both arms and all four quantities. Per cell (baseline / strategy means, $n = 10$): `val-curve` maximum 11.242 / 12.657 and 16.963 / 23.927; `best.pt` on `test` 10.210 / 10.396 and 11.626 / 15.263.

The cells' realization rates (§8.6's quantity, computed the same way) are **82 % / 104 %** and **56 % / 29 %**, pooled **55 %** — the opposite end from §8.6's aggregate realization rate, which is why the article says

the realization rate is not a constant across corpora. The exact three-component decomposition of each Δgap is in Tables S10's conventions: for `p_aitovis` $-1.226 = (-1.583) + (+0.090) + (+0.267)$ and for `p_vistod15` $-3.315 = (-7.072) + (-0.213) + (+3.971)$; both identities check to the printed digit. Scripts: `work/x1_gap_analysis_20260918.py`, `work/x1_aux_20260918.py`.

Table S31 — moved from *3.2 Six experiments, five judged criteria, on GPU machines, each with its criterion written down first*

Table S31. 3.2 (*G2*) — the selection split's **size** is the only thing that changes: two nested validation splits drawn from one verified-disjoint pool (30 and 90 images), the same training set, and an identical test side. Columns: selection split, arm, n, prem_val, prem_test, WC, realization rate. Source: g2_valsize_20260919_runs/, g2t_g1p_20260919/g2_test.csv.

selection split	arm	n	prem_val	prem_test	WC	realization rate
---	---	---	---	---	---	---
small (30)	base100	5	+2.747	+0.646	+2.101	23.5 %
small (30)	lr005_100ep	5	+1.258	+0.064	+1.194	5.1 %
large (90)	base100	5	+2.549	+0.688	+1.861	27.0 %
large (90)	lr005_100ep	5	+1.062	+0.165	+0.897	15.6 %

Pre-declared criteria. Direction: the realization rate rises with the selection split's size — **met** (the two arms move the same way). Magnitude: WC_{small} / WC_{large} within $[1.4, 2.1]$ ($\sqrt{3} \approx 1.73$) — **not met** (measured 1.19, pooled over the two arms), so the $1/\sqrt{n}$ step is a scale estimate, not a law.

Table S32 — moved from *3.2 Six experiments, five judged criteria, on GPU machines, each with its criterion written down first*

Table S32. 3.2 (*G3*) — the epoch budget is the only thing that changes (four budgets, $E = 50$ –400, same corpus, recipe and validation split). Columns: arm, n, premium at $E = 50$, at $E = 100$, at $E = 200$, at $E = 400$, ratio 400/100. Source: g3_epochs_runs/.

arm	n	premium ($E = 50$)	premium ($E = 100$)	premium ($E = 200$)	premium ($E = 400$)	ratio 400/100
---	---	---	---	---	---	---
base	5	+0.724	+0.897	+1.187	+1.315	$\times 1.47$
lr005	5	+0.163	+0.468	+0.756	+1.241	$\times 2.65$

Pre-declared criteria (C1–C3, fixed before the runs). C1 — **monotone rise of the premium with E — is not met:** the four-budget series peaks at $E = 200$ and falls at $E = 400$ (four of the ten $E = 400$ runs stopped early at effective budgets 388/318/362/375; on the six full runs the $E = 400$ mean is +1.278 pp). The $\ln E$ regression over $E = 50$ –200 gives slope **+0.3990** (se 0.0592), **$p = 9.466e-08$** . C2 — **the $\sigma\sqrt{(2 \ln E)}$ form — is not rejected:** premium(400)/premium(100) = **1.329** (the $E = 400$ arm-equal mean over all ten runs is **+0.907 pp**) against the predicted 1.141, **inside** the pre-declared band $[0.86, 1.43]$. C3 — **the shape control — passes** ($\hat{e}/E$ 0.833 $\rightarrow$ 0.815). The extension ($E = 50$ and 400) was added for this revision; it uses the same corpus, recipe, seeds and arms as the $E = 100/200$ runs, and only --epochs differs. **The early-stopped runs decide the outcome:** the four $E = 400$ runs that stopped under the protocol's own patience (effective budgets 388/318/362/375) are what makes the series non-monotone; dropping them flips **both** criteria — the six full $E = 400$ runs run to a mean of +1.278 pp on a monotone series (C1 met) and move the ratio outside the band (C2 rejected). Both readings are conditional on that restriction and are reported as such; the pre-declared verdict above is the one computed on **all** runs. **This note supersedes an earlier reading of the two-budget series, which reported only three budgets and mislabelled them as monotone; the four-budget verdict is in S11 and §3.2. The table below is the with-early-stopping reading; the same four budgets re-**

run on one machine with early stopping disabled satisfy C1, put C2 outside its band and pass C3 on the pre-declared mean reading (only the lr005 arm exceeds it) — see Table S34b.

Secondary criterion C4 (WC on a disjoint readout) — not met, and systematically so. With $\hat{e}$ selected on val and read out on the disjoint third C3 (100 images; $C3 \cap val = \emptyset$ and $C3 \cap train = \emptyset$ verified programmatically), $WC = prem_val - prem_C3$ is **negative in all 16 readouts** (mean **-0.616 pp**): the peak-to-endpoint gap is *larger* on the small disjoint split than on val. That is consistent with the shape reading above — the premium is largely the split's own curve shape, and a 100-image split has a steeper curve — and it means Proposition 4's inequality **cannot be tested by reading out on a small disjoint curve**. Two limitations are stated with the result: the readout split has 100 images only, and the four early-stopped $E = 400$ runs are excluded, leaving $n = 6$ at that budget.

Table S33 — moved from *3.2 Six experiments, five judged criteria, on GPU machines, each with its criterion written down first*

Table S33. 3.2 (*G4*) — replication on a third corpus (mende20_3way, 2 arms $\times$ 10 seeds, test side evaluated separately, 40 evaluations). Columns: run, prem_val, prem_test, WC, realization rate. Source: g4_mende3way_runs/, g4_mende3way_20260919_g4_test.csv.

run	prem_val	prem_test	WC	realization rate
---	---	---	---	---
base100 s42n	+8.873	+3.384	+5.489	38.1 %
base100 s43n	+8.309	+2.701	+5.608	32.5 %
base100 s44n	+7.731	+3.783	+3.948	48.9 %
base100 s45n	+7.818	+2.796	+5.022	35.8 %
base100 s46n	+7.257	+2.481	+4.776	34.2 %
base100 s47n	+7.608	+3.666	+3.942	48.2 %
base100 s48n	+7.534	+3.046	+4.488	40.4 %
base100 s49n	+8.185	+4.312	+3.873	52.7 %
base100 s50n	+8.456	+3.834	+4.622	45.3 %
base100 s51n	+7.671	+2.930	+4.741	38.2 %
lr005 100ep s42n	+9.613	+4.893	+4.720	50.9 %
lr005 100ep s43n	+9.450	+3.434	+6.016	36.3 %
lr005 100ep s44n	+6.249	+0.416	+5.833	6.7 %
lr005 100ep s45n	+8.592	+2.205	+6.387	25.7 %
lr005 100ep s46n	+8.528	+1.960	+6.568	23.0 %
lr005 100ep s47n	+9.957	+5.044	+4.913	50.7 %
lr005 100ep s48n	+7.300	+1.684	+5.616	23.1 %
lr005 100ep s49n	+12.210	+4.314	+7.896	35.3 %
lr005 100ep s50n	+7.976	+2.983	+4.993	37.4 %
lr005 100ep s51n	+7.355	+2.563	+4.792	34.8 %

Result. $WC = +5.212$ pp, 95 % bootstrap CI [+4.822, +5.655], positive in **20 of 20** runs — the same direction as the 81 runs of §8.6 and the 40 of §5.1. No pass/fail is set for the rate itself (Proposition 4 bounds only its expectation).

Table S34 — cross-lineage realization test (non-YOLO detector)

Table S34. 6.5 — the mechanism outside the YOLO lineage. A **Faster R-CNN (ResNet-50 FPN)** detector

(torchvision fasterrcnn_resnet50_fpn), a family that **reports the final epoch by default** (the

cross-framework audit above), trained from COCO-initialised weights on the same **three-way protocol** as

the rest of this paper: train = a 20 % subset (1,294 images), selection (val) = a ****verified-disjoint** **carve**** (500), readout (test) = the corpus's held-out split (1,610). 40 epochs, batch 8, SGD lr 0.005

with a 1,000-iteration warmup, 640-px input, **six seeds**. best is chosen by the **`val`** curve's argmax; test is inferred **once** for best and once for last, and evaluated with the **same** COCO implementation used on the YOLO side (IoU 0.50:0.95, 101-point interpolation).

Source: nonyolo_realization.json (the original run) and nonyolo_settle_20260925/ (settle_all.json,

nonyolo_n6_aggregate.json; the five added seeds are aggregated by work/aggregate_nonyolo_n6_20260925.py).

The per-epoch val predictions kept by the training run are written one file per epoch under the run's prediction directory, from epoch 1 to epoch 40.

maxDets	n	prem_val mean	prem_test mean	realization rate (ratio of means)	95 % CI
---	---	---	---	---	---
100	6	+0.0809	+0.0551	68.2 %	[26.9 %, 123.5 %]
300	6	+0.0809	+0.0551	68.2 %	[26.9 %, 123.5 %]
1000	6	+0.0809	+0.0551	68.2 %	[26.9 %, 123.5 %]

Per-run detail (seed 42 is the run reported in the previous version; it is the *first* of the six, not a separate experiment). Rates are prem_test / prem_val per run, so they vary far more than the aggregate:

run	best ep	prem_val	prem_test	per-run rate
---	---	---	---	---
original (seed 42)	30	+0.0932	+0.0641	68.8 %
seed 43	27	+0.0753	+0.0422	56.1 %
seed 44	33	+0.0670	+0.0147	22.0 %
seed 45	24	+0.1017	+0.1777	174.8 %
seed 46	28	+0.0315	+0.0009	2.8 %
seed 47	21	+0.1165	+0.0310	26.6 %

Table S34b — the G3 clean re-run (early stopping disabled, one machine, four budgets)

Table S34b. 3.2 (G3, clean re-run) — the four-budget experiment re-run with the protocol's early

stopping **disabled** (injected at the call site; the training script is byte-identical, md5

316e074050dde23cde33e4a223ec46a8). Same corpus, recipe, two arms and five seeds as G3; only the

patience setting differs. The whole four-budget series was run **on one machine, on one stack**: the

earlier clean batch was cross-machine (E = 50 and E = 400 on one host, E = 100 and E = 200 on a second)

and one of its runs was silently degraded under memory pressure, which is why the series was repeated

rather than patched. **40/40 runs completed**, each to its configured budget. The E = 50/100/200 rows

coincide with the published budget series because no run early-stopped at those budgets; only E = 400

changes.

Source: g3cleanb_20260925/ (per-run results.csv; recomputation work/clean_series_b_20260925.py,

output work/_clean_series_b_20260925.txt).

E	arm	n	premium mean (pp)	seed sd	bootstrap 95 % CI	$\hat{e}/E$
---	---	---	---	---	---	---
50	base	5	+0.724	0.094	[+0.663, +0.806]	0.760
50	lr005	5	+0.163	0.176	[+0.030, +0.298]	0.876
100	base	5	+0.897	0.101	[+0.820, +0.974]	0.788
100	lr005	5	+0.468	0.220	[+0.290, +0.633]	0.878
200	base	5	+1.187	0.084	[+1.119, +1.250]	0.813
200	lr005	5	+0.756	0.163	[+0.628, +0.900]	0.909
400	base	5	+1.295	0.083	[+1.222, +1.349]	0.760
400	lr005	5	+1.248	0.085	[+1.180, +1.317]	0.814

Verdict under the pre-declared criteria, on the single-machine series. The arm-equal means are

+0.4439 / +0.6824 / +0.9712 / +1.2717, i.e. **monotone non-decreasing $\Rightarrow$ C1 is *met*** (per-run

regression on $\ln E$: slope **+0.3999**, se **0.0503**, t = **7.950**, p = **1.325×10^{-9}** , n = 40),

where the with-early-stopping reading scored **not met**. **C2 is out of band**:

$\text{premium}(400)/\text{premium}(100) = \mathbf{1.8636}$ against the predicted 1.141, outside the pre-declared

$[0.86, 1.43] \Rightarrow$ the $\sigma\sqrt{2 \ln E}$ scaling **stays withdrawn** and the level reads as shape-driven.

C3 — the shape control — passes on the pre-declared *mean* reading ($\hat{e}/E$ 0.833 $\rightarrow$ 0.787,

$|\Delta| = \mathbf{0.046} \leq 0.05$) and **fails per arm** ($\text{lr005 } |\Delta| = \mathbf{0.065}$; base $|\Delta| = \mathbf{0.028}$); the

pre-registered criterion names the mean, so both readings are reported side by side and neither is

dropped. **The registered criteria still forbid reporting the rise as an `E` effect**: the peak's

relative position moves **in one arm and not the other** across budgets (base 0.760 $\rightarrow$ 0.760, lr005

0.876 $\rightarrow$ 0.814), which is the movement C3 exists to catch.

Superseded reading, kept on the record (cross-machine batch). The first clean batch carried only

$E = 50$ and $E = 400$ and was split across two hosts; its per-arm values were

E	arm	n	premium mean (pp)	seed sd	bootstrap 95 % CI	$\hat{e}/E$
---	---	---	---	---	---	---
50	base	5	+0.672	0.180	[+0.526, +0.820]	0.744
50	lr005	5	+0.094	0.110	[+0.013, +0.185]	0.928
400	base	5	+1.298	0.082	[+1.223, +1.343]	0.836
400	lr005	5	+1.211	0.284	[+1.058, +1.469]	0.725

and its verdict read: *using the published $E = 100$ and 200 means (0.682, 0.971), the four-budget

series is **monotone** (0.383 / 0.682 / 0.971 / **1.255**) and the $\ln E$ regression gives slope

+0.419 (se 0.060), **t = 7.03**, **p $\approx 2 \times 10^{-12}$** $\Rightarrow$ the monotone-rise criterion **met**, where the

with-early-stopping readings scored **not met**. The **shape control fails**: $\text{'max_arm } |\hat{e}/E(400) -$

$\hat{e}/E(50)| = 0.203$ ` against the pre-declared 0.05, and the movement is **opposite in the two arms**

(lr005 falls 0.928 $\rightarrow$ 0.725, base rises 0.744 $\rightarrow$ 0.836). *Those values are replaced** by the

single-machine table above; they are retained so that the earlier reading can be checked against its

replacement. Two things about that batch are the reason for the re-run, and both are on the record:

it **mixed two machines**, and one of its runs was **silently degraded under memory pressure** (the

training path changed without a marker, recorded in this revision's run audit). Its shape control

also compared **E = 400 with E = 50**, because that batch had no $E = 100$ point — a mismatch of

comparison points against the pre-declared $\hat{e}/E(400) - \hat{e}/E(100)$, and the reason its "failure" and the single-machine "pass on the mean" are not in conflict.

Result. The val-selected premium is **partly realized** outside the YOLO lineage

($\text{prem_test} = +0.0551$ pp against $\text{prem_val} = +0.0809$ pp, 68.2 %, 95 % CI

[26.9 %, 123.5 %]), in the **same direction** as the five arm-cells of §8.6 — and in the same

direction in **all six runs** ($\text{prem_val} > 0$ and $\text{prem_test} > 0$ in 6/6; no run has a negative rate).

All three maxDets conventions give **identical** numbers, confirming (for this corpus) that the parameter is not binding here.

Pre-declared criteria for this escalation (fixed before the five added seeds were run):

① same direction as the original single run; ② a 95 % interval reported; ③ *if that interval contains zero, *the wording stays "an anecdote"*. ① met, ② met, ③ *not triggered** — the interval excludes zero — so this table is upgraded from an anecdote to an estimate **with** an interval.

Limits. One detector, one corpus, six seeds — so the result establishes that the

mechanism is **not confined to the YOLO lineage**, and **not** that non-YOLO pipelines realize ~68 % generally. The aggregate's agreement with the single-run value (68.8 %) is a ****coincidence** of a wide per-run spread: **individual rates run from 2.8 % to 174.8 %****, one of them above 100 %. The realization rate also has a **small denominator** (mean $\text{prem_val} = +0.0809$ pp; the original run's was +0.0932 pp), so the ratio is far more

sensitive to single-run variation than the directional statement $\text{prem_test} > 0$. The absolute premium is an order of magnitude below the YOLO-side cells of §8.6 (+0.865 ... +1.469 pp) and

must not be compared with them: the detector, corpus, dataset scale and absolute mAP level all

differ. The evaluation-side self-test (ground truth as predictions = **0.990099** at

``maxDets = 1000, 0.962854 at 100`) is reported in the project log for that pass (2026-09-16). Why it is not

exactly 1.0**, since the predictions fed in are the ground truth itself: the ceiling of this

self-test is **1/1.01 = 0.990099** by construction (COCO's 101-point interpolation), so the

maxDets = 1000 reading is *at* its ceiling rather than below it; the lower reading at
maxDets = 100 is the evaluator's own cap truncating a crowded image's boxes. Neither figure is
evidence about the detector — it is a check on the evaluation stack, and it is ****1.0** up to that
ceiling**.

Figure and table inventory (moved here from the article, B3, 2026-09-26). The article carries
its argument, its four headline tables and its disclosure; this is the map of which tables and
figures are printed there and which live here. It sits at the end of §S8 so that it directly follows
the moved tables it indexes.

Where the evidence lives. The article keeps its claims, argument, **five headline tables** and
its disclosure; the other tables are in the Supplementary **unchanged**, and **four of the six figures**
are printed in the article — §6.1 (Fig. 1), §8.6 (Figs. 2 and 4) and §8.7 (Fig. 3) — with the
remaining two in §S9, **and the printed numbers map to the deposited files as** `Fig. 1` =
`fig3\_four\_units.png`, `Fig. 2` = `fig2\_realization\_rate.png`, `Fig. 3` =
`fig5\_epoch\_curve.png`, `Fig. 4` = `fig6\_sigma\_vs\_rate.png`, `Fig. S1` =
`fig1\_dgap\_decomposition.png`, `Fig. S2` = `fig4\_public\_logs.png`, so a reader can pair every
caption with its file,

every number backed by an artifact pointer. Five tables are printed **in the article**
too, each carrying a headline number to check:

In this article	§S8 counterpart	What it settles
---	---	---
Table 1	Table S4	Tier 1 — premium per arm and the paired difference
Table 2	Table S6	Tier 2 — Δgap by corpus, with its reading
Table 3	Table S11	the four counting units over the 19 audited benchmarks
Table 4	Table S20	the premium selected on <code>val</code> , and what it realizes on <code>test</code>
Table 5	Tables S31–S34b	the six pre-registered experiments, their criteria and their verdicts

Everything else is in §S8–§S9, keyed by section, including **S24–S27** (the registration, now the companion's), plus **two tables this revision adds in §S12 rather than moves from the article** — **Table S35** (the estimand and scope map) and **Table S36** (the analysis-status ledger) — which the §S8 index does not list, since that index lists only the tables moved from the article; the six figures are:

Figure	Main-text section
---	---
Fig. 1	this article, §6.1

Fig. 2	this article, §8.6
Fig. 3	this article, §8.7
Fig. 4	this article, §8.6
Fig. S1	Supplementary §S9, §5.6
Fig. S2	Supplementary §S9, §5.5

§10's defect table is **kept in the article**, deliberately: disclosure belongs in the article, as do first-report attribution and methodological premises. The Supplementary's §S6 records the ****shared** process protocols****** — the same set of protocols used by this paper and its companion — with their file hashes.

S9 · Figures moved from the main text

Same reason as §S8. Captions are unchanged apart from the S in the figure number.

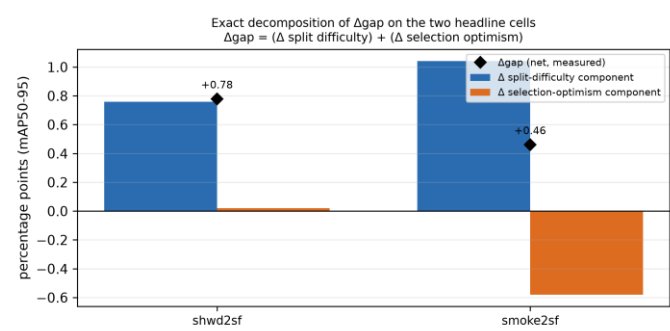


Fig. S1. Exact decomposition of Δgap into a split-difficulty component and a selection-optimism component on the two headline cells. The split-difficulty component dominates (+0.759 of +0.779 for shwd2sf; +1.041 against −0.580 for smoke2sf, where the two components partly cancel). This is an identity, not a fit: the three components sum to Δgap for all 13 cells. Source: gap_mechanism_20260916.txt, xeval_analysis_20260916.txt.

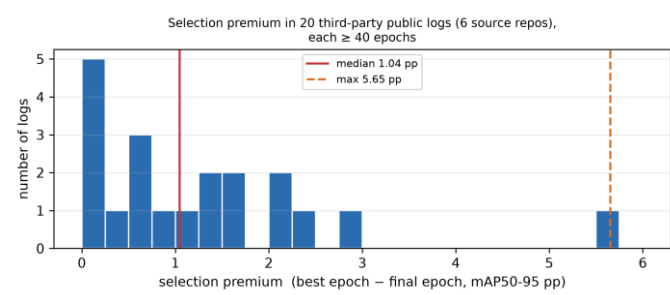


Fig. S2. Selection premium (best – final epoch, mAP50-95 pp) in the 20 complete third-party logs (≥ 40 epochs, 6 source repositories). Median +1.04 pp, maximum +5.65 pp, 18/20 positive, minimum +0.00. The same component measured on our own runs (main text §5.1) is present in logs we did not produce. Source: main text §5.5; figures/FIGURES.md.

S11 • Formalisation: propositions and proofs (companion to §3.1 of the article)

Notation as in §3.1 of the article: $V(e) = M_{\{S_{\text{val}}\}}(W(e))$, $T(e) = M_{\{S_{\text{test}}\}}(W(e))$, $\hat{e} = \text{argmax}_e V(e)$,

e_{f} = the final checkpoint, $\text{premium}_{\text{val}} = V(\hat{e}) - V(e_{\text{f}})$, $\text{premium}_{\text{test}} = T(\hat{e}) - T(e_{\text{f}})$.

Proposition 1 (non-identifiability of the raw gap)

Statement. $V(\hat{e}) - T(\hat{e}) = [V(e_{\text{f}}) - T(e_{\text{f}})] + [(V(\hat{e}) - V(e_{\text{f}})) - (T(\hat{e}) - T(e_{\text{f}}))]$; hence for any candidate difficulty term δ' the residual $o' = G - \delta'$ reproduces the observation exactly, and no estimator using only the endpoint pair $(\hat{e}, e_{\text{f}})$ identifies δ or o separately.

Proof. Substitute $V(\hat{e}) = V(e_{\text{f}}) + (V(\hat{e}) - V(e_{\text{f}}))$ and $T(\hat{e}) = T(e_{\text{f}}) + (T(\hat{e}) - T(e_{\text{f}}))$; the two expressions are equal identically. For the second claim, fix any $\delta' \in \mathbb{R}$ and set $o' = G - \delta'$; then $\delta' + o' = G$, which is the only equation the data impose. ■

Remark (which assumption buys identifiability). Differencing two arms with matched seeds gives

$\Delta \text{gap} = E[G_{\text{base}} - G_{\text{strat}}] = E[(\delta_{\text{base}} - \delta_{\text{strat}}) + (o_{\text{base}} - o_{\text{strat}})]$. The pairing assumption is

$E[\delta_{\text{base}} - \delta_{\text{strat}}] = 0$, which holds when the two arms are trained and evaluated on one corpus and one

split pair. Under an aliasing configuration both arms' δ are maxima of the *same* split's noise, so the assumption fails by construction: this is the formal content of §12.

Proposition 2 (exact three-component identity)

Statement. $\Delta \text{gap} = \Delta \delta_{\text{f}} + \Delta \text{prem}_{\text{val}} - \Delta \text{prem}_{\text{test}}$, where $\Delta \delta_{\text{f}} = (V(e_{\text{f}}) - T(e_{\text{f}}))_{\text{base}} - (V(e_{\text{f}}) - T(e_{\text{f}}))_{\text{strat}}$, $\Delta \text{prem}_{\text{val}} = (\text{premium}_{\text{val}})_{\text{base}} - (\text{premium}_{\text{val}})_{\text{strat}}$, and $\Delta \text{prem}_{\text{test}}$ is the

same for the realized premiums.

Proof. Apply Proposition 1 to each arm and subtract; the split-difficulty terms do not cancel but

combine into $\Delta\delta_f$, and the bracket terms are $\Delta\text{prem_val} - \Delta\text{prem_test}$. ■

Verification. The identity holds to the printed digit on all 13 scanned cells (Table S10) and on both escalated cells at $n = 10$: $-1.226 = (-1.583) + (+0.090) + (+0.267)$ and $-3.315 = (-7.072) + (-0.213) + (+3.971)$ (work/x1_aux_20260918.txt).

Remark 3 (the sign of Δgap does not identify a mechanism; not a separate result)

Statement. If the three components of Proposition 2 are of the same order, then for every target sign $s \in \{+1, -1\}$ there exist admissible component values whose sum has sign s ; hence the observed

sign does not determine which component dominates.

Proof. Formalise "of the same order" as $K/2 \leq |a|, |b|, |c| \leq K$. Take $(a, b, c) = (K, K/2, -K/2)$, whose sum is $+K > 0$, and $(a, b, c) = (-K, -K/2, K/2)$, whose sum is $-K < 0$. In both choices every component lies in $[-K, K]$ and has magnitude in $[K/2, K]$, so both are admissible under the hypothesis and both are of the same order. Since the target sign s was arbitrary, either sign is reachable. ■

Empirical support. Measured medians of the three components are **1.07 / 0.54 / 0.93 pp** (same order), the 13-cell scan contains **both signs** (ten positive, three negative; -2.46 to $+13.66$ pp), and candidate mechanisms M1/M2/M4/M5 are each refuted by sign mismatches (Table S9).
**Two

corrections of record.** ① An earlier version of the companion script tried to demonstrate reachability by resampling *cells*; that inherits the cells' own sign split (98.4 % negative on the four cells used) and tests nothing. ② A stronger claim — that the sign is *fragile* — is **not supported: **perturbing one arm-difference term by ± 10 % flips the sign in 0 %** of cells (± 50 %: 23 %), with a median cancellation ratio of **0.485**. The proposition is therefore stated as "the dominant term varies by corpus and the sign is a sum of same-order terms", which is what the

data support (work/theory_p3a_correct_20260919.txt).

Proposition 4 (realization rate = 1 – winner's-curse share)

Statement. Assume $V(e) = v(e) + \varepsilon_e$ and $T(e) = v(e) + \tau + \eta_e$ with $E[\varepsilon] = E[\eta] = 0$, ε, η independent of v , and η independent of the selection event $\{\hat{e} = e\}$. Then

$E[\text{premium_test}] = E[\text{premium_val}] - WC$ with $WC := E[\varepsilon_{\hat{e}} - \varepsilon_{\{e_f\}}] \geq 0$, so

realization rate $:= E[\text{premium_test}] / E[\text{premium_val}] = 1 - WC / E[\text{premium_val}] \leq 1$.

Proof. (i) $E[\text{premium_test}] = E[T(\hat{e}) - T(e_f)] = E[v(\hat{e}) - v(e_f)] + E[\eta_{\hat{e}} - \eta_{\{e_f\}}]$; the second term is 0 by the stated independence and mean-zero assumption. (ii) $E[\text{premium_val}] = E[V(\hat{e}) - V(e_f)]$

$= E[v(\hat{e}) - v(e_f)] + E[\varepsilon_{\hat{e}} - \varepsilon_{\{e_f\}}]$. (iii) Subtracting (i) from (ii) yields

$E[\text{premium_test}] = E[\text{premium_val}] - E[\varepsilon_{\hat{e}} - \varepsilon_{\{e_f\}}]$. (iv) Since $\hat{e}$ maximises $V = v + \varepsilon$, the selected

checkpoint carries positive noise in expectation, so $E[\varepsilon_{\hat{e}} - \varepsilon_{\{e_f\}}] \geq 0$; the equality case is selection on the signal alone. ■

Which assumption carries which half. The inequality ($\leq$) needs only " $\hat{e}$ is chosen using V " — that

is exactly the convention under audit, so the *direction* is robust. The closed form additionally needs η to stay out of the selection; under aliasing (val and $test$ the same split) the two noises coincide and the closed form does not apply — which is why §12 treats the aliased cells' realization

figure as a magnitude statement about bias, not as mechanism evidence.

Tests on the archive (`work/theory_p4_test_20260919.py`,
`work/theory_meta_and_tests_20260919.py`):

test	result
---	---
inequality per run ($WC > 0$)	+0.463 pp, $t = 8.34$, $p = 1.8 \times 10^{-12}$; realized premium smaller in 68 / 81 runs (sign test $p = 3.8 \times 10^{-10}$)
rate vs <code>val</code> noise (terciles)	65.6 % → 49.8 % → 13.8 % ; Spearman $\rho = -0.466$, $p = 1.3 \times 10^{-5}$, $n = 80$
premium level vs noise ($\sigma\sqrt{2 \ln E}$)	withdrawn : slope 0.026 ± 0.222 , $p = 0.82$, $n = 101$ — the premium's level is set by the curve's shape, not its jitter

Proposition 5 (family dependence of Benjamini–Hochberg)

Statement. For a fixed multiset of p -values and level q , BH's rejection set is determined by the step-up condition on $i/m \cdot q$; consequently the rejection count is not monotone under adding hypotheses:

adding sufficiently small p -values raises the thresholds behind them and can turn a previously

non-rejected hypothesis into a rejected one.

Proof. BH rejects $H_{(1..k)}$ for the largest k with $p_{(k)} \leq (k/m)q$. Since $(k/m)q$ is increasing in k , inserting new small p -values shifts existing ones to higher ranks k , which relaxes their thresholds; a hypothesis that failed at rank j can therefore pass at rank $j + t$. ■

Instance in this paper. Adding the two escalated cells' $n = 10$ p -values (6.5×10^{-9} , 1.3×10^{-8}) moved

shwd2sf from "does not survive" to "survives" (0.0053 against a line of 0.01154), and the 13-cell count went from **0 to 3** (work/recompute_bh_20260917.py). Declaring the family is therefore a precondition for the count to be meaningful, not a presentational choice.

Corollary to Proposition 4 (shape v. noise) — derivation and measurements

Derivation (exact). For one arm and one budget E let $V_r(e)$ be run r 's val metric at epoch e ,

$\text{premium}_r = \max_e V_r(e) - V_r(E)$, and $m(e) = \text{mean}_r V_r(e)$. Put

$\text{shape}(E) = \max_e m(e) - m(E)$ and $v_r = \text{premium}_r - \text{shape}(E)$. Then

$\text{premium}_r = \text{shape}(E) + v_r$ **by construction**: no assumption about the noise distribution is used.

Two immediate consequences: shape is a functional of the mean curve alone and therefore ****independent**

of σ^{**} ; and $E[v_r] \geq 0$, because replacing each run's curve by the mean curve can only lower the maximum — this is the val-side member of the winner's-curse family of Proposition 4.

Measurement 1 — the premium's level is shape, and the noise term is flat in E (same corpus, pretraining, two arms, five seeds; only the epoch budget changes; run as the **single-machine clean batch** with early stopping disabled — work/prop6_clean_b_20260925.py, the same pipeline as work/prop6_shape_noise_20260920.py and the generator of release_shape-noise-by-budget.csv):

E	premium (mean of arms)	shape	shape/premium		
---	---	---	---	---	---
50	+0.444	—	71.2 %		
100	+0.682	—	90.5 %		
200	+0.971	—	87.7 %		
400	+1.272	—	87.6 %		

Selection rule behind these four rows, and the `n` of each cell. The series is the

single-machine clean batch: one host (B), early stopping **disabled** (G3CLEAN_NOSTOP=1), and each

budget keeps only the runs that **completed epochs 1..E**; two arms (base, 1r005) $\times$ seeds 42–46,

so **n = 5 per arm** (8 arm-cells, 40 runs), the merged row weighting the two arms equally after

averaging within an arm. shape is the arm's **mean-curve peak minus its final value**, exactly as in

the derivation above. Recomputation and release file: work/prop6_clean_b_20260925.py $\rightarrow$

release_shape-noise-by-budget.csv (the same pipeline as work/prop6_shape_noise_20260920.py, the

entry point the reproduction map names, which computed the earlier cross-machine reading).

The with-early-stopping reading is not deleted: +0.907 over all ten E = 400 runs — four of them

stopped early, at effective budgets 388/318/362/375 — remains the reading the article's §3.1 corollary

carries, where it holds the pre-declared verdict, and Table S34b keeps the re-run's per-arm record.

Only this table's premium and share columns move to the clean reading, which is why E = 400 now reads

+1.272 and **87.6 %** and agrees with that corollary's primary reading (**+1.2717**).

Four-budget outcome (pre-registered criteria C1–C3). On the **with-early-stopping** runs the verdict is the pre-declared one and stands: C1 — monotone rise of the premium with E — is **not met**, the series peaking at E = 200 and falling at E = 400 (four of the ten E = 400 runs stopped early under the protocol's own patience, at effective budgets 388/318/362/375; on the six full runs the E = 400 mean is +1.278 pp). C2 — the $\sigma\sqrt{(2 \ln E)}$ form — is **not rejected**: premium(400)/premium(100) = 1.329 against the predicted 1.141, inside the pre-declared band [0.86, 1.43]. C3 — the shape control — **passes** ($\hat{e}/E$ 0.833 $\rightarrow$ 0.815). **The single-machine clean re-run (early stopping disabled, all four budgets present) gives a different verdict, and it is the one now reported:** arm-equal means **+0.4439 / +0.6824 / +0.9712 / +1.2717**, so **C1 is met** (monotone non-decreasing; per-run regression on $\ln E$: slope **+0.3999**, se **0.0503**, t = **7.950**, p = **1.325×10^{-9}** , n = 40); **C2 is out of band** (premium(400)/premium(100) = **1.8636** against the predicted 1.141, band [0.86, 1.43] $\Rightarrow$ the scaling **stays withdrawn** and the shape reading holds); **C3 passes on the pre-declared mean reading** ($\hat{e}/E$ **0.833** $\rightarrow$ **0.787**, $|\Delta|$ = **0.046** $\leq$ 0.05) while **failing per arm** (1r005 $|\Delta|$ = **0.065**; base **0.028**). Table S34b carries the per-arm values, the superseded cross-machine reading and the reason the series was re-run.

$E = 50 \rightarrow 200$: shape $\times 2.70$, premium $\times 2.19$, $sd(v) \times 0.92$; the $\sigma\sqrt{(2 \ln E)}$ form would predict $\times 1.24$.

The pre-registered decisive test of that form is criterion C2 (band [0.86, 1.43] on premium(400)/premium(100)); its verdict is reported in §S31–S33 and in §3.2.

Measurement 2 — where the realized gain goes (40 runs with per-epoch test readouts on a 5-epoch

grid, val from each run's own results.csv; selection on val only, readout on test only):

quantity	value		
---	---		
premium(val)	+0.981 pp		
premium(test), oracle peak on test	+0.704 pp (71.7 % of the val premium)		
premium(test), using the val argmax	+0.211 pp (21.5 %)		
runs whose val and test peaks agree	17.5 %		
median \	$\hat{e}_{val} - \hat{e}_{test}$		10 epochs (grid step 5)

So the low realization rate is a **peak-alignment** phenomenon, not an absence of gain: an oracle that

could pick the test peak would realize 71.7 % of the val premium, and val-selection retains 30 % of that.

Measurement 3 — selection rules (same 40 runs; every rule chooses on val, readout on test):

rule	realized test premium	captured share	cost to the reported number	within one step of the test peak
---	---	---	---	---
argmax (current convention)	+0.211	30.0 %	0.000	45.0 %
mean of top-2 val epochs	+0.406	57.7 %	0.690	55.0 %
mean of top-3 val epochs	+0.380	54.0 %	0.596	72.5 %
mean of top-5 val epochs	+0.490	69.6 %	0.727	62.5 %
argmax of a 5-epoch moving average	+0.459	65.2 %	0.526	70.0 %
final epoch (no selection)	+0.000	0.0 %	0.981	2.5 %

What these measurements are not. They are a **post-hoc comparison on the same runs**, not a pre-registered experiment: the rules were evaluated after the runs existed. The grid is 5 epochs wide,

so "within one step" means ± 5 epochs; all choices are made on val and every readout is on the other

split; and the oracle row is an upper bound, not an attainable protocol.

A candidate mechanism for the plateau rule — stated as a candidate, not as a result. The natural

explanation of Measurement 3 is that the argmax of a noisy curve is pulled towards epochs whose noise

happens to be high, so its underlying value sits below the true peak (the classical peak-shift penalty,

of order $\sigma^2/|v''|$), while a top-k average estimates the peak *region* with lower variance. That story makes a sharp prediction: the plateau advantage should **grow with the noise of the `val` curve**. We

tested it, and it is **not established**:

* noise estimator: $\hat{\sigma}_{\text{seed}}$ = mean over epochs of the cross-seed standard deviation at that epoch ($\sqrt{V_r(e) - \text{mean}_r V_r(e)}$), i.e. the run-to-run dispersion the ten-seed design is about — not the residual around a smoothing of one run's own curve (which mixes curvature into "noise" and, with that

estimator, gives $\rho = +0.04$, $p = 0.79$);

per run ($n = 40$): $\rho(\hat{\sigma}_{\text{seed}}, \text{plateau} - \text{argmax}) = +0.309$, $p = 0.052^*$;

per cell ($n = 4$ cells, the honest counting unit): $\rho = +0.800$, slope $+0.406 \pm 0.101^*$ per pp of

$\hat{\sigma}_{\text{seed}}$, $r = +0.943$, but **$p = 0.057$** ;

* the four cells read: $\hat{\sigma}_{\text{seed}}$ 0.629 $\rightarrow$ +0.194 pp, 0.639 $\rightarrow$ +0.154 pp, 1.125 $\rightarrow$ +0.318 pp, 1.201 $\rightarrow$ +0.447 pp —

i.e. both noisy cells show about twice the advantage of both quiet cells, with no exceptions.

What this is and is not. The *advantage* of the plateau rule is established (Measurement 3, and the

paired test +0.278 pp, $t = 3.678$, $p = 7.1 \times 10^{-4}$, positive in 26 of 40 runs). The *mechanism* is

consistent but not significant on four cells, so we report it as a candidate and not as a result.

Settling it needs more corpus cells with per-epoch test readouts — the direction of the effect is already pinned down by the four we have, but four points cannot carry a significance claim.

13.2 §3.2: the per-comparison reasoning, moved from the article

Where the bias does move, it moves with what the selection sees: the split-size comparison held the test

side fixed and varied only the size (both arms moving the same way), and the epoch-budget comparison left

the peak's relative position essentially unchanged, so neither is a shape artefact. The third-corpus replication evaluated its test side separately and points the same way as the 81 runs of §8.6 and the 40

of §5.1.

Sources: `work/g1p_verdict_20260919.py`, `work/g2_and_g1p_verdict_20260919.txt`,
`work/g1pp_g3_g4_verdicts_20260919.txt`, `work/theory_p4_test_20260919.py`.

8.7b The per-epoch sign detail, moved from the article

shwd2sf's Δgap is positive at **all 19 checkpoints**; smoke2sf's is ****negative** at the first

three — epochs 5, 10 and 15 — and positive at all 16 checkpoints from epoch 20 onward**;

the four-epoch grid began at epoch 20, so it could not have seen this. At epochs 60 and 80 the val-side

arm difference is $-0.176 / -0.384$ and the test-side one $-1.051 / -1.445$ — ****reproducing the published**

values to the printed digit — **while Δgap stays positive**: the arm ordering flips on both splits at once, their difference does not.**

13.2b The four-budget detail, moved from the article

Four of the ten $E = 400$ runs stopped early under the protocol's own patience (effective budgets 388/318/362/375); on the six that ran to 400 the $E = 400$ mean is $+1.278$ pp — and that one restriction

flips both criteria: on the six full runs the series is monotone (C1 met) and

$\text{premium}(400)/\text{premium}(100) = 1.873$ falls *outside* the band (C2 rejected, the shape reading restored).

5.4 The two external critiques, in full (moved from the article)

The correction the critique forced: the paired-difference form cancels the difficulty term under the pairing assumption stated in §3.1, so the quantity we report is identifiable where the raw

$M_{val} - M_{test}$ is not.

13.3 The peak-agreement predictor (companion to §3.2 of the article)

Setup. The released per-run file carries, for each of the 40 paired runs, the validation peak (e_{val_peak}), the test peak (e_{test_peak}) and the two premiums. Both peaks are taken on the **same 20-point checkpoint grid** (0, 5, 10, ..., 95); that is the convention under which $same_epoch$ is meaningful, and it is verified here: recomputing both peaks on that grid reproduces the released e_{val_peak} **40/40**, e_{test_peak} **40/40** and $same_epoch$ **40/40**.

Per-run values for the group where the peaks agree (the other 33 runs' values are the released columns themselves):

run	e_{val_peak}	e_{test_peak}	Δgap
---	---	---	---
r10_shwd2sf_base100_3way_s49n	75	75	+0.0561
r10_shwd2sf_lr005_100ep_3way_s45n	75	75	+0.4099
r10_smoke2sf_base100_3way_s42n	80	80	-0.2144
r10_smoke2sf_base100_3way_s46n	85	85	-0.5838
r10_smoke2sf_base100_3way_s47n	65	65	-0.2019
r10_smoke2sf_base100_3way_s49n	75	75	-0.2174
r10_smoke2sf_lr005_100ep_3way_s43n	85	85	+1.6649

Group summaries. Peaks agree ($n = 7$): mean Δgap **+0.1305 pp**, median **-0.2019 pp**, four of seven non-positive. Peaks differ ($n = 33$): mean **+0.9056 pp**, median **+0.9537 pp**, four of thirty-three non-positive. Across all 40 runs Δgap spans **-0.7724 to +2.0454 pp**.

Permutation test. Reassigning the agreement label at random across the 40 runs 20 000 times, the absolute difference of the two group means is at least the observed **|0.7751| pp** in **0.0091** of draws (seed 42, `random.shuffle`); the agreement indicator's AUC for $\Delta gap > 0$ is **0.2969**, with a two-sided permutation p of **0.0196**.

The same relabelling on the median, which is the estimator these groups actually support.
The

agreement groups are separated far more clearly by their medians than by their means. Observed $|\text{median}(\text{agree}) - \text{median}(\text{differ})| = |-0.2019 - +0.9537| = \mathbf{1.1556\ pp}$; 20 000 relabellings of the 40 Δgap values (seed 42, same convention) reach that or more in **0.0011** of draws (seeds 7 and

2026 give 0.0009 and 0.0014; the lower-median definition gives 0.0006). **The group means are the

weaker summary, and the mean of the agreement group is carried by one run**: the agreement group's

mean is **+0.1305 pp** on a group sum of **+0.9134 pp**, of which the single run

r10_smoke2sf_1r005_100ep_3way_s43n contributes **+1.6649 pp**; dropping it moves the group mean to

-0.1252 pp while leaving the median at **-0.2019 pp**, and the median-difference permutation p becomes **0.0007**. **The reading this licences is a lower median Δ gap under agreement — not "the

premium is realized almost in full"**: four of the seven agreement runs are non-positive, and the summary that survives the single-run sensitivity is the median. Script (read-only, no retraining):

work/r98_median_perm_20260927.py, over release_selection-transfer_perseed.csv and xeval_perepoch_20260918/matrix_perepoch.csv.

What this table is not. The agreement group is **not** a group in which the premium is realized: its **median Δ gap is negative** and four of its seven runs are non-positive, so agreement is a condition under which the *mean* deviation is small, not a guarantee that the selected checkpoint carries over. The result is also a **partial** predictor: it conditions on one observable and leaves the sign's variation inside each group unexplained, it rests on **7 runs** in the agreement group, and it is defined **only on the 5-epoch grid used here**, which was not refined.

Sources: work/partial_predictor_final_20260922.py (grouping, permutation test, AUC),

work/probe_evalpeak_convention_20260922.py (the grid convention and its 40/40 reproduction); the

grid and premium columns are the released matrix_perepoch.csv and

release_selection-transfer_perseed.csv.

S12 · Method detail moved from the article

This appendix holds the measurement, protocol and record detail whose **conclusions stay in the article**. It is moved rather than cut: a reader who wants to check a claim reads the same measurements here, while the article keeps one sentence at each site. Identifiers that describe one

machine's internal bookkeeping have been replaced by descriptors; the measurements themselves are unchanged.

3.1 Formalisation — the statements displaced from the article

§3.1 of the article now keeps each statement in compressed form, the adoption declaration, the two testable

consequences of the adopted identity, and the corollary; the full statements with their derivations and

proofs are in §S11. What the compression displaced is reproduced here **verbatim**, so that no number and

no qualification leaves the paper — including the attribution sentence that separates what the generalization-gap literature establishes from what this paper states as its own formalisation, and the

disclosure that the sign of Δ_{gap} is not claimed to be *fragile*.

Definition (the raw gap is not identifiable from the pair alone). $V(\hat{e}) - T(\hat{e}) = [V(e_{\text{f}}) - T(e_{\text{f}})]$
+

$[(V(\hat{e}) - V(e_{\text{f}})) - (T(\hat{e}) - T(e_{\text{f}}))]$, i.e. $\mathbf{G} = \boldsymbol{\delta} + \mathbf{o}$ with δ a split-difficulty term and $\mathbf{o}$ a

selection-optimism term. **The additive decomposition, and the qualitative reading that a single-model endpoint pair cannot

separate its two terms, are established in the generalization-gap literature [20]; **its** strong

form* — *that* any* δ' fits the observation equally well, so that no endpoint-pair estimator

separates them [22] — is **stated here as a formalisation rather than attributed to that

literature. The pairing assumption is what makes Δ_{gap} meaningful**:

with matched seeds and one split pair, $E[\delta_{\text{base}} - \delta_{\text{strat}}] = 0$, and only then does the arm difference

identify the difference in selection optimism. Under an aliasing configuration that assumption

fails by construction — the two arms' δ are two maxima on the same split — which is why §9 cannot

be read as a clean-protocol result.

Adopted identity, not a claim of this paper (the realization rate is one minus the winner's-course share). With $V(e) = v(e) + \varepsilon_e$

and $T(e) = v(e) + \tau + \eta_e$, $E[\text{premium_test}] = E[\text{premium_val}] - WC$ where $WC := E[\varepsilon_{\hat{e}} - \varepsilon_{e_f}] \geq 0$,

so the realization rate is $1 - WC / E[\text{premium_val}] \leq 1$, with equality only if selection is driven by signal alone [16] — Smith & Winkler's optimizer's curse written in this paper's notation; **we adopt it as the frame for §8.6–§8.7 and claim no part of it.** The inequality needs only that $\hat{e}$ is chosen on val (the convention under audit); the identity

additionally needs the test noise not to enter the selection. **Two consequences, both tested on the archive** — the per-run inequality and the monotonicity in the val noise; per-run values, terciles and rank correlation: **Supplementary S11**. This is why §8.6's realization rate and §8.7's per-cell variation are the

same statement at different noise levels, not a contradiction. A third form — that the premium scales as $\sigma\sqrt{2 \ln E}$ — was **tested and withdrawn**; the regression and its diagnostic are in **Supplementary S11**. Contemporaneous work measures $\approx 5\times$ on the same $\sigma\sqrt{2 \ln K}$ bound [21].

Proposition 2 (exact three-component identity). Differencing the definition across arms gives

$\Delta\text{gap} = \Delta\delta_f + \Delta\text{prem_val} - \Delta\text{prem_test}$ — an identity, not a fit, holding value by value on all 13

scanned cells and both escalated cells at $n = 10$ (§5.6).

Remark (the sign of Δgap does not identify a mechanism; not a separate result). Three measured facts carry this:

the components are of the same order (medians **1.07 / 0.54 / 0.93 pp**), so which dominates

differs by corpus; **both signs occur** in the scanned cells (per-cell table: Table 2, Supplementary S6); every candidate mechanism we could

state is refuted by a

sign mismatch (M1/M2/M4/M5, Table S9). **What we do not claim** is that the sign is *fragile*: a perturbation analysis (numbers in **Supplementary S11**) leaves it stable under a $\pm 10\%$ change of a single arm-difference term, so the honest statement is "the dominant term varies by corpus and the sign is a sum of same-order terms", not "the sign is noise".

Proposition 5 (Benjamini–Hochberg depends on the declared family). The step-up thresholds are

$i/m \cdot q$, so adding hypotheses with smaller p-values **raises** the thresholds behind them and the

rejection count is not monotone in the family (proof: **Supplementary S11**). We are ourselves an

instance: escalating two cells to $n = 10$ moved the 13-cell count from **0 to 3** surviving cells

(§5.1) [23] — so declaring the counting unit and the family is a condition for the number to mean anything.

§3.2 as worded in the previous revision — the per-experiment criteria and their values, replaced in

the article by the verdict list plus the Table S31–S33 pointer. Kept here verbatim for the same reason:

The propositions above are testable, so we ran them on rented GPUs with the ****criterion** fixed before the

runs. **Six experiments were run**; five of them had their pre-declared criteria judged — **G1 is void (its design was diagnosed as confounded), so no verdict is entered for it**. Two of the five judged outcomes are negative** (G1' and G2) and one design had to be redone — reported as they came out.

The six experiments, their **pre-declared criteria** and how each came out are tabulated in

Supplementary S31–S33: G1 **void** (the two splits differ by 14.7 pp in difficulty on the same checkpoints, so its -0.582 pp mixes the effect with a level effect); G1'' **met**; G1' **not met** (WC = -0.069 pp, $t = -0.23$, $p = 0.83$, $n = 10$ — no inflation detectable); G2 ****direction met**, magnitude not met; **G3 met (and, in the clean re-run without early stopping, C1 flips to met while the shape control fails — S34)**; **G4 met (WC = $+5.21$ pp**, CI [$+4.82, +5.66$], 20/20 runs positive)**. Where the outcome is negative it is reported as negative.

The criteria that were met and those that were not carry one message: ****the selection bias moves with**

the absolute amount of selection noise and with the number of selection opportunities, but not with

whether the two splits are the same file.** Two consequences do not follow from the numbers alone: the

configuration-level fact of §4 stands — the reported number *is* the selected number — while the stronger reading, that aliasing *adds* a measurable amount on top of it, does not. That identity's WC

is best read as the winner's-curse share of the selected gain, a quantity set by the noise. **That reading yields a partial predictor of Δ gap's sign, which we test here**. On the shared 5-epoch checkpoint grid the val and test peaks agree in only **17.5 %** of runs, and conditioning on that single observable splits the 40 runs cleanly: where the peaks agree the val-selected premium is

realized almost in full (mean **+0.13 pp**, $n = 7$), and where they differ it is **systematically not** (mean **+0.91 pp**, $n = 33$; permutation $p = 0.009$). **This is a partial predictor, not an explanation of the sign, and its limits are part of it:** the agree-group contains four of seven runs whose gap is non-positive, so agreement is not a guarantee of realization; n is small; and the split is defined on the 5-epoch grid, which we did not refine. The per-run table and the permutation test are in **Supplementary S11**. The

per-comparison reasoning and the sources are in **Supplementary S31–S33**.

5.3 The dota15 cell: now reproducible

The local version repaired it for **two reasons**: a stale root snapshot (trees only

up to 2026-09-14; it adds two *_results_20260915 archives), and **the

matcher could not see unseeded runs** — prefix + '_s' matched nothing while

dota15_base100 and dota15_lr005_100ep (**no seed suffix**) exist, so it reported

no runs found forever. The seed suffix is now optional.

It agrees **value by value** with the published readings: three paired best-endpoint

differences **+1.12 / +0.95 / +0.93**, mean **+0.999 $\approx$ +1.00** (last endpoints

+1.77 / +1.46 / +1.54 — again larger for the baseline arm, as in §5.1

Tier 1).

A wording correction, retracted (2026-09-21). An earlier version of this note said that only two of the three runs carried a seed. **That is wrong.** The third run's seed is **s42** — the campaign's fixed --shuffle-seed 42, recoverable from its recorded configuration; what it lacks is the **_s42n suffix in its directory name**, not its seed identity. The correct statement is **three runs on seeds s42/s43/s44, the third's directory name omitting the suffix** — a naming convention, not a missing seed. Each of the three readings **+1.12 / +0.95 / +0.93** is now owned (the suffix-less pair contributes the **+1.12** best-epoch reading). The companion made the same correction and kept its own anchor count at 567/567.

Status after the §5.1 escalation (2026-09-19). The cell now exists as **ten seeded runs**

(seeds 42–51: dota15_base100_s*n / dota15_lr005_100ep_s*n). All twenty args.yaml files —

including the two pre-existing seeds — are identical in every recipe field (same pretrain, same

YAML, 100 epochs, batch 32, imgsz 640, SGD, lr0 0.001 vs 0.005), so the old and the new runs are

poolable; and the seed now appears in every directory name. **The wording defect above stays on the record as a defect of the old three-run cell, not of the row now printed.**

9.3 The seed convention: how the ten seeds are realized

Per-run randomness is installed as a **deterministic permutation of the training file order**

(`--shuffle-seed`, set to the number in the run's own name), not through the framework's `seed` argument: ultralytics 8.4.120's data pipeline does not draw from `args.seed`, and on two of these corpora different `--seed` values produced **identical** epoch-by-epoch replays.

The same convention and the same check hold for the cells escalated in §5.1: all twenty `args.yaml` of the dota15 cell record `seed: 42` while its ten per-seed premiums are ten distinct values, and the forty runs of the two escalated Tier-2 cells likewise give ten distinct values in each arm-cell. Script: `work/check_seed_convention_20260918.py`.

5.4 Two external critiques, checked (one refuted, one judged non-identifiable)

Full report: the two-critique verification record; script

`analysis/work/verify_two_critiques_20260916.py` →
`eval_validity/verify_two_critiques_20260916.txt`.

(I) "The premium difference may be noise-level" — refuted. The critique compared it against **0.515 pp**,

the **single-run** data-order component of §8.4, and concluded "underpowered, not significant". The

quantity under test is a **difference of arm means**, with uncertainty the SE of that difference (**0.058–0.120 pp**); the *t* and *p* values of §5.1 Tier 1, and their four CIs excluding zero, settle it.

The wrong denominator was used; the data are sufficient.

(II) "You should report $M_{val}(\hat{e}) - M_{test}(\hat{e})$ directly" — judged non-identifiable. That quantity mixes

the two splits' difficulty with selection optimism and is not identifiable here; we report the ****paired**

difference Δ_{gap} **instead and** state honestly that its sign is not uniform (§5.1 Tier 2). A hard fact that came with it:** in

`shwd2sf` the arm ordering on `val` is the reverse of that on `test` — see §5.1.

What these two checks changed. We used to write "the problem **understates** rather than manufactures a

difference between arms"; (II) corrects this to "***it systematically shifts arm-to-arm comparison, and the

direction varies by corpus**" — the abstract, §2.1, §2.2 and §5.1 rewritten accordingly.

5.5 The same component in third-party public logs

Sections 5.1–5.4 use **our own** 569 runs; this asks whether the component ****exists** in other people's

published logs**.

Sample. 24 per-epoch training logs; the **20

complete logs (≥ 40 epochs), from 6 source repositories**, are analysed. The definition is identical to ours:

$\text{premium} = \text{best-epoch mAP50-95} - \text{final-epoch mAP50-95}$. Ultralytics-style logs report the

best-validation checkpoint, so this difference is **an upper bound for** the component contributed by selecting on the

reported split.

Table S7 → Supplementary §S8.

What matters most. The largest premium (**+5.65 pp**) occurs in an **ordinary published log** (a 47-epoch

segmentation run; final epoch 0.6521, best epoch 42 at 0.7086). Beside this paper's own magnitudes —

+1.434 pp (the companion paper's reading, §8.1) and a published controlled ablation of **0.040 mAP50-95**

(§2.1) — **the premium's long tail is larger than both**, its median of the same order as the former.

(A magnitude comparison, **not** a same-unit subtraction: a within-run selection difference versus a between-arm effect.)

What it can and cannot support. Can: the component exists in **other people's** published logs, with a

per-log magnitude (all 20 → Supplementary S5).

Cannot: a **field-level** distribution — 20 logs from 6 sources, most not stating dataset or protocol.

The sample is a convenience sample, not a frame: logs came from public dataset hosts and one code repository, filtered only by the inclusion rule above; no platform-side query is named because none was systematic. **Any quotation of these magnitudes** (§2.2, §5.5) carries the four limitations listed below.

Verifiability (upgraded to "byte-checkable *and* traceable"). Each log's **SHA-256 and byte count are in ``published_logs/SHA256SUMS.txt``; at source level, 24/24 files** matched to a

source path by **content hash or unique byte count** — 8 Kaggle datasets plus 1 GitHub repository

pinned at commit 2417aff0457c, with **0 untraceable entries** and **24/24 agreement** with SHA256SUMS.txt.

Four limitations:

1. **It cannot establish aliasing**. Public logs carry **no YAML**, so we cannot tell whether that run's `val`

and test share a directory. **"The premium exists"** and **"the benchmark aliases"** are two independent

pieces of evidence and cannot substitute for one another** — this section supports only the former (§4, §6 the latter).

2. **The exact downloaded version is unprovable**. What is provable is **"the bytes we analysed equal these**

hashes**", not **"these bytes equal the Kaggle version at the time"** (the source table records

`currentVersionNumber at probe time: ikram0703 2`, the rest 1).

3. **The endpoint reading is an upper bound**: repositories rarely record which epoch was reported, so we

assume it was the best.

4. **Licensing: 5 sources do not state a licence** (Kaggle Unknown), and the GitHub source's LICENSE

was not retained. We **do not redistribute** these CSVs, only hashes and source pointers; derived statistics should be cited with that caveat.

A defect we must record (one silent exclusion). The first script version identified metric columns by the

substring 'mAP50-95' in column, recognising **only the new Ultralytics spelling**, so the YOLOv5/v3

spelling metrics/mAP_0.5:0.95 used by ayyappakorla-yolov3-results made read_log return None and the

file **disappeared from the sample with no diagnostic anywhere**. It is a

200-epoch complete log (final epoch 0.9011, best epoch 198 at 0.9041, premium **+0.31 pp**). After the fix

the sample went **19 → 20 logs, 5 → 6 sources**, the median **+1.18 → +1.04 pp**, and positives **17/19 → 18/20**.

Three conventions adopted from that failure (project-wide): ① annotate per row ****which column**

spelling **the log uses**; ② a "logs that could not be parsed must be listed" **section** — a

guard may exclude, but it may not exclude silently; ③ **read the best epoch from each log's own epoch**

column**, not the row index (most start at 1, the ayyappakorla one at 0).

The four logs not analysed run for 30, 20, 5 and 1 epoch; the

5-epoch one's "best – mean of last five" gives a meaningless +12.94 pp — ****which is why a minimum length must**

be imposed**, and we report only max – final, for 20 logs.

Relation to §5.1. This section supplies the existence and magnitude of the same component in ****other**

people's **data**. It does not enter** §5.1's two-tier conclusion (whose evidence is our own paired design), and the two bodies of evidence do **not** corroborate one another.

6.1 Which reported number the COCO row judges, and this table's first-version correction

Table S11 → Supplementary §S8.

Which number the `reported` unit judges. For one row — COCO — it depends which reported number is

meant: community tables report the **server-scored** ``test-dev`` value, which no local run can be selected

on (we count it independent), while a paper's own ablations report its ``val`` value, which is also the

selection split. First reading: reported **12/19**, clean **3/19**; second: **13/19** and **2/19**. No

other row is affected; the abstract prints the first.

Correction to the first version of this table. It ****put 2/19 under**

yolo_dist, contradicting the recomputation: ``yolo_dist`` is 4/19 (PASCAL VOC and SFCHD's literal

two-key **alias, SHWD's ``train == val``, Mendeley's generator alias)**, 2/19 being only the subset** the audit

reported. The table makes **a unit a whole column**, ordered as §3's four definitions.

6.2 "Same directory" has four levels, and the audit reported only the narrowest

Table S12 → Supplementary §S8.

L1 is what the audit enumerated; L3 carries most of the *reported* layer's non-independence.

6.3 We do not reproduce the original audit's two numbers

Table S13 → Supplementary §S8.

6.5 Across frameworks: is the convention specific to YOLO?

Is "the reported metric comes from a non-independent split" a YOLO writing problem, or the default of the

field? §6.1–§6.3 rest on the **generic YOLO distribution package**.

We retrieved, at **pinned commits**, the official configurations and training scripts of **five pinned repositories — 25 files**,

covering six frameworks in total, since the Ultralytics distribution package is the `yolo_dist`

unit of §6.1 and is supplied separately (SHA-256 and byte counts per file in `_xframe/SHA256SUMS.txt`; pins in

`PINNED_COMMITS.txt`; script `work/analyze_xframe_20260916.py` → `xframe_evidence_20260916.txt`), then

read mechanically which split each framework **reports** on and whether that split is **also** used for checkpoint selection.

Table S14 → Supplementary §S8.

Two conclusions: one widens the problem, one narrows our own claim.

****(C-1, widening)** Reporting on val is the shared default of all six frameworks, because of annotation availability. ****** COCO's test annotations are not public — Detectron2 writes the test split's

annotation file as image_info_test2017.json (**image information only**), PaddleDetection reads the

same val annotation file through a ground-truth-free ImageFolder. This is why the reported-layer count is what it is: where annotations are not public, the default

path is to report on whichever split can be scored locally, and that split is usually also the

selection split. **COCO sits on the boundary** — its test-dev is server-scored and cannot be selected on, so we count its community-reported number as independent, while a paper's own ablation numbers are not; §6.1 states both readings and both counts. ******On benchmarks like COCO the

coincidence is not an author's oversight — annotation availability pushes everyone onto the same split******, the default path itself.

****(C-2, narrowing)** But "select the checkpoint on that split" is not the shared default — only the YOLO

lineage does it. ****** Ultralytics (best.pt) and YOLOX (best_ap) select by the validation metric; Detectron2 (EVAL_PERIOD = 0), MMDetection (no save_best), DETR (no best tracking) and PaddleDetection (none found) **report the final epoch**. §5's selection-premium claim therefore reads:

******"within the pipeline family that selects by the validation metric by default (the YOLO lineage), the

convention systematically shifts arm-to-arm comparison" — not****** "the default of the field". The abstract and the §2.1 table claim about YOLO distributions only, consistent with this.

Four limitations of this section (so it is not read as a census): ① we read ******default

configurations, **which users may change**; ② **only** 5 pinned repositories / 25 official files — fixed-point sampling,

not a census; ③ **the pins are** default-branch heads****** and will move, hence the SHA-256 and commit per

file; ④ as in §4 and §6, a **configuration-level audit** — **no dataset was downloaded**.

8.1 The protocol

Split the pool three ways — a training subset, a **carved validation** subset for selection, a **test** subset for reporting; carved and test are disjoint, so the reported number is no

longer the maximum a selection rule saw. Three requirements make it trustworthy:

carve out of the pool (the test split was never in training), ****verify all three intersections are zero (by filename), and freeze the endpoint definition in writing**** — which checkpoint counts as "the"

reported number, whose omission is defect 3 in §10 and the source of the conflict in §7.3.

8.3 A direct measurement of selection inflation

Extending the same cell from $n = 6$ to $n = 9$, the mean falls **systematically**:

Table S18 → Supplementary §S8.

The magnitude is 9 %–29 % of the headline effect. ****The estimand is the change in the cell mean when the seed count rises from 6 to 9**** (two nested counts of the same runs; no new

training), so this measures **seed inflation**; reading it as "how much was selection rather than signal" is an interpretation we do not test here.

8.4 The three variance components: the excluded one is the smaller in this sample

Table S19 → Supplementary §S8.

Reading (favourable to this paper): on the same unit, ****initialization (0.146 pp) fluctuates less than**

data order (0.515 pp) — the component we excluded is not the dominant one** — so "the reported power is

an optimistic upper bound" tightens to "the excluded initialization component is smaller than the dominant

seed component".

Two limitations must be stated with it: ① only 3 initializations, so the SD estimate is very uncertain (with 2 degrees of freedom the 95 % interval for σ is roughly $[0.5\hat{\sigma}, 6\hat{\sigma}]$); we may write

"consistent with", **not** "established". ② That experiment ran on machine A with A-side third-party YAMLS,

the $n = 10$ cells on machine B under the same protocol; cross-machine comparability must be noted.

8.5 Why "same-pipeline pairing" is a necessity, not a habit

Three initializations move the **absolute endpoint** of one configuration by 43.250–43.630 (range 0.38 pp, SD 0.209 pp), comparable to seed noise. ****Absolute endpoints are not comparable across source**

endpoints; only paired differences within one source endpoint are stable.**

5.6 Why Δgap 's sign varies by corpus — measurement detail

§5.1 Tier 2 reports the **fact**; with the archived quantities

the mechanism is not identifiable, though three candidates are **refuted** and two unsupported.

Data: 13 three-way corpus cells $\times$ 2 arms (paired seeds $n = 3$ or 10). Script

work/gap_mechanism_20260916.py $\rightarrow$ gap_mechanism_20260916.txt; the data path matches §5.4's and

every cell's Δgap agrees **value by value** with the §5.1 Tier 2 table.

Tables S8–S10 $\rightarrow$ Supplementary §S8.

First, an identity: $\Delta\text{gap} = (\text{arm difference on val}) - (\text{arm difference on test})$, hence

$\Delta\text{gap} > 0 \Leftrightarrow$ **the relative arm difference on `val` is smaller than on `test`**. It compares the two splits

only and **points to no generating cause**.

An exact decomposition (an identity explaining the unstable sign — **not** a mechanism).

Because $M_{\text{val}}(\hat{e}) = V_{\text{final}} + \text{premium}_{\text{val}}$, Δgap expands into three components, ****all** computable from the

archive**:

$$\Delta\text{gap} = (\text{arm difference at the final epoch on val}) + (\text{val-side selection gain difference}) - (\text{test-side arm difference})$$

This is an **identity, not a fit**: for all 13 cells the "check" column (the sum of the three components)

equals Δgap . The three components' median magnitudes are **1.07 / 0.54 / 0.93 pp — the same order**, so

any one of them can flip Δ gap's sign, consistent with §5.1's instability.

The most informative cell is `mask20`: the baseline leads the strategy by **+15.78 pp** at the ****final**

epoch on val, **but the strategy's** val-side selection gain **is 13.10 pp**** larger; the two nearly cancel, leaving Δ gap **+1.75** ($p = 0.66$, not significant). ****This cell shows that selection alone can absorb an arm difference of order 13 pp**** — it supports "the convention changes arm-to-arm comparison"

(§5.1 Tier 1) while showing that Δ gap's sign **cannot** be used to infer any single cause.

What this section claims. Δ gap's sign **varies by corpus** is firm (§5.1). Its generating ****mechanism is**

identifiable in this archive only in part^{**}: separating "the two splits' difficulty" from "selection optimism

(one part on val, one on test)" requires **test metrics at non-selected epochs**, and the archive stores only

the test value of the **selected** checkpoint — supplying it means re-running the test evaluation (see §8.6 and §10, defect 5, for what that closed).

What remains is bounded and stated, not left as "mechanism to be explained".

7.2 Both conventions become significant only at $n \geq 6$ — measurement detail

The clean-protocol seed extension (seeds 42–51) reaches **$n = 10$** :

Table S15 → Supplementary §S8.

First-report attribution (a convention we hold to). The two Δ values here (**+1.434 / +0.507 pp**), the three-way remediation protocol itself, and the pre-registered multi-target replication (§9) belong to the companion paper and are **cited, not claimed**: both modules were transferred to it by agreement (2026-09-21) and are its first report there. What this paper reports first is ****the selection premium (§5), the 2×2 realization of that premium on a disjoint split (§8.6–§8.7), and the formalisation of the realization rate (§3.1)****. The companion manuscript is named, and its relation to this paper stated, in the cover

letter; it carries no identifier while under review, and is available to the editor and the reviewers on request. A formal citation follows once both papers are accepted.

Seed extension is therefore not optional: it upgrades the conclusion from "significant under the parametric test only" to "significant under both".

7.1 The smallest attainable permutation p at each n — moved from the article

Why the floor is $2/2^n$, and what each n licenses. For a sign-flip permutation test on n paired differences the smallest attainable two-sided p is $2/2^n$, because the observed sign pattern is the most

extreme of the 2^n equally likely relabellings; at n = 3 that is **0.25**, so such a cell has **no

resolution at all** and may be read only as "significant under the paired t-test; the non-parametric test

is unavailable at this n". The floors at the n used in this paper are: n = 3 **0.25**; n = 6 **0.031**;

n = 7 **0.0156**; n = 10 **0.0020** — the last being the same quantity as $2/2^{10} = 0.001953$, which is

how the tables print it, so the two spellings denote one number and not two. **After §5.1's escalations no

headline cell (Tables 1–2) is n = 3.** The article keeps the general form, the n = 3 case and the fact

that no headline cell is n = 3, and points here for the derivation and the remaining floors.

7.3 A cross-archive endpoint conflict on the same run (investigated — and itself a disclosure) — measurement detail

One run, t2_mask2mende_base100_s42n, is recorded with **two different checkpoints**: one row reports

the **final** epoch (67.12), the other the **best** (70.06).

Table S16 → Supplementary §S8.

We follow B and report both. Not an error in either archive: evidence that **the project had no

written convention for which endpoint constitutes "the" reported number** — §10's defect 3, the reason §3

insists that a count declare its unit.

5.1 Direction: the two tiers — measurement detail (moved from the article)

The pairing arithmetic, in full. The tested quantity is the paired difference

$d(\text{seed}) = \text{premium_baseline}(\text{seed}) - \text{premium_strategy}(\text{seed})$, within cell and seed. The right scale is the

SE of the difference (**0.058–0.120 pp**), and the four paired differences are

+0.416 / +0.526 / +0.272 / +0.449 pp, with p from 7.6×10^{-6} to 1.1×10^{-3} and all four 95 % CIs excluding zero (CI bounds [**+0.255, +0.578**], [**+0.386, +0.665**], [**+0.141, +0.403**], [**+0.309, +0.589**]; the unpaired Welch t is **5.68**).

Metric sensitivity, in full. Re-running the estimator on `metrics/mAP50(B)` keeps the sign in all four cells but moves one cell's paired difference into the noise: `smoke2sf` falls from **+0.416 pp** ($p = 1.9 \times 10^{-4}$) to **+0.063 pp** ($p = 0.54$), while `shwd2sf` **+0.847**, `a2d15` **+0.406** ($p = 0.0019$) stay significant. Script `work/t9_ap50_sensitivity_20260924.py`.

The `dota15` escalation. It entered at $n = 3$ with a parametric $p = 0.039$, below the resolution of any sign-flip test there; the pre-declared consequence was that a ten-seed paired difference failing to

be positive with $\geq 8/10$ same-sign and a CI excluding zero would remove the cell. Ten seeds (42–51) gave

+0.449 pp, $t = 7.23$, $p = 4.9 \times 10^{-5}$, 95 % CI [**+0.309, +0.589**], **10+/0–** — ******met as

stated. **Its mean is 24 % below the $n = 3$ value** ($+0.589 \rightarrow +0.449^{**}$), a fourth instance of §8.4's seed inflation inside the published 9 %–29 % band.

The two overstating cells. The plan fixed before the runs (40 runs, seeds 42–51 $\times$ 2 arms $\times$ 2 cells;

paired *and* sign-flip tests; {best, last} $\times$ {val, test}; a prespecified correction) named the strong

branch of its outcome rule: *"both remaining negative and significant with permutation $p \leq 0.002$ ".*

At $n = 10$: `p_aitovis` **–1.226 pp** (95 % CI [**–1.370, –1.081**], $t = -19.17$, $p = 1.3 \times 10^{-8}$,

0+/10–, permutation $p = 0.001953$) and `p_vistod15` **–3.315 pp**

(**[–3.676, –2.953]**, $t = -20.76$, $p = 6.5 \times 10^{-9}$, **0+/10–**, permutation $p = 0.001953$ —

$2/2^{10}$, the attainable floor at $n = 10$). Both conditions are met, including ****Holm** across the two targets******.

The raw gap and the marginal cells. The raw quantity $M_{val}(\hat{e}) - M_{test}(\hat{e})$ ranges from **-33.0 to +21.2 pp** and cannot be separated without per-epoch test evaluation. Four of 13 cells cross

the uncorrected line; the three marginal ones are $aitod20 +1.481$ ($p = 0.054$), $fire$

$+13.658$ ($p = 0.061$) and $mende20 +2.472$ ($p = 0.099$). The declared confirmatory family

is this 13-cell scan, with raw and BH-adjusted p-values side by side (**Table S8**), BH at $q = 0.05$ and

Holm across the two targets of the escalation plan; the $**0 \rightarrow 3$ move after escalation is

descriptive. **The three cells BH leaves significant are** $p_vistod15$ ($6.5 \times 10^{-9**}$), $p_aitovis$

(1.3×10^{-8}) and $shwd2sf$ (**0.0053** against a line of **0.0115**), because the two very small

p-values at the front of the ordering lift the lines behind them; $smoke2sf$ (**0.0263** against

0.0154) does not survive. Before the escalation the smallest p (**0.0053**) lay above the first line

(**0.00385**) and nothing survived.

The 13-cell family, moved out of the article (effective base: four cells). The article's

Tier 2 no longer carries the scan itself: it states the effective base (**four** cells at

$n = 10$, not thirteen) and points here. The sentence displaced from §5.1, verbatim: *"Three*

marginal cells **do not cross the line and we make no claim about them**; the 13-cell scan is

descriptive, and **no formal power analysis was run.**"* The aggregate row that the article's

Table 2 carried for the remaining cells is likewise here rather than there:

| other 9 cells | 3-10 | -2.46 to +13.66 | ≥ 0.054 | all include 0 | ****not significant**** |

— nine cells that do not cross the uncorrected line, with 95 % intervals including zero.

Why the base is four. Nine of the thirteen scanned cells sit at $n = 3$, and §7.1 of the

article shows that a sign-flip permutation test has **no resolution** there (the smallest

attainable two-sided p is $2/2^3 = 0.25$), so those cells are descriptive and carry no inference;

the four cells at $n = 10$ — the four rows the article's Table 2 retains — are the cells the

direction statement rests on. The cell-by-cell values and the raw and BH-adjusted p-values

side by side are above in this subsection and in Table S8; the $0 \rightarrow 3$ move after the escalation

is descriptive.

The selection term, cell by cell. Table 2's caption names the selection contribution proper as the difference $\Delta \text{prem_val} - \Delta \text{prem_test}$; per arm-cell that term is $\text{prem_val} - \text{prem_test}$, the winner's-curse quantity WC of §3.1 read on the archive. **Verified against the archive, and not complete:** the scan record carries prem_test for **three of the thirteen** corpora and no test curve at all for the other ten, because separating the two sides needs per-epoch test evaluation — the limitation stated above. We therefore list what is recorded and mark the rest **not recorded in the archive**; nothing is imputed.

What is recorded for **all thirteen** scanned cells is the val-side selection gain $\text{prem_val} = V_{\text{max}} - V_{\text{final}}$, per arm, at the cell's scanned n — baseline $\rightarrow$ strategy, pp: shwd2sf ($n = 10$) $+1.47 \rightarrow +0.87$; smoke2sf ($n = 10$) $+0.69 \rightarrow +0.61$; p_aitovis ($n = 3$) $+0.45 \rightarrow +0.26$; p_vistod15 ($n = 3$) $+1.11 \rightarrow +0.88$; aitod20 ($n = 3$) $+0.70 \rightarrow +0.26$; d15d15 ($n = 3$) $+2.13 \rightarrow +0.67$; dota ($n = 3$) $+1.20 \rightarrow +0.67$; fire ($n = 3$) $+10.95 \rightarrow +8.87$; mask20 ($n = 3$) $+16.07 \rightarrow +29.17$; mende20 ($n = 3$) $+4.83 \rightarrow +5.68$; p_d15toai ($n = 3$) $+0.49 \rightarrow +0.39$; p_masktomende ($n = 3$) $+2.33 \rightarrow +1.34$; vis ($n = 3$) $+0.44 \rightarrow +0.22$. Source: gap_mechanism_20260916.txt, the same record §5.6 and Table S8 use. The two cells escalated in §5.1 (p_aitovis, p_vistod15) carry $n = 10$ after the escalation, which is the four-cell base the article's Table 2 prints.

The selection term `prem\_val – prem\_test` where the archive carries it is the five realization arm-cells of §8.6, read off Table S20's printed values (**the printed `+0.736` / `+0.509` supersede the verbatim S4 dump's third decimal, `0.735` / `0.507`**): shwd2sf baseline $+1.469 - 0.736 = +0.733$ and strategy $+0.865 - 0.152 = +0.713$; smoke2sf baseline $+0.691 - 0.509 = +0.182$ and strategy $+0.605 - (-0.157) = +0.762$; aitod20 baseline ($n = 10$) $+0.766 - (-0.218) = +0.984$. The other ten scanned corpora and the aitod20 strategy arm: **not recorded** — no per-epoch test curve exists for them, so prem_test , and hence WC, cannot be formed at all. **In arm-difference form** the term is recorded for exactly two cells, from the clean-protocol batch: shwd2sf $+0.020$ and smoke2sf -0.580 , against Δgap of $+0.779$ and $+0.461$. In smoke2sf the selection term is therefore **opposite in sign to the Δgap** the article's Tier 2 prints — which is why the direction sentence rests on the four-cell base and **not** on Δgap 's sign.

The sharpest single fact. In the shwd2sf cell the arms differ on **val** by

$60.669 - 60.397 = +0.272$ pp (baseline better) and on **test** by $43.559 - 44.066 = -0.507$ pp

(baseline worse): **the sign reverses**, so reading only **val** inverts the arm ordering.

The endpoint convention, in full. On **val** the per-epoch maximum; on **test** the checkpoint the pipeline's own fitness rule kept (best.pt), read on the disjoint split. Handing a "selected checkpoint" to a split therefore always resolves to one of these two endpoints, and no other convention

is used anywhere in this paper. The four paired differences and their intervals are in Table 1; the 13-cell Δgap values and readings are in Table 2.

Sign-flip permutation tests on the four Tier-1 paired differences. Table 1 of the article reports a

paired t test per cell. Its non-parametric counterpart needs no distributional assumption and is exact

at these n : enumerating all 2^n sign flips of the per-seed paired differences

$d(\text{seed}) = \text{premium_baseline}(\text{seed}) - \text{premium_strategy}(\text{seed})$, where premium = best-epoch – final-epoch

mAP50-95 on the run's own monitored split, gives

Cell	n	$\bar{d}$	sign split	paired t p	exact permutation p	attainable floor $2/2^n$
---	---	---	---	---	---	---
smoke2sf	11	+0.416	11+/0–	1.9×10^{-4}	0.00098	0.00098
shwd2sf	11	+0.526	11+/0–	7.6×10^{-6}	0.00098	0.00098
a2d15	10	+0.272	9+/1–	1.1×10^{-3}	0.00391	0.00195
dota15	10	+0.449	10+/0–	4.9×10^{-5}	0.00195	0.00195

The permutation test agrees with the paired t in every cell. Three of the four sit exactly at the smallest two-sided p attainable at their n ; a2d15 carries one negative pair and therefore reaches $4/2^{10} = 0.00391$, one step above its own floor. Wilcoxon signed-rank returns the same p to four decimals

(0.00098 / 0.00098 / 0.00391 / 0.00195). Per-seed values, so that each p can be recomputed — the two

$n = 11$ cells carry ten seeded runs plus one unseeded run, listed last:

Cell	d(seed)
---	---
smoke2sf	+0.354, +0.247, +0.873, +0.217, +0.738, +0.394, +0.217, +0.088, +0.357, +0.482, +0.611
shwd2sf	+0.305, +0.506, +0.919, +0.466, +0.702, +0.280, +0.515, +0.756, +0.641, +0.354, +0.339
a2d15	+0.241, +0.211, +0.467, +0.560, +0.324, +0.415, –0.073, +0.216, +0.261, +0.096
dota15	+0.627, +0.821, +0.524, +0.190, +0.257, +0.369, +0.494, +0.545, +0.226, +0.437

a2d15's negative pair is its seventh listed run (seed 48); the other nine are positive. The attainable two-sided

floor for a sign-flip test on n pairs is $2/2^n$, so the $n = 11$ cells can in principle reach 0.00098

where the $n = 10$ cells cannot go below 0.00195. The dota15 per-seed values are the ones tabulated in

Table S28; the other three cells' per-seed values are recomputed from the archived per-epoch curves.

Metric sensitivity: the full mAP50 columns for Tables 1 and 4. The article reports the headline metric

mAP50-95 throughout and states one metric-sensitivity datum in the text. Both tables recomputed on

metrics/mAP50(B), from the same run sets and the same code path, follow, so that the two metrics can be

compared cell by cell rather than by assertion.

Table 1 on mAP50 (same cells and runs as the printed table):

Cell	n	premium baseline	premium strategy	paired difference	t	p	95 % CI	per-seed signs
---	---	---	---	---	---	---	---	---
smoke2sf	11	+0.972	+0.910	+0.063	0.63	0.545	[−0.160, +0.285]	6+/5−
shwd2sf	11	+1.678	+0.831	+0.847	7.00	3.7×10^{-5}	[+0.577, +1.116]	11+/0−
a2d15	10	+0.803	+0.396	+0.406	4.33	0.0019	[+0.194, +0.619]	9+/1−
dota15	10	+1.318	+0.625	+0.693	5.67	3.0×10^{-4}	[+0.417, +0.970]	9+/1−

Table 4 on mAP50 (the same five arm-cells and runs as the printed table):

Cell	Arm	n	prem_val	prem_test	realization rate
---	---	---	---	---	---
shwd2sf	baseline	10	+1.387	+1.082	78.0 %
shwd2sf	strategy	10	+0.866	+0.283	32.6 %
smoke2sf	baseline	10	+0.995	+0.839	84.3 %
smoke2sf	strategy	10	+0.840	+0.001	0.1 %
aitod20	baseline	10	+1.744	−0.369	−21.2 %

Three things the columns show, stated rather than left to the reader. (i) **The sign of the Tier-1 paired

difference is positive in all four cells on both metrics**, and all three cells other than smoke2sf keep

significance on mAP50. smoke2sf is the single cell that does not: its paired difference is +0.063 pp with

a 95 % interval [−0.160, +0.285] that includes zero, and its per-seed signs are **6+/5−** rather than

unanimous, so on this metric the weakest cell loses both significance and sign unanimity — which is why the

article attributes metric-independence to the *sign* claim and not to any per-seed claim. (ii) On Table 4's

realization rate, mAP50 gives **31.5 %** with a run-level 95 % interval [15.0 %, 48.9 %] — the three

weightings again coincide, every arm-cell being at $n = 10$ — against 23.3 % on mAP50-95, and the number of

arm-cells with a negative realized premium is **1 of 5** rather than 2 of 5. Partial realization is

therefore directionally metric-robust, while its magnitude and that count are metric-specific. (iii) Both

tables' positive controls are the printed mAP50-95 values, recomputed on the same code path: the four

Tier-1 rows reproduce Table 1 exactly at the printed precision, and the five realization cells reproduce

Table 4 to within 0.001 pp, the aggregate returning 23.26 % against the printed 23.3 %.

mAP75 is now reported on both sides: the archive still carries no per-epoch mAP75 *reading*, but the per-epoch checkpoints are in register, so the `val` half is re-measured rather than missing — on a five-epoch grid, for four of the five arm-cells.

A same-estimand ratio exists elsewhere: the genomics relative-bias literature. The article's §2.2 scopes its

"no ratio of gains" statement to a ratio of *difference* premiums on disjoint splits in detection; a ratio of the

same estimand — how much of a selected effect survives on an independent split — is however long established in

quantitative genetics, where it is written as a **relative bias** and reported as a number. Würschum & Kraft,

Cross-validation in association mapping and its relevance for the estimation of QTL parameters of complex traits,

Heredity 112(4):463–468 (2014), doi 10.1038/hdy.2013.126, define it as $1 - (pG-TS / pG-ES)$ — the complement of a

realized-to-selected ratio on a disjoint estimation/test split — and report a mean overestimation of **38 %** for the

proportion of genotypic variance explained by QTL, **30 %** averaged over four further traits, and a literature range

of **10–60 %**; Utz, Melchinger & Schön, *Genetics* 154(4):1839–1849 (2000), had already reported that in independent

validation $p(TS.ES)$ "was on average only **50 %** of $p(ES)$ ". What is not found in that literature, and what this

paper reports, is the ratio of two **difference** premiums — $(T_{best} - T_{final}) / (V_{max} - V_{final})$ — on disjoint

splits, in detection, from third-party public logs, with measured `val` noise as moderator; and the sign flip of

§8.6 (in two of the five arm-cells the realized premium is negative) has no counterpart there either.

No per-epoch mAP75 reading exists in the archive: the per-epoch

training logs and every evaluation file in this release record `mAP50` and `mAP50-95` only — a scan of

the 55 metric files behind the two tables above finds no `mAP75` column. That is why the **`val` half** of a

`mAP75` sensitivity column cannot be *read*; it does **not** block *measuring* it, and it does **not** block the **test half**. Both halves are analyses of what exists rather than new experiments: `best.pt` and `last.pt` are the released endpoints for every cell in the two tables, so re-scoring those endpoints at

`IoU 0.75` needs no further training; and the per-epoch checkpoints are in register for the four `*2sf` arm-cells on a **five-epoch grid**, so re-evaluating those checkpoints on `val` at that threshold needs none either. Both passes have been run and are reported with their scopes below: the test half (80 evaluations, 41 runs, exact positive control against the archived matrix) and the `val` half (880 evaluations, 40 runs, positive control against the training logs at a median absolute difference of **0.0161 pp**, which is what a re-measurement of a recorded training run should look like). We report the

`mAP50` columns (recomputable from the released artifacts) **and** both `mAP75` columns — the `val` one on the

five-epoch grid, labelled as such.

8.6 The realization rate — measurement detail (moved from the article)

Scope of the headline number. The **pooled 55 %** covers the four arm-cells §5.1's escalation adds

(**82 % / 104 %** on `p_aitovis`, **56 % / 29 %** on `p_vistod15`); the two batches are **not pooled**

(different runs, different protocol), so the headline is **not** re-averaged — it is defined over

§8.6's five arm-cells and Table 4 stands. The **104 %** is within noise on ~ 0.27 pp. Source:

`work/x1_aux_20260918.py`.

The theory-driven check, in full. §3.1 derives the realization rate as $1 - \text{WC} / \text{premium}$, WC being

the winner's-curse part of the selected gain. Two consequences are testable: ① the inequality is

per run, not only on average — over the **81** runs with both sides measured, WC averages

+0.463 pp ($t = 8.34$, $p = 1.8 \times 10^{-12}$) and the realized premium is smaller in **68 of 81**

runs; ② the rate should **fall as the 'val' curve gets noisier** — sorting those runs by their own

curve's noise scale $\hat{\sigma}$ gives terciles of **65.6 % / 49.8 % / 13.8 %**, Spearman $\rho = -0.466$

($p = 1.3 \times 10^{-5}$, $n = 80$). **This section's 23.3 % and §8.7's per-cell variation are therefore the

same statement at different noise levels. **What does not**** hold is the stronger form we first wrote

down — the premium scaling with the noise as $\sigma\sqrt{(2 \ln E)}$: slope **0.026 ± 0.222** ($p = 0.82$,

$n = 101$) — because the premium's *level* is set by the *val* curve's shape; §12 records the

withdrawal. Scripts: `work/theory_p4_test_20260919.py`,

`work/theory_meta_and_tests_20260919.py`. **Re-read with the arm-cell as the resampling unit,**

as the κ fit of Table S22 already is (81 runs in 9 arm-cells: the 41 three-way runs of §8.6 plus

the 40 paired runs of §5.1): WC = +0.463 pp, cluster-robust 95 % CI [+0.204, +0.721], $p =$

0.0033 (CR1 sandwich on 9 clusters; a cluster bootstrap gives [+0.253, +0.661]); the noise-

rate Spearman $\rho = -0.466$, cluster-bootstrap 95 % CI [-0.651, -0.131], $p = 0.0115$ — and at

the 9-cluster level $\rho = -0.433$, $p = 0.244$ (`work/r102_cluster_20260927.py`).

The three weightings. The three figures are ratios of means computed three ways — runs weighted

equally, unweighted arm-cell means, and arm-cells with $n \geq 5$ — and because every cell carries $n = 10$

they coincide at **23.3 %**. On the previous **41-run** sample they did not, under the same declared

`prem_val = V_max - V_final` endpoint as Table 4: the arm-cell-equal reading was **19.6 %**, the

run-weighted one **32.4 %**, and the arm-cells-with- $n \geq 5$ one **34.5 %** — a ~15 pp spread coming from a single cell holding **1** run. Each arm-cell is at $n = 10$ here, so the estimator's own interval is reported instead: 95 % CI [**8.4 %**, **37.4 %**], run-level bootstrap, $P(\text{rate} \leq 0) = \mathbf{0.002}$.

The two intervals, and the primary one. The run-level bootstrap on the same 50 runs gives 95 % CI [**8.4 %**, **37.4 %**] with $P(\text{rate} \leq 0) = \mathbf{0.002}$; the **arm-cell-equal (five-cluster) bootstrap** gives [**-17.3 %**, **53.2 %**] with $P(\text{rate} \leq 0) = \mathbf{0.118}$, and it is the interval the claim is read at, because the claim is about five arm-cells. The five-cell rates are 50 %, 18 %, 74 %, -26 % and -28 % (Table S20), i.e. a median of **17.6 %**, a span of **-28.5 % to +73.7 %** (**102.2 pp** from the printed endpoints, 102.1 pp from the unrounded cells) and a leave-one-out range of **9.77 % to 34.16 %** — dropping the shwd2sf baseline arm-cell gives 9.77 %, dropping the aitod20 baseline arm-cell gives 34.16 %. Script `work/item2_final_v2_20260925.py`; the two-cell negative set is Table 4's 2/5.

§5.1's escalation in this section's units. The four arm-cells it adds are **82 % / 104 %**

(`p_aitovis`) and **56 % / 29 %** (`p_vistod15`), pooled **55 %** against **23.3 %**.

Metric and endpoint detail. `best.pt × val` versus the per-epoch maximum in `results.csv` differs by

up to **0.083 pp**: the selection action and the readout action can differ; the earlier attribution to a `best.pt` selection rule other than mAP50-95 is **withdrawn** — in the pinned v8.4.120 `fitness = mAP50-95` on the monitored split — and the residual is recorded as **unidentified**. `last.pt`'s `val` recomputation differs from the final `results.csv` row

by **~0.02 pp**.

The closest concurrent work, moved out of the article. The article's §8.6 no longer carries this comparison; it is recorded here. [17] splits selection leakage into diversity and noise over **2,047** tabular classification datasets (the noise share is reported, the non-tabular case is undone). [15] measures checkpoint selection raising a reported score by **6.24 pp** on EEG when the candidate set grows from **5 to 80** — and, in the same table, **-1.24 pp** (95 % interval **-2.67 to 0.26**) on its other pool, so **its own sign does not reproduce across its two pools**. This paper instead uses **third-party public logs**, reports a **ratio**, on **detection**, with **measured `val` noise** as moderator. **No number left the paper when this paragraph moved**: every value above is the article's former wording.

8.6b The eleven arm-cells with both sides measured: a batch-stratified re-aggregation, no new training

What this is, and what it is not. The article's realization rate rests on **five** arm-cells, and a reviewer asked for every arm-cell whose **two sides** are both in the archive, aggregated with the

batch as the stratum. This section is that aggregation, and it is a ****re-reading of measurements this submission already reports**** — the per-epoch val curves and the best/last × test readouts of runs

that already exist. **It contains no new training, no new seed, no new evaluation and no new data.**

Nothing in the article changes because of it: the five-cell rate of §8.6, its interval and its scope stand, and this is a wider reading of the same archive, not a replacement headline.

The eleven, by batch. A cluster is one corpus × arm pair measured on **both** sides, paired inside

the run: $\text{prem_val} = \max_e V(e) - V(e_f)$ from the run's own `results.csv`, and $\text{prem_test} = T_{\text{best}} - T_{\text{final}}$

from its test readouts; a cluster's rate is $\text{prem_test} / \text{prem_val}$, and a pooled rate is the ratio of the

summed means.

*Batch 1 — the article's headline batch** (Table S20 / Table 4; three corpus pairs, $n = 10$ per cluster,

50 runs; **5 clusters**). Cluster rates: shwd2sf baseline **50.1 %**, shwd2sf strategy **17.6 %**,

smoke2sf baseline **73.7 %**, smoke2sf strategy **−26.0 %**, aitod20 baseline **−28.4 %**; median

17.6 %, span **−28.4 % to +73.7 %** (**102.1 pp** from these cells; **102.2 pp** from the printed endpoints) — the article prints the lower end as **−28.5 %**,

from Table 4's 3-decimal cells, the full-precision value being **−28.39 %** — **2/5** negative,

run-weighted **23.26 %**, cell-level 95 % CI [**−17.3 %**, **53.2 %**] ($P(\text{rate} \leq 0) = \mathbf{0.118}$). This batch is §8.6.

*Batch 2 — both arms of the two Tier-2 cells §5.1 escalated** (§5.1; $n = 10$ per cluster, 40 runs;

4 clusters). p_{aitovis} baseline **81.7 %**, p_{aitovis} strategy **103.7 %**, p_{vistod15}

baseline **55.7 %**, p_{vistod15} strategy **28.8 %** — the four values the article prints as

82 % / 104 % and **56 % / 29 %**, pooled **55 %**; median **68.7 %**, span **+28.8 % to +103.7 %**,

0/4 negative, run-weighted **55.0 %**.

*Batch 3 — the third-corpus replication** (§3.2, G4; mende20, two arms × ten seeds, 20 runs;

2 clusters). Baseline **41.5 %**, strategy **33.8 %**; median **37.6 %**, span ****+33.8 % to**

+41.5 %, **0/2 negative**, run-weighted **37.5 %**. Its winner's-curse term is the **+5.212 pp**** that

Table S33 already prints.

*All eleven, batch as the stratum. Median 41.5 %, mean 39.3 %, span -28.4 % to +103.7 %**
(132.1 pp), 2/11 negative; run-weighted pooled 36.9 % over 110 runs. A cluster bootstrap that resamples arm-cells **within** their batch (B = 20,000; seed 20260927) gives 95 % CI [19.3 %, 60.4 %] for the arm-cell-equal statistic, [28.9 %, 44.4 %] for the run-weighted one, and [26.2 %, 60.8 %] for the median — each with $P(\text{rate} \leq 0) = 0.000$.

*Difference scale, the reviewer's other request** (prem_test - prem_val, pp, clusters equally weighted): -1.367 pp, 95 % CI [-1.59, -1.15]. It is reported with a warning rather than as a co-primary, because a pp difference is not scale-free: the mean is carried by the batch with the largest

prem_val — batch 3 is -5.212 pp, against -0.675 pp (batch 1) and -0.310 pp (batch 2).

The other axes. By arm: six baseline-style ('base100') clusters, median 52.9 %, one negative, against five learning-rate-style (lr005_100ep) clusters, median 28.8 %, one negative. By budget: ten clusters ran E = 100 and one (aitod20 baseline) ran E = 30, too lopsided to be read as a budget effect. By corpus family: four clusters transfer into SFCHD (shwd2sf, smoke2sf), four come from the p_aitovis / p_vistod15 pair, one is AI-TOD on its 20 % subset, and two are mende20. By metric: the five-cluster mAP50 column is the one printed in the 5.1 measurement detail of this supplement (31.5 %, 1/5 negative, against 23.3 % and 2/5 on mAP50-95); this section does not extend that column to the other two batches, and no eleven-cluster*

mAP50 rate is stated.

What the wider reading does not license. At eleven clusters the arm-cell-equal interval no longer

includes zero, where at five it did; that is the outcome and it is stated rather than smoothed. It does

not upgrade the article's claim. ① The six added clusters **were not run for this question**: the two

Tier-2 cells were escalated *because* their val overstatement had already been flagged, and the third

corpus replicates the same protocol, so the eleven are not an exchangeable sample of corpora. ②
The strata

differ in **level** — batch 1 spans **−28.4 % to +73.7 %**, batch 2 **+28.8 % to +103.7 %** — which is why

the batch is the stratum and no single pooled rate is printed as *the* rate. ③ It is still a ****ratio** of small means: **`prem\_val` across the eleven clusters runs from +0.268 pp to +8.723 pp****, so a few

hundredths of a point in a numerator move a cluster's rate by tens of points. ④ The reviewer's own power

note applies: at this between-cluster spread, clearing zero at cluster level would take of the order of

25 clusters, and the interval above excludes zero **for these eleven archived clusters**, which is all it says.

What is paired, what is single-sided, and what has no denominator.

*Paired on both sides, on the same run: all eleven clusters**, and within a batch the two arms share

their seeds. No cluster in the eleven is assembled from different runs on its two sides.

*Single-sided or off-metric, and therefore outside the eleven.** ① The epoch-budget series (§3.2, G3; Tables S32 and

S34b) carries **`prem\_val` only**: its readout side is the disjoint carve **C3** (100 images), not the published test split, and the archive holds no per-epoch test curve for it, so no **prem_test** — and

hence no rate in this section's sense — can be formed; its own readout is recorded where it belongs

($WC = \text{prem_val} - \text{prem_C3}$ is negative in all **16** readouts, mean **−0.616 pp**, Table S32).

② The IoU-0.75 pass now carries **both sides** for the four ***2sf** arm-cells: the **val** half is

re-measured from the in-register checkpoints (880 evaluations, five-epoch grid), so the one cluster it

touches (**smoke2sf** strategy) has a denominator at mAP75 — **+1.2862 pp** against **−0.3201 pp**, a rate

of **−24.9 %** — while the **aitod20** baseline cell stays single-sided at that threshold because its nine

remaining runs' weights are not in this archive. ③ The test-curve sub-sample pass (25 % / 50 % / 100 %) re-scores both endpoints but reports

premium means over pairs, not a per-cluster rate.

*Paired and double-sided, but outside the eleven for its sample size.** The selection-split-size batch

(§3.2, G2; Table S31) is measured on both sides at **n = 5 per cluster** — four clusters, the small and

large selection splits × two arms — with rates **23.5 % / 5.1 % / 27.0 % / 15.6 %**, median **19.5 %**,

0/4 negative and run-weighted **20.5 %**; it is outside the reviewer's set because that set is the

n = 10 clusters, and adding it would make **fifteen**.

No denominator, recorded as such rather than imputed. ① The `aitod20` strategy arm*: batch 1 could have been six clusters and is five, because that arm's prem_test is **not recorded** — it has no

per-epoch test curve. ② **Ten of the thirteen scanned corpora** of §5.1: prem_test exists for three

corpora only, so for the other ten neither prem_test nor a rate can be formed. ③ The ****mAP75 val**

half: **the per-epoch `val` curve at IoU 0.75 is still not recorded**** as a reading (the 55 metric files

carry mAP50 and mAP50-95 only), but it has now been **re-measured** from the in-register checkpoints

for the four *2sf arm-cells (880 evaluations, five-epoch grid), so a mAP75 realization rate exists for

those cells (arm-cell median **+26.8 %**, run-weighted **+22.3 %**, **1 of 4** negative); what remains

without a denominator is the **`aitod20` baseline cell**, its other nine runs' weights not being in this

archive. ④ **Initialization** is not an axis

here — what varies across these replicates is the training **file order** (--shuffle-seed) — so an

across-initialization realization rate is **not recorded** in this archive.

Provenance, and the positive controls that bind this section to the printed record. The four batch-2

cluster values are recomputed from `x1_tier2_20260918/runs/` and `teval.csv`; the two batch-3 values from

`g4_mende3way_runs/` and `g4_mende3way_20260919_g4_test.csv`; the five batch-1 values from the code path

behind Table S20 (`work/item2_final_v2_20260925.py`). Three checks make the re-aggregation auditable: the

five batch-1 clusters reproduce Table S20 to **0.0019 pp**, with the aggregate returning **23.26 %** against the printed **23.3 %**; the four batch-2 clusters return **81.7 / 103.7 / 55.7 / 28.8 %** against the printed **82 / 104 / 56 / 29 %**, and their Δ gap selection terms **+0.090** and **-0.213 pp** reproduce the printed decomposition of §5.1's two cells; and the two batch-3 clusters reproduce Table S33's printed

per-run `prem_val / prem_test` pairs to **0.0003 pp**.

8.7 The per-epoch view — measurement detail (moved from the article)

κ accounting, in full, and the relation between the two estimates. Two estimates of the same quantity appear in this submission, at two grids, and they must be read together. **0.734** is the earlier fit over the **five-epoch grid** (R^2 **0.902**, $n = 161$ points); **0.708** is the corrected value over the **complete grid** (R^2 **0.937**, $n = 765$ (seed, epoch) points over 41 runs), and the completed sweep moves the estimate by **-0.026** with overlapping intervals. **The interval printed in

the article is the run-clustered (sandwich) one, taken on runs as the clusters: 95 % CI

[0.671, 0.745] (**sandwich SE 0.018**); **a run-level bootstrap** cross-check gives**

[0.672, 0.744], i.e. the same interval to the printed precision, and the within-run estimator,

which removes run-level differences entirely, gives **0.698**. The five-epoch grid carries its own

run-clustered interval, **[0.683, 0.784]** (sandwich SE 0.025, bootstrap [0.685, 0.784]), and that

interval belongs to the **0.734** fit, not to the value the article prints. Scripts:

`work/x2_kappa_clustered_20260918.py` (which reproduces the 0.734 / 0.902 / $n = 161$ fit exactly before

adding the interval) and `work/x3b_perepoch_corrected_20260918.py` (the complete-grid refit; its

printed lines give the sandwich interval, the bootstrap cross-check and the within-run estimator).

Per-arm κ and the five-epoch comparison. baseline $\kappa = 0.600$ (95 % CI [0.581, 0.619], R^2 0.946) against **strategy** $\kappa = 0.767$ ([0.735, 0.800], R^2 0.951); the per-arm fits and the resampling are in **Supplementary S11**.

The per-checkpoint sign detail. On the complete curve smoke2sf's Δ_{gap} is negative at epochs 5/10/15 and positive from epoch 20 on; the four-epoch sample had suggested uniformity. The per-checkpoint values are in **Supplementary S11** and Tables S21–S23.

The three-layer cross-check. The same checkpoints were evaluated twice independently and agree on

all 202 overlapping (run, epoch) points to 0.0000 pp; the full sweep covers 806 rows (820 minus the 14 the single-run aitod20 cell does not have, its 30 epochs contributing 6 checkpoints where the others contribute 20).

10 Our own defects — item-by-item detail (moved from the article)

Defect 1. The clean-protocol results document cites `_r10_final_stats.txt`, and the file later identified as the "real log" is `_r10_stats_0403.txt`; **neither** contains any of that document's headline values (6 probes each, 0/6 hits), so those numbers have **no surviving stdout ledger**.

We do **not** point at a replacement file — renaming would only manufacture a new false citation — and

the p-values remain recomputable from the local `pvalue_table_local.py`.

Defect 2. The dota15 cell of `selection_premium.py` was not reproducible because the root directory

was a stale snapshot and the matcher used `prefix + '_s'` while the actual run names carry no seed

suffix; after the fix it reproduces M3's +1.12 / +0.95 / +0.93 value by value. The "3 seeds" wording

note it once triggered was **retracted on 2026-09-21**: the third run's seed is **s42** (the campaign's fixed shuffle seed, recoverable from its configuration), so what is missing is the directory-name suffix, not the seed.

Defect 4. The two headline scripts originally depended on a rented pod (`import paramiko` plus a

plaintext password); selection_premium_local_20260916.txt and
r10_pvalue_table_20260916_local.md now

read the local archive, and the pod versions are listed only as cross-checks.

Defect 5. For a long time no single script computed the σ decomposition: the three components came

from three unrelated analyses. **Two of the three are now recomputed by one script**

(work/sigma_decomposition_20260926.py) — the data-order component and the initialization component are

recomputed from the archived per-run values and each is asserted against the number printed in Table S19.

The augmentation component is the exception and is named as such: the archive records its SD without

its per-run values, so the script checks only that the record carries it, and its output says

cited, not recomputed rather than printing a constant as though it had been computed. Still open, and

still disclosed: there is **no frozen effect-size rule file**, and ****no script generates**

benchmark_split_audit.md (**hand-written, 28 judgments as literals; its counts**** are in fact recomputed, by work/recompute_split_units_20260916.py).

Defect 6. The local archive holds no copy of the three-way carve directories and no copy of 6 of the

7 corpora. The 2026-09-16 GPU run repaired part of this: the per-epoch curves of the 41 three-way runs,

sio_b_results.csv and the **2 × 2 evaluation matrix (164 evaluations)** are now local with a

SHA-256 list in xeval_20260916/, so §8's readings and §5.6's decomposition are locally recomputable;

what remains non-recomputable is retraining, which needs the corpora themselves.

Defect 9. Our first per-epoch report claimed curve-wide stability from a four-point sample, and reading the complete curve overturned it for one of the two cells (§8.7): smoke2sf's Δ gap is negative

at epochs 5/10/15 and positive from epoch 20 on, so the claim is **narrowed** rather than deleted, with

the retracted sentence quoted and the sampled grid's apparent uniformity named as partly an artefact of

where it sampled.

Defect 10. *Moved here from §10, item 10, verbatim, so that the article can state the disclosure and the bound without carrying the event counts.*

Memory pressure silently changes the training path: under GPU memory pressure Ultralytics switches implementation without changing the recorded configuration — (a) CUDA out of memory with batch=32. Reducing to batch=16 (a **configuration-level** change), or (b) OutOfMemoryError in TaskAlignedAssigner, using CPU (same batch, but label assignment re-runs on CPU, where a boundary argmax can differ and the trajectory forks). Case (a) is **invisible in the standard artifacts** — args.yaml still records batch: 32, and the only fingerprint is the log’s per-iteration denominator (32 → 0/9 / 0/41; 16 → 0/18 / 0/81).

Disposition. Disclosed, not retracted; we do not call the affected batches “clean”. Configuration-level hits: 2 places — §3.1’s E = 50 clean batch, 1 run (Δ premium +0.308 pp against the same-machine re-run), and §5.1 Table 2’s p_vistod15 cell, 3 runs — so the whole clean four-budget series was re-run on one machine. Assigner fallback occurs in **every** batch, 1–4 times per run ($\approx$ 1 per 1000 iterations). **We bound it rather than estimate it:** on the clean E = 50 batch 7 of 10 same-config pairs are bit-identical, so $\leq$ 0.05 pp applies *there* and is **not extrapolated**; in the p_vistod15 and aitod20 cells the treated runs’ A→B differences (Δ premium $\leq$ 0.35 pp) fall **inside the spread of the zero-event controls** (up to +0.704 pp), so at this scale the effect is **not separable from machine/run variation** and we give a bound, not a point estimate; the one clear case is the configuration-level run (Δ Vmax +0.769 pp). This sits against the paper’s own realization-rate 95 % CI [8.4 %, 37.4 %].

The five realization arm-cells: what each one pairs, and whether the corpora are public

All five are public corpora, and the corpus pair is in the cell name. A cell name reads source→target: shwd2sf and smoke2sf transfer from one public corpus to SFCHD, and aitod20 is the **AI-TOD** corpus on its own 20 %-budget subset. None of the five is an internal corpus, so no cell's result rests on data a reader cannot obtain; what is *ours* is the reproduction and the three-way split built on top of them, and the per-cell values are in Table S20.

Cell	Source → target	Corpus status
---	---	---
shwd2sf (baseline, strategy)	SHWD → SFCHD	both public
smoke2sf (baseline, strategy)	public smoke corpus → SFCHD	both public
aitod20 (baseline)	AI-TOD → AI-TOD (20 % budget)	public ; the subset is ours

What this does not settle. That the corpora are public does not make these cells a sample of

the field: they are the cells this design measured, and §8.6's rate is scoped to them.

What the 19 benchmarks are, and what is recorded about them

****Every row carries an evidence class and a confidence level; the inclusion rule is not recorded, and we do not reconstruct one after**

the fact. **The marking table was** established 2026-09-16, **and each row carries its** evidence class, **its** confidence level **and the** document version****** the split configuration was read

from — so a reader can trace any row back to the release archive or the dataset's own paper it came

from, and the four subtotal equalities are recomputed mechanically rather than hand-summed.

What is *not* in the archive is a stated rule for why these 19 and not others. Two things can still be said without inventing one. First, the rows are the benchmarks for which this archive holds the split configuration read from **primary documentation** rather than from a secondary description — that is a property of the rows, and it is visible in them. Second, ****no sampling frame of papers was built****, which is why the article says it does not sample papers per benchmark

and makes no prevalence claim: the counts describe **these** benchmarks, and a stated-frame sample remains future work.

Read as: a census of the configurations this archive could read, not a sample of the field.

Two later measurement batches: what they add, and the bounds they carry

****Both are evaluation-side; neither changes a headline number, and both are stated with the scope they**

have.****** They answer two questions a reader is entitled to ask about §8.6: is the paired premium an

artefact of *which* sub-sample of the held-out split was scored, and is it an artefact of the metric?

The paired premium does not move with the size of the test sub-sample. For the four Table 4 families

that are not the aitod20 baseline, the same two endpoints were re-scored on random sub-samples of the

held-out split at 25 %, 50 % and 100 % of its images, ten independent draws per size (21 curve points,

1,974 completed evaluations; the driver is `work/b51_eval.py`, the summary `work/b51_summary.py`).

The paired best – last premium keeps its sign and its order of magnitude at every size: **+0.2096 pp** at

25 % (470 pairs), **+0.2538 pp** at 50 % (470 pairs), **+0.2325 pp** at the full split (47 pairs), with the run-to-run spread (SD 0.49–0.59 pp) an order of magnitude larger than the shift between sizes.
**What this

does not licence**: it shows the premium is not an artefact of one particular sub-sample, not that these

cells are a sample of the field; and the 100 % row rests on fewer pairs than the other two because

126 of the 2,100 scheduled evaluations could not be run — the weights of three runs

(`r10_aitod20_base30_3way_s43n`, ..., `s44n`, ..., `s45n`) are not held in this archive, so their 42 curve points

each are missing rather than reported.

The sign instability is not an artefact of the metric. The same best/last endpoints were re-scored

at **IoU 0.75** (82 evaluations, 41 runs, no failures; drivers `work/b55_m75.py` and

`work/b55_val.py`, controls `work/b55_m75_control.py` and `work/b55v_control.py`, pairing

`work/b55v_pair.py`, same-sample metric comparison `work/b55v_metric_compare_20260928.py`). The positive

control is exact: the mAP50-95 values from this pass reproduce

the archived matrix to **0.0000 pp on all 82 run-checkpoint pairs**. The paired premium is **+0.2933 pp** at

mAP50-95, **+0.2982 pp** at mAP75 and **+0.5201 pp** at mAP50, and the arm-cell that is negative at

mAP50-95 (smoke2sf, strategy arm, **–0.1571 pp**) is **still negative at mAP75 (–0.3201 pp**, positive

in 2 of its 10 runs). **Both halves are now measured on the same runs, and the rate is metric-specific.** The val half

was obtained by re-evaluating the in-register checkpoints of the same four arm-cells on their own val

split at IoU 0.75 — **880 evaluations** (20 five-epoch-grid checkpoints, `last.pt` and `best.pt` for each of

40 runs), no failures — with a positive control against the training logs: the re-measured mAP50-95 at

those checkpoints reproduces `results.csv` at a **median absolute difference of 0.0161 pp (800/800)**

within 0.2 pp, none above 1 pp; that the training log's epoch column is 1-based against the checkpoint

filenames is itself part of the record, and comparing the two at the same index instead produces a spurious

median of 0.6462 pp). Per arm-cell the mAP75 pair (`prem_val`, `prem_test`) and its rate read ****+1.8828 /**

+1.0072 = +53.5 % (shwd2sf baseline), +1.8043 / +0.0017 = +0.1 % (shwd2sf strategy), +0.7205 /

+0.5792 = +80.4 % (smoke2sf baseline) and +1.2862 / -0.3201 = -24.9 % (smoke2sf strategy), against**

+50.1 % / +17.6 % / +73.7 % / -26.0 % on mAP50-95 over the same 40 runs: arm-cell median **+26.8 %**

against **+33.8 %**, run-weighted **+22.3 %** against **+34.2 %**, and **1 of 4** cells negative in both.

Three cells move by at most a few points; the one that moves materially is the shwd2sf strategy arm

(**+17.6 % → +0.1 %**), where the realized premium essentially vanishes at the stricter threshold.

What this does not licence: the `val` side is a **five-epoch grid** whose peak is not the per-epoch argmax — the grid peak falls short of the per-epoch peak by **0.13 to 0.33 pp** on mAP50-95 for these

cells — so these `prem_val` values are grid values and are labelled as such; the aitod20 baseline cell is

not covered on the `val` side, its other nine runs' weights not being in this archive; and a per-run rate

stays unreadable, the median per-run ratio being **+0.016** against a mean of **+0.023**. Both batches cover

the runs whose weights this archive holds, and the per-run files are held with this revision's measurement

batch rather than inside the archived reproduction package.

Read as: two robustness checks on §8.6's readings, scoped to the cells and the sub-samples they cover —

one on the carve, one on the metric — not as further evidence about the field.

An independent second coding of the 19-row marking table

What was done, and what was not. The $19 \times 4 = 76$ judgements printed in S1 were re-coded independently.

Three coders -- three language models from distinct vendors -- each worked blind: they received the

published coding rules and, per row, the evidence pointer the audit recorded, and nothing else: no marks,

no confidences, and no access to this supplement or to the audit's records. They worked from the recorded

pointers rather than from the primary documents, which this archive does not hold, and that limitation is

part of the result rather than incidental to it. A second pass then repeated the exercise under rules

clarified in the four places the first pass's coders had independently flagged: whether release answers

structural independence or local evaluability; the reference frame for reported; corpora with only

train and val; and same-family versions such as VOC 2007/2012. The clarification is recorded verbatim

and was written AFTER the first pass, so the two passes are reported separately rather than pooled.

What the two passes show. Against the printed coding, agreement is fair to moderate and does not

improve when the rules are clarified: per-coder kappa is 0.272 / 0.321 / 0.437 in the first pass and

0.209 / 0.217 / 0.348 in the second (mean 0.343 -> 0.258), while the coders' agreement WITH EACH OTHER

rises over the same interval (pairwise kappa 0.559-0.672 -> 0.671-0.723). Excluding the cells a coder

marked unknown moves the first pass to 0.323-0.570. Because the mapping from the table's marker

vocabulary to yes/no/unknown is itself a convention, the whole computation is repeated under the alternative reading of the two vocabulary items that are genuinely ambiguous; that moves the mean by

about ± 0.06 (0.316-0.405) and changes nothing qualitative.

What it means, and what it does not. The residual disagreement is therefore not mainly rule wording:

two coders who agree with each other at kappa ~ 0.7 agree with the printed coding at only kappa ~ 0.2 -0.3,

and clarifying the rules widens that gap rather than closing it. The most plausible reading is an evidence-channel difference rather than a disagreement about the rules: the printed coding was made with

the primary documents in hand, and the second coders had the recorded pointers. Two limits belong with

those numbers and are stated here rather than left implicit: the three coders are language models, so

their errors may be correlated and three of them are not three independent raters; and the weakest unit

is reported (per-unit kappa ~ 0.10), which is the unit the headline count rests on, because which split

the literature's number comes from is not decidable from any single file.

The consequence for this paper's claims. The counts printed in section 6.1 -- 12/19, 80 % and the

rest -- are the judgements of a single coder, and independent re-coding from the recorded evidence

reproduces them only at fair-to-moderate agreement. They are therefore presented as a convenience audit

by one coder with the reproducibility bounded here, not as a measured rate, which is what the abstract

already says. The per-coder, per-unit and per-pass values, both vocabulary readings, and the coders' own

notes on the ambiguities they found are held with this revision's measurement batch.

Reproduction map: headline number → entry point

Every headline number that a script produces maps to one entry below — a producing script, or an explicit

note that no script produces it — so a reader can either recompute a number or see at once that we

cannot offer a script for it. Three headline counts come from the audit's own records rather than from a produced table — §4.1's 6 of 14, §5.5's 20-log sample and §8.3's 9 %–29 % bound — and are marked at their point of use rather than mapped here. Paths are inside the archived reproduction package (see the Data

availability statement in the article);

02_release_data/ holds the derived tables the article's tables are generated from.

Headline number	Derived table (package)	Producing script
---	---	---
23.3 % realization rate and its three (coincident) weightings (§8.6, Table 4)	release_selection-transfer_perseed.csv, release_shape-noise-by-budget.csv	work/item2_final_v2_20260925.py
κ and its clustered error (§8.7)	x3b_tableS22_20260918.csv, x3b_tableS23_20260918.csv	work/x2_kappa_clustered_20260918.py, work/x3b_perepoch_corrected_20260918.py
shape share of the premium (§3.1 corollary)	release_shape-noise-by-budget.csv	work/prop6_shape_noise_20260920.py
transfer slope for the shape/noise decomposition	release_selection-transfer_shape-share.csv	work/prop6_transfer_20260920.py
Tier 1 / Tier 2 paired and permutation statistics (§5.1)	release_X1_tier2_best-last_summary.csv	work/xeval_analyze_20260916.py
registered replication and its saturated control (§9)	release_T1_registered-replication_summary.csv, release_T2_saturated-control_summary.csv	work/preregister_E_ext_20260919.py
G1' / G1' / G2 / G3 / G4 verdicts (§3.2)	release_G1p_AB-half-level-match.csv, release_G1pp_thirds_levels.csv, release_G2_selection-split-size_summary.csv, release_G3_epoch-budget_summary.csv, release_G3ext_E50-400_perrun.csv,	work/g1p_verdict_20260919.py and the verdict scripts named in §S31–S33

	release_G4_mende20_3way_perseed.csv	
the 19 × 4 counting-unit table (§6.1, Table 3)	recomputed in-package	work/recompute_split_units_20260916.py
the framework-default check (§6.5)	line-by-line evidence in §S3	work/x10_alias_audit_20260918.py
the three variance components (§8.4, Table S19)	recorded in Table S19 only	work/sigma_decomposition_20260926.py — recomputes the data-order and initialization components from the archived per-run values; cites the augmentation component, whose record has no per-run values (Defect 5)
the 289 audit anchors (Declarations)	in-package	work/audit_anchors_fupaper.py

****The three variance components of §8.4 are the one entry in this map whose script covers two of its**

three components rather than all three.** The data-order and initialization components are recomputed

from the archived per-run values; the augmentation component's record holds an SD and no per-run values,

so the script cites it and labels that line cited, not recomputed. §10 defect 5 and the item-by-item

Defect 5 below state the same thing, and we would rather name the gap than write a script that only

re-prints a recorded constant.

The pre-declared criteria are files, not recollection. work/preregister_E_ext_20260919.py writes

the G-batch criteria *before* the runs; work/g1p_verdict_20260919.py applies them *after*. Each verdict in §3.2 is therefore recomputable from a criterion that existed in advance of its data.

The audit's coding rules are stated, not implied. The 19-row marking table of §S1 is normative: each

row records its evidence class and the reading it was coded under, and the four subtotal equalities are

recomputed mechanically by work/recompute_split_units_20260916.py rather than hand-summed. ****A second**

rater would be needed for a true inter-rater statistic and we do not claim one**; what is recomputable

is the *mechanical* consistency of the table with its own markings, and that recomputation ships with

the table (§S1, "Recomputation results").

Row-level evidence pointers for the 19-row marking table of §S1. The marking table is coded by

hand, by one coder, from the configuration audit: no script assigns a mark, and a second rater would be

needed for a true inter-rater statistic, which this paper does not claim. What *is* mechanical is the step from the marks to the numbers — and that step, and only that step, is scripted.

Where the mechanical recomputation starts and stops. Every count and percentage in §S1's "Recomputation results", the "among decidable rows" denominators, the four levels of "same directory"

and the cross-check against the audit's published counts are recomputed from the 19 rows' own marks by

work/recompute_split_units_20260916.py, with no percentage hard-coded; the text of §S1's table and of

its recomputation block is that script's own output, and the same script is the entry point this table is given in the reproduction map above. **It covers that step and nothing upstream of it:** it consumes

the 19 rows as literals, and it does not choose a benchmark, open a release package, read a dataset paper, or check a document version. Two caveats follow. Running it is not a read-only reproduction —

it writes its two artifacts to an absolute path and overwrites them in place, so a verifier should read

the archived artifacts rather than re-run the generator. And its own upstream is a hand-written audit:

the marks it recomputes *from* were not derived by any script either.

What each row's evidence pointer resolves to, and what is not on record. For every row, the

recomputation artifact `split_units_recompute_20260916.txt` carries that row's own entry — its evidence

class, its self-assessed confidence and its derivation or correction note — and

`work/recompute_split_units_20260916.py` carries the same text as that row's literal. Where a row's

evidence class names a mirror configuration or a comparison script, only the name is on record.
**No row

records the concrete source it was coded from**: not the release package or its version, not the dataset

paper, and not the document version or access date of the page, README or PDF its class names;
and **no

row records where in the audit its (A) marks stand**, the one exception being row 19, whose note names

two audit lines. Those fields are marked *not archived* below rather than reconstructed. Read as a

chain, one row of §S1 is walkable end to end — the English row, that row's entry in

`split_units_recompute_20260916.txt`, the same row's literal in the script, and the audit's mark — but

the last link is the audit's assertion, not a recorded path, and that gap is disclosed rather than papered over.

How to read the list below. Each line carries that row's evidence class and self-assessed confidence

as §S1's table prints them. *marks* gives the row's four evidence marks in the table's column order

(release / protocol / yolo_dist / reported). *archive* names that row's entry in

`split_units_recompute_20260916.txt`, whose text is also that row's literal in

`work/recompute_split_units_20260916.py`. A field recorded as *not archived* is not recorded anywhere

in the archive, and is not reconstructed here from what the row says.

1. COCO — evidence class: Official counts + mirror yaml; confidence: high; marks: A/A/A/A;
archive: row 1; concrete source (release package, dataset paper, document version, access date)
and upstream audit line numbers: not archived.

2. PASCAL VOC — evidence class: mirror VOC.yaml (both keys cited per row); confidence: high; marks: A/A/A/A; archive: row 2; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
3. Objects365 — evidence class: mirror yaml + Ultralytics docs; confidence: high; marks: A/A/A/D; archive: row 3; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
4. Open Images v7 — evidence class: Official facts page + mirror yaml; confidence: high; marks: A/A/A/D; archive: row 4; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
5. DOTA v1.0 — evidence class: Official download page + mirror DOTAv1.yaml; confidence: high; marks: A/A/A/A; archive: row 5; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
6. DOTA v2.0 — evidence class: Official/mirror documentation; confidence: high; marks: A/A/A/D; archive: row 6; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
7. VisDrone-DET — evidence class: mirror + converter; confidence: high; marks: A/A/A/A; archive: row 7; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
8. AI-TOD — evidence class: Official README (contains two mutually contradictory release statements); confidence: low; marks: A/A/A/?; archive: row 8; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
9. UAVDT — evidence class: Official paper + two mirrors (the audit calls these unverified beyond these two); confidence: medium; marks: A/A/A/A; archive: row 9; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
10. xView — evidence class: mirror xView.yaml; confidence: low; marks: A/A/A/A; archive: row 10; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
11. DIOR — evidence class: Official description (no counting) + mirror DIOR-R yaml; confidence: low; marks: A/A/A/?; archive: row 11; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
12. NWPU VHR-10 — evidence class: Official TorchGeo (the split means positive/negative); confidence: high; marks: A/A/A/A; archive: row 12; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.
13. SHWD — evidence class: Official README + mirror Reflective_vests.yaml; confidence: medium; marks: A/A/A/A; archive: row 13; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.

14. SFCHD — evidence class: Official first-party release (machine comparison `_ev_sfchd_check2.py`); confidence: highest; marks: A/A/A/A; archive: row 14; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.

15. MAFA — evidence class: Official CVPR PDF (extracted with `pypdf`); confidence: high; marks: A/A/?/A; archive: row 15; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.

16. Mendeley face-mask — evidence class: Official data record + mirror generator; confidence: lowest; marks: A/A/D/?; archive: row 16; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.

17. WIDER FACE — evidence class: Official site (GT not released, verified word by word) + mirror; confidence: high; marks: A/A/A/A; archive: row 17; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.

18. CrowdHuman — evidence class: Official paper + mirror; confidence: high; marks: A/A/A/A; archive: row 18; concrete source (release package, dataset paper, document version, access date) and upstream audit line numbers: not archived.

19. D-Fire (FireSmoke) — evidence class: Official README; confidence: medium; marks: A/A/A/?; archive: row 19; concrete source (release package, dataset paper, document version, access date): not archived; upstream audit line numbers: A:52, A:59 (§S1, "Corrections already applied").

Scope of reproducibility: two things that are not the same

What	Status	Why
---	---	---
Evaluation-side recomputation — every premium, rate, κ , interval and verdict in the article	reproducible from the package	needs the per-epoch matrices and the released tables, all shipped
Retraining — re-running training to regenerate the checkpoints those numbers were computed on	not reproducible from the package	the archive holds no copy of the three-way carve directories and no copy of 6 of the 7 corpora (§10 defect 6)

This distinction is why the article claims **evaluation validity** rather than full reproducibility:

every number we report is recomputable from what is shipped, while regenerating the checkpoints behind

them would require corpora we do not redistribute.

Why a second pass, and what it is not. The single-rater disclosure above is unchanged,

and a second human rater is still what a true inter-rater statistic would need. What has

been added is a **second coding pass, model-assisted and non-blind** (run 2026-09-27): it

re-coded all 19 rows $\times$ 4 units of the §S1 table under the published rules, after the rules were first separated from the printed conclusions into a standalone manual (work/coding_manual_19x4_20260927.md). Because §S1 prints rules and marks in one place, the pass could not be blinded — the printed coding was visible throughout, and the statistics below are a consistency check against a visible target, not a blind inter-rater result. **The second pass is model-assisted, not a second human rater, and it does not replace the single-rater disclosure printed with the table.** Its per-cell marks are archived with the audit ground for each row (work/second_coding_20260927.csv), and every statistic below is recomputed mechanically by a script that reads the printed table and that file (work/kappa_second_coding_20260927.py; machine output work/kappa_computed_20260927.txt, readable summary work/kappa_summary_20260927.md). The

pass coded against the archived configuration audit — the evidence base one step upstream of the printed table, whose English rows mirror a Chinese authoritative file.

Agreement with the printed coding. On the decision scale the printed counts use (non-independent / clean / no-test-key / unknown / not-applicable), the per-unit κ is **0.716** for release (95 % interval 0.422–1.000), **1.00** for protocol (all 19 cells agree; the interval degenerates at 1.00), **0.929** for yolo_dist (0.792–1.000) and **0.784** for reported (0.501–1.000), and the overall κ over the 76 judgments is **0.874** (95 % interval 0.778–0.971). Treating the four printed unknown cells as a third category or dropping the eight cells where either coding is unknown moves the overall κ between **0.874** and **0.922** (0.835–1.000); on the literal marks, exact agreement is 68/76. The intervals are wide at $n = 19$, and the two lower units sit in the "substantial" rather than "almost perfect" band — the disagreement concentrates exactly where the manual is under-specified, which is itself a finding.

The eight cells that differ (of 76), each with its ground.

- Rows 1–2 (COCO, PASCAL VOC) · release · $n_a \rightarrow$ independent_test: the printed n_a

has no stated applicability rule in any published material, and the audit's row facts yield a decidable mark — a manual gap, not an evidential conflict.

- Row 18 (CrowdHuman) · release · test_gated → independent_test, and protocol · alias → test_gated: the audit documents the same structure here as in row 17 (WIDER FACE), which the printed table itself marks independent_test / test_gated; under any fixed rule one of the two structurally identical rows must move. The release cell is decision-bearing; the protocol cell is not (both marks are non-independent).

- Row 9 (UAVDT) · yolo_dist · unknown → no_test_key: the audit's §1 verdict for the row is explicit ("F-B ... empty test:"), so "the audit takes no position" — what unknown means — does not hold; the manual had to fix the precedence of a §1 verdict over a §4 generality caveat.

- Rows 11 (DIOR) and 16 (Mendeley face-mask) · reported · unknown → alias: the audit's §3b header and its 14/19 roll-up both enumerate these rows, so the audit does take a position; the manual had to fix the precedence of the enumerations over the rows' own caveats (missing primary counts; a naming ambiguity; a "treat as hypothesis" self-rating that the audit's count does not honor).

- Row 16 · yolo_dist · alias → train_val_alias: the verified generator writes train == val — the F-C shape, for which the mark vocabulary reserves train_val_alias; the printed alias follows the audit's own "F-A generated (train == val)" label, which conflates F-A with F-C. Both marks are non-independent, so no count moves.

****The n_a rule the release subtotal rests on, and why that subtotal is not mechanically recomputable from the marking table. The printed `release` subtotal — 10/19 = 52.6 %**, with unknown 0 — is computed under a rule the table itself does not state: **a row marked n_a(A) enters neither the non-independent numerator nor the undecidable column**. Rows 1–2 (COCO, PASCAL VOC)**

carry n_a(A) in the release column because that layer supplies **no independent test artifact at

all* — `n\_a` means not applicable, which is not the same thing as undecidable* — so they stay in the

19-row denominator while counting nowhere else. Of the nineteen rows, **ten carry a non-independent

mark, two carry n_a, and seven are independent**; read the other way, i.e. with n_a treated as

non-independent, the same column yields $12/19 = 63.2\%$. **The subtotal therefore cannot be

recomputed from the marking table alone*: the step `12 - 2 = 10` is* the rule, not a recomputation,

and it is stated here so that a reader re-deriving the four subtotals can reproduce them. Under this

one stated rule and no other, the four subtotals are mechanically recomputed

(work/recompute_split_units_20260916.py); the n_a mark's lack of a stated applicability rule is

also one of the eight cells the second coding moved, recorded above.

What this does to the printed counts. Two of the eight cells are count-bearing. Under

the printed resolution of rows 11 and 16 the reported layer reads $12/19 = 63\%$

($12/15 = 80\%$ of decidable rows); under the second pass's reading of the audit's own

enumeration, the same rules yield $14/19 = 74\%$ ($14/17 = 82\%$, two rows unknown). The

printed coding is not modified here — the two readings bound the claim, and the

qualitative conclusion (a majority of these benchmarks report a split that is not

independent of checkpoint selection) is stable across the range. Where the reported-layer

count is quoted, the strength of the claim should follow the bound rather than the point:

*"12–14 of 19 benchmarks (63–74 %; 80–82 % of the decidable rows), where the range spans

the printed coding's two undecidable rows and the second pass's reading of the audit's own

enumeration; the printed coding is the conservative end."*

What this does not settle. The marking is not double-rated: one human coder's marks

remain the printed coding, and this pass is reliability evidence, not a replacement.

What the archive holds of the three-way carve, and what the two n = 11 cells pair

The carve manifest is archived; the other carve directories and 6 of the 7 corpora are not.
The

article's §10 defect 6 states the recomputability boundary, and this is the precise form of it. The

archive holds the **manifest** of the SFCHD three-way carve: the dataset YAML
(sfchd20_3way.yaml,

naming train/20_percent/images, carve_sfchd20/images and test/images), the three
filename lists

(train_20_percent_images.txt **2413**, carve_sfchd20_images.txt **1000**, test_images.txt

6033), their **SHA-256 list** (split_defs_20260916/SHA256SUMS.txt) and the recomputed

zero pairwise intersections (VERIFY_INTERSECTIONS.md: all three intersections **0**, recomputed

locally without a pod and without a GPU by work/verify_split_defs_20260916.py). The
transfer set also

holds the image and label files of that one carve directory (rest/root__carve.tgz, **1000** images

with their labels). **What is not held:** any of the other carve directories, and 6 of the 7 corpora —

so **retraining remains non-recomputable**, exactly as the article says. **What the manifest does
buy**

is the layer the protocol's premise rests on: the zero-intersection check by filename is now a local
recomputation from archived lists rather than a pod-side report.

What the `n = 11` Tier-1 cells pair. Table 1's smoke2sf and shwd2sf rows pair **11** runs each,
and the file that table cites lists the paired units explicitly

(verify_two_critiques_20260916.txt): [42, 43, 44, 45, 46, 47, 48, 49, 50, 51,
unseeded], the same

line for both cells. **The eleventh unit is therefore a run whose directory carries no seed suffix**
—

the archive records it as unseeded — while the two $n = 10$ cells of the same file (a2d15) are fully

seeded. This is stated rather than left implicit because " $n = 11$ " can otherwise be read as eleven

independent training seeds, which is not what the design varies: what varies across these
replicates is

the **training file order** (--shuffle-seed), and one of the eleven units records no seed suffix at
all, so its data order is not identified by name. Runs without a seed suffix exist in both archives
for

these cells (the pod-side run lists carry entries such as shwd2sf_base100 and shwd2sf_lr005_100ep

beside the _sNNn ones), so this is a property of the run set, not of one machine's naming.

Method detail relocated from the article

Why these paragraphs are here. The article is capped at 35 double-spaced pages with no margin left, and eight external reviews of the previous version said the same thing: the prose read as an *audit trail* rather than as an article, because round-by-round bookkeeping and repair history occupied the argument's place. These paragraphs are moved here **verbatim and unedited** — every value, every date, every file name and every disclosure sentence is unchanged. The article keeps a one-sentence bridge and a pointer at each former location, and keeps every disclosure that bears on a claim.

This paper and the budgeted fine-tuning study it re-measures — the companion paper — share ****process**

protocols**: a multi-model review checklist and a local/cloud workflow discipline (scripted remote

execution, all-or-nothing patching, audit anchors, a local artifact pointer behind every number).

****Both are project-internal protocols, not publications, and not a**

methodological contribution of this paper. **This paper displays only their outputs****: the ten self-reported defects in §10 and the

artifact pointer behind every number. So does the **audit-anchor system** (a mechanically checked set of anchors that must all

pass on every rebuild of this manuscript): shared tooling, no contribution claimed. What is asserted is only that the same protocols were used; **no originality is claimed** for them.

What is ours, and what is not. Of the statements that follow, the raw gap's definition [20], the impossibility result for endpoint-pair estimators [22] and the family-dependence procedure [23] are **machinery rather than results of ours**, and the identity is **adopted** from the selection-bias literature; **the corollary alone carries measurements of this paper.**

The same script on the **pod root** (347 runs, $n = 10$) gives a smoke2sf premium of **+0.786 / +0.390**;

on the **local archive** (569 runs, $n = 11$) **+0.799 / +0.382** — agreeing in sign and magnitude,

not value. **The main text uses the local version**; the pod version is recorded as a cross-check.

The refuted one

compared the premium difference against a single-run component (0.515 pp) rather than the SE of an

arm-mean difference; the other asked for the raw $M_{val} - M_{test}$, whose difficulty term cannot be

separated from selection optimism.

The audit classified the FireSmoke mirror family row as "mixed" solely from a config with

`train == val`, but **another line of the same document records that this config is based on SHWD**; all

three FireSmoke/D-Fire copies have **three separated keys**, so D-Fire moves to the clean group.

A direct measurement of seed inflation — reproduced inside this setting: adding seeds lowers the mean by **9 %–29 %**, each figure taken **against that cell's own $n = 6$ mean** and not against the pooled

headline effect, the **seed-budget discipline** measured rather than assumed (the setting in which [52] locates the first-order component, §8.4). **What these replicates are, and what the bound is a bound on.** The per-run randomness the ten runs install is a **deterministic permutation of the training file order** (`--shuffle-seed`, the number in the run's own name), **not** the framework's seed argument: the Ultralytics data pipeline does not draw from `args.seed`, and on two of these corpora different `--seed` values produced **identical** epoch-by-epoch replays (**Supplementary S12**, section 9.3). "Seed inflation" here therefore means **data-order replication**: the 9 %–29 % it bounds is the **file-order** component of the reported mean, and is **not** an estimate of the variance between independent training seeds. That best-of-K selection inflates a reported score is **not new** — §8.6 positions the closest concurrent work, which measures the **dose–response** over 2,047 datasets — so **what is measured here is the magnitude under a protocol that selects cells for significance first and expands seeds afterwards**; design and the check against recomputed means: **Supplementary S12**, Table S18. **The title is deliberately the weaker, sample-scoped claim:** with three initializations the interval is wide, so the components are **consistent with, not established by**, these three initializations.

The three variance components, and the excluded one is the smaller in this sample: the single-run data-order component is **0.515 pp**, against initialization and seed components of the same order — the **same ranking** as the shuffle-order result of [52] and consistent with the run-to-run variance literature [26]; we therefore **reproduce the ordering inside this setting** rather than discover it. What was held fixed, and why this is a sample statement, is in **Supplementary S12** and Table S19.

Mixing two pipelines' `val/test` conventions would confound the split convention with the framework. The reasoning and the counterexample are in **Supplementary S12**. **Pairing is not neutrality, however:** the paired design used here covers the **budget and data** axes only, and **not** the optimiser's.

One run is recorded with two different endpoints in two archives — "final" (67.12) in one and "best" (70.06) in the other — i.e. **which checkpoint is recorded was never specified**; §10 discloses it and the two readings' consequences are worked out in **Supplementary S12** and Table S16:

Recomputation status. The readings were originally **not locally recomputable** (the archive lacks

6 of 7 corpora; **defect 5 in §10**); a GPU **2 × 2 evaluation** on **41** three-way runs of

YOLOv12n [54] × {best, last} × {val, test} makes the readings locally recomputable and **retraining is not**. That batch reproduces the archived best × test values to within **0.0047 pp** (median **0.0021 pp**) — the positive control for this paper's endpoint rule (**Supplementary S5**); the remaining record is in **Supplementary S12**.

Five limitations: ① the two cells escalated in §5.1 reach $n = 10$ with **local** runs and test-side evaluations, but the remaining 9 cells' runs are **not local**; ② `prem_test` uses only `**best` and `last**`, not a per-epoch curve; ③ `last.pt`'s val recomputation differs from the final `results.csv` row by ~ 0.02 pp; ④ this section does **not** touch §6's 19-benchmark census; ⑤ the aggregate is a **ratio of small means** — `prem_val` across the five arm-cells spans **+0.605 to +1.469 pp** (Table S20) — so a change of a few hundredths of a point in the numerator moves the rate by tens of points, and the aggregate summarizes a wide spread rather than a typical run's outcome.

The **41 original** runs behind §8.6 **retain intermediate checkpoints** (`save_period=5: epoch0,5,...,95, 20 files; 3,854 epoch*.pt`), evaluated twice independently with exact agreement (**Supplementary S6, Supplementary S11**) — a free reproducibility check of the endpoint rule.

Three limitations: ① only the two headline cells reach $n = 10$; `aitod20` has a single run (30 epochs, 6 checkpoints where the others contribute **20** — hence 806, not 820); ② `**closed`: every retained checkpoint is now evaluated*, *and the stability** reading is narrower than the sampled grid suggested;

③ κ is relative to each run's own final epoch and is **not** causal. The 806-row evaluation is archived

as `xeval_perepoch_20260918/matrix_perepoch.csv` with a SHA-256 list.

- `**Four` disclosed uncertainties are limits on what our numbers can be read as claiming, and we keep

them visible rather than absorbing them into a headline: **the 19-row audit is single-rater**** (no

second coder, so no inter-rater statistic exists and we do not claim one); the third-party log sample is

a **convenience sample** found by repository search, so it corroborates that the component exists elsewhere and is not a rate; the GPU memory-pressure effect is **bounded, not estimated** (≤ 0.05 pp

where same-config pairs can be compared, inside the zero-event-control spread elsewhere, §10 defect 10);

and the §8.3 replicates vary the **training file order**, not independent training seeds.

None of the four has an interval that could be folded into §8.6's rate — the first two have no numeric bound at all without a second coder — so they are stated here as scope, not as error bars.

*(The Chinese draft numbers these rows 1–3, 3, 4–7 with a duplicate "3"; renumbered here without

changing content.)* The

four rule gaps it had to fix — n_a's applicability, which unit absorbs ground-truth

withholding, the two precedence questions (§1 verdict vs §4 caveat; §3 enumerations vs

row-level caveats), and version anchoring for multi-version rows — are exactly the points

a human second rater would have to rule on before any mark could be mechanical, and they

are recorded as findings in the manual and the summary, not smoothed over.

Table S35 — the estimand and scope map

Table S35. *Added by this revision; not moved from the article, and therefore not listed in the §S8

index (which indexes tables moved from the article).* One row per quantity the article measures, with the

assumption that makes it identifiable and the scope that bounds it. The last column collects the

"we do not claim" statements that the article's prose carries at §5.1, §5.4, §8.6 and §12, so that each

quantity's boundary is stated once, beside the quantity rather than scattered across sections.

Quantity	Estimator	What has to hold	What it identifies	What we do not claim
---	---	---	---	---
selection premium	best – last within one run , mAP50-95 pp, on the monitored split;	that best is the argmax of the same curve the reported split scores (the	the checkpoint- selection term of run-to-run variation under the published	not that the convention <i>understates</i> the effect; not a single

	paired across arms	definitional part)	convention	bias direction across corpora
Δgap	the paired difference of the premium between the two arms, per corpus	that the two arms see equal split difficulty — an assumption we state, not one this design demonstrates	a split-dependent arm contrast	not an isolated selection effect; the selection contribution proper is $\Delta\text{prem_val} - \Delta\text{prem_test}$
realization rate	$\text{prem_test} / \text{prem_val}$, run-weighted, over the five arm-cells at $n = 10$	that the readout split is disjoint from the selection set and untouched by any selection rule	the share of the val -selected premium that survives on a disjoint split, for these five cells	not a constant of the convention; not significant read at the level of the claim; the aggregate is a ratio of small means
κ (transfer slope)	through-origin slope of ΔT on ΔV over a run's epochs; run-clustered inference with runs as clusters	that "epoch" is the unit of the curve and that runs are the clusters	how much of a run's movement on val appears on test	κ is relative to each run's own final epoch and is not causal; κ is not constant across arms
peak-agreement predictor	whether the val and test peaks fall on the same 5-epoch checkpoint	the shared 5-epoch grid, and that agreement is an observable, not an intervention	a partial predictor of Δgap 's sign, via a lower median Δgap when the peaks agree	not an explanation of the sign; the agreement group's mean is carried by one run; not refined beyond the 5-epoch grid

Table S36 — the analysis-status ledger

Table S36. *Added by this revision; not moved from the article, and therefore not listed in the §S8

index. *Every quantity and procedural step this paper reports, classed as pre-registered** (frozen before the runs existed), **pre-planned** (fixed in advance of the analysis but not registered) or **post-hoc** (decided after the runs were seen). The class each item carries is the one the article's own

prose asserts; the ledger collects them so that no post-hoc reading can be mistaken for a registered one. **Any p-value this paper quotes for a post-hoc row is uncorrected and descriptive:** the only multiplicity correction here is the Benjamini–Hochberg step over the declared 13-cell family, and only the pre-planned rows belong to that family.

Item	Status	What fixes the class
---	---	---
the registered multi-target replication	pre-registered	its own frozen registration stamp, with zero runs existing at freeze time; reported by the companion
the four-budget epoch series' criteria C1–C3	pre-registered	the criteria are recorded as fixed before the runs
the $n = 10$ escalation criterion for the Tier-2 cells	pre-planned	the criterion was fixed in advance of the escalations
the endpoint convention best – last	pre-planned	frozen in writing before the re-analysis
the 13-cell family and its Benjamini–Hochberg correction	pre-planned	the family was declared before the correction was computed; its

		thirteen members and their raw and adjusted p-values are Table S8 , and the two cells escalated to n = 10 after that scan are named in Table 2's note
the $\sigma\sqrt{2 \ln E}$ scaling	post-hoc	tested, then withdrawn — the form does not survive the archive
the complete-curve κ and its run-clustered interval	post-hoc	the sweep was completed after the five-epoch reading
the per-epoch Δ gap sign reading	post-hoc	read off the completed curve, correcting a four-point sample
the second coding of the 19-row table	post-hoc	reliability evidence gathered after the printed coding; it does not replace it
the peak-agreement predictor	post-hoc	a reading of our own 40 runs
the mAP50 / mAP75 re-scoring batches	post-hoc	evaluation-side robustness checks run for this revision
plateau selection	post-hoc	offered as a candidate remedy, not as a validated rule

Detail relocated from the article's single-rater audit

Two clauses. Which reported number the COCO row judges, and this table's first-version correction

(yolo_dist is **4/19**, not 2/19).

Algorithm self-check. On the **literal two-key subset** the table **reproduces the audit's 2/19**, so the

difference comes from **the unit**, not a transcription error; the full yolo_dist column and the four subtotal equalities are in **Supplementary S1**.

One row belongs to a different unit. Mendeley face-mask's train == val comes from a **generator**

(a VOC→YOLO script writing two keys from one images_dir), not a distribution package: not counted in the

2/19, described separately.

A number we deliberately do not use: the audit's "63–79 %" bound (15/19) has **no enumeration**.

"Same directory" has four levels: the levels, the rows each admits, and the resulting shift in the headline number are in **Supplementary S12** and Table S12.

The literal-search check, displaced from the article's section 2.1 (#31, 2026-09-28)

Non-collision, checked by literal search. Three exact queries return **0** hits — "same directory" train val test yaml, configuration-level leakage "data.yaml" and "split aliasing" benchmark — so the configuration-level phrasing is not in the indexed literature; the channel is **metadata-only**, which establishes that these strings are unused, **not that the idea is new**. The comparison that carries the claim is the **estimand** contrast with [14], which stays in the article: [14] reports a raw val-minus-test gap that mixes the two splits' difficulty, whereas this paper's paired difference cancels it and still shows a residual shift.